THE PREMIUM NOTEBOOK SERIES

{ Java }

CORE JAVA ZERO TO HERO

Handwritten Interview Notes

From your very first Java program to confident senior and architect-level interview answers - with friendly explanations, Java code, diagrams and practical follow-ups.

180 QUESTIONS | 180 FOLLOW-UPS | 24 CODING LABS

Java 8 foundations · Java 17 and 21 features · 30-day study route
Designed for beginners, experienced engineers and interview revision.

START HERE

How to read these notes

Use this notebook like an interview rehearsal, not as a collection of paragraphs to memorize. Read the question, cover the answer, then explain it using plain English.

01	02	03
Explain the rule	Walk through code	Answer the follow-up

EVERY INTERVIEW PAGE INCLUDES

A 20-second answer for the basic idea, **a worked example** drawn from the attached guide, **a visual explanation**, a **common pitfall** and an **answered follow-up**.

VERSION RULES

Foundation code uses Java 8 unless a newer feature is labelled. Records, virtual threads, switch pattern matching and modern collection methods need their stated Java releases. The 24 coding labs use Java 8 APIs and compile with `javac --release 8` on a supporting full JDK.

Evidence boundary. The extracted 24 Java lab solutions compiled and passed 24 focused smoke checks in this edition. Those checks are not a proof of every input or every thread schedule. Example fragments on question pages are not all standalone programs.

HOW TO GIVE A STRONG ANSWER

Definition → small example → why it works → limitation → follow-up. For concurrency, mention what is shared, which operation must be atomic, who owns cleanup, and how a waiting task ends.

THE 21 CHAPTERS

Table of contents

01 Java foundations and execution

02 Object-oriented design

03 Equality, copying and immutability

04 Strings, numbers and arrays

05 Exceptions and resource ownership

06 Generics, nested classes and annotations

07 Lists, sets, queues and ordering

08 HashMap and map design

09 Concurrent collections

10 Lambdas, functional interfaces and Optional

11 Streams fundamentals

12 Stream recipes and date-time

13 Threads and cooperative cancellation

14 Java Memory Model

15 Locks and coordination primitives

16 Executors, futures and overload

17 CompletableFuture and asynchronous composition

18 JVM, class loading and garbage collection

19 I/O, serialization, reflection and security

20 Modern Java for interviews

21 Senior interview scenarios

Follow with: 24 complete coding labs · 30-day roadmap · Interview revision checklist. Each numbered question begins on a fresh study page.

MODULE 01 / 21

01

Java foundations and execution

The JDK develops and runs Java programs; the runtime supplies libraries and the JVM; the JVM executes bytecode.

8 interview questions · clear examples · follow-up answers

Q001 What is the difference between the JDK, JVM and runtime?

20-SECOND ANSWER

The JDK develops and runs Java programs; the runtime supplies libraries and the JVM; the JVM executes bytecode.

WHY IT WORKS · EASY EXPLANATION

javac is a development tool supplied by the JDK. The java launcher starts the JVM, which uses the runtime libraries. Traditionally, JDK contains JRE, and JRE contains JVM. Modern runtime packaging can differ.

JAVA EXAMPLE / SCENARIO

```
javac Hello.java  // Compile source to Hello.class
   java Hello        // Launch the JVM and run bytecode
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
javac Hello.java // Compile source to...	The JDK develops and runs Java programs; the runtime...	javac is a development tool supplied by the JDK.

Watch out: Bytecode portability still depends on a compatible Java version, libraries and native dependencies.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you package a smaller runtime for one application?

Use jdeps to identify required modules, then jlink to build a runtime image with only those modules. Test the image with the real application and its native dependencies.

Original source: Core Java Zero to Hero, page 11 · Q001 · Interview study edition

Q002 Why can Java run on different operating systems?

20-SECOND ANSWER

Java runs on different operating systems because compatible JVMs execute the same platform-independent bytecode.

WHY IT WORKS · EASY EXPLANATION

Compile ordinary Java code once. Each operating system supplies its own JVM, which executes the bytecode using instructions for that machine. The bytecode is portable; the JVM executable is platform-specific.

JAVA EXAMPLE / SCENARIO

```
Hello.java -> javac -> Hello.class
   Windows JVM -> executes Hello.class
   Linux JVM   -> executes Hello.class
   macOS JVM   -> executes Hello.class
```

DRAW THE IDEA

START / CASE
Hello.java -> javac -> Hello.class

JAVA RULE
Java runs on different operating systems because...

RESULT
Compile ordinary Java code once.

Watch out: Java portability does not mean the JVM executable itself is portable.

INTERVIEWER FOLLOW-UP - ANSWERED

What does javac --release 8 check?

It checks Java 8 language syntax, available standard APIs, and the generated class-file version together. It helps prevent accidental calls to newer JDK APIs.

Original source: Core Java Zero to Hero, page 12 · Q002 · Interview study edition

Q003 Is Java pass by value or pass by reference?

20-SECOND ANSWER

Java is always pass by value. For an object, the copied value is its reference.

WHY IT WORKS · EASY EXPLANATION

p and names initially refer to the same list, so add changes that list. Assigning a new list to p changes only the method's local reference. It does not replace names. The same rule means a primitive argument's reassignment cannot change the caller's variable.

JAVA EXAMPLE / SCENARIO

```
static void change(List<String> p) {
       p.add("Kiran");           // Change shared object
       p = new ArrayList<>();   // Change local reference
   }
   List<String> names = new ArrayList<>();
   change(names);
   System.out.println(names);   // [Kiran]
```

DRAW THE IDEA

START / CASE
static void change(List<String> p) {

JAVA RULE
Java is always pass by value.

RESULT
p and names initially refer to the same list, so add...

Watch out: Seeing a mutation in the caller does not imply pass by reference.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can a method not swap two caller variables of type Integer?

The method receives copies of the two Integer references. Swapping those local copies cannot reassign the caller's variables; Integer values are also immutable.

Original source: Core Java Zero to Hero, page 12 · Q003 · Interview study edition

Q004 How do fields and local variables get initial values?

20-SECOND ANSWER

Fields and array elements receive defaults automatically. Local variables must be assigned before reading.

WHY IT WORKS · EASY EXPLANATION

Numeric fields default to zero, boolean to false, char to '\u0000', and references to null. These defaults apply to static fields too. A local int[] a = new int[3] has assigned its reference; its three elements still default to zero.

JAVA EXAMPLE / SCENARIO

```
class Example {
       int count;           // Default: 0
       boolean active;      // Default: false
       String name;         // Default: null
       void show() {
           int x;
           // System.out.println(x); // Compile-time error
           x = 10;
           System.out.println(x);   // 10
       }
   }
```

DRAW THE IDEA

START / CASE
class Example {

JAVA RULE
Fields and array elements receive defaults automatically.

RESULT
Numeric fields default to zero, boolean to false, char...

Watch out: A null field is initialized, but dereferencing it can still fail.

INTERVIEWER FOLLOW-UP - ANSWERED

What are the default values of char and boolean?

A char field or array element defaults to the null character ('\\u0000'); a boolean defaults to false. Unassigned local variables have no usable default.

Original source: Core Java Zero to Hero, page 13 · Q004 · Interview study edition

Q005 What happens when an integer calculation overflows?

20-SECOND ANSWER

Ordinary integer overflow wraps around; assigning the result to long afterward does not repair it.

WHY IT WORKS · EASY EXPLANATION

The first multiplication uses int operands, so it overflows before assignment. The L suffix makes the second multiplication use long. multiplyExact checks overflow and rejects it instead of silently wrapping.

JAVA EXAMPLE / SCENARIO

```
long wrong = 50_000 * 50_000;
   long right = 50_000L * 50_000;
   System.out.println(wrong); // -1794967296
   System.out.println(right); // 2500000000
   Math.multiplyExact(50_000, 50_000); // ArithmeticException
```

DRAW THE IDEA

START / CASE
long wrong = 50_000 * 50_000;

JAVA RULE
Ordinary integer overflow wraps around; assigning the...

RESULT
The first multiplication uses int operands, so it...

Watch out: Assigning the result to long cannot undo an earlier int overflow.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you safely compare two integers without subtraction?

Use Integer.compare(a, b), or compare with a < b and a > b. Subtraction can overflow and reverse the intended order.

Original source: Core Java Zero to Hero, page 13 · Q005 · Interview study edition

Q006 What is autoboxing and where can it fail?

20-SECOND ANSWER

Boxing converts primitives to wrappers; unboxing converts wrappers to primitives and fails for null.

WHY IT WORKS · EASY EXPLANATION

Unboxing behaves like calling intValue on an Integer, so null cannot be unboxed. equals compares wrapper values. Comparing wrapper references with == is unreliable for value equality because identity can depend on caching.

JAVA EXAMPLE / SCENARIO

```
Integer boxed = 10;           // Boxing
   int value = boxed;            // Unboxing
   Integer missing = null;
   // int x = missing;           // NullPointerException
   Integer a = 1000, b = 1000;
   System.out.println(a.equals(b)); // true
```

DRAW THE IDEA

START / CASE
Integer boxed = 10; // Boxing

JAVA RULE
Boxing converts primitives to wrappers; unboxing...

RESULT
Unboxing behaves like calling intValue on an Integer,...

Watch out: Integer == Integer usually compares references, unless an operand causes unboxing.

INTERVIEWER FOLLOW-UP - ANSWERED

Why might IntStream be preferable to Stream<Integer>?

IntStream operates on primitive int values, avoiding much boxing/unboxing overhead and supporting primitive operations such as sum() and average(). Measure before claiming a speedup.

Original source: Core Java Zero to Hero, page 14 · Q006 · Interview study edition

Q007 How do widening, narrowing and numeric promotion differ?

20-SECOND ANSWER

Widening usually needs no cast; narrowing needs a cast and can lose information; arithmetic promotes small integers to int.

WHY IT WORKS · EASY EXPLANATION

A byte cannot hold 130, so narrowing retains only the byte-sized representation. b + 1 has type int. Compound assignment includes a conversion back to byte, which explains why += compiles.

JAVA EXAMPLE / SCENARIO

```
int n = 130;
   byte small = (byte) n;
   System.out.println(small); // -126
   byte b = 1;
   b += 1;                    // Compiles: implicit cast
   // b = b + 1;              // Compile-time error
```

DRAW THE IDEA

START / CASE
int n = 130;

JAVA RULE
Widening usually needs no cast; narrowing needs a cast...

RESULT
A byte cannot hold 130, so narrowing retains only the...

Watch out: A compound assignment includes an implicit conversion back to the left-hand type.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is a char a UTF-16 code unit rather than every possible Unicode character?

Java's char stores one 16-bit UTF-16 code unit. Supplementary code points require surrogate pairs, so one Unicode code point may occupy two chars.

Original source: Core Java Zero to Hero, page 14 · Q007 · Interview study edition

Q008 How do short-circuit operators and evaluation order work?

20-SECOND ANSWER

&& and || short-circuit; boolean & and | always evaluate both operands. Java evaluates operands left to right.

WHY IT WORKS · EASY EXPLANATION

With &&, the false null check prevents isEmpty from running. With &, both sides run, so the call dereferences null. Similarly, true || expensiveCall skips the call.

JAVA EXAMPLE / SCENARIO

```
String name = null;
   boolean ok = name != null && !name.isEmpty();
   System.out.println(ok); // false
   // name != null & !name.isEmpty(); // NullPointerException
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
String name = null;	&& and || short-circuit; boolean & and | always...	With &&, the false null check prevents isEmpty from...

Watch out: Replacing && with & can introduce a null dereference.

INTERVIEWER FOLLOW-UP - ANSWERED

What does false && expensiveCall() evaluate?

Only false is evaluated; expensiveCall() is not called because && short-circuits once the left operand is false.

Original source: Core Java Zero to Hero, page 15 · Q008 · Interview study edition

MODULE 02 / 21

02

Object-oriented design

Abstract classes share instance state and construction rules; interfaces define capabilities and can supply default behavior.

8 interview questions · clear examples · follow-up answers

Q009 Why do abstract classes still matter after Java 8 default methods?

20-SECOND ANSWER

Abstract classes share instance state and construction rules; interfaces define capabilities and can supply default behavior.

WHY IT WORKS · EASY EXPLANATION

Each Exporter object has its own destination, initialized through the constructor. Printable supplies a reusable method but cannot own per-object instance fields or a constructor. Java 8 default methods therefore do not replace abstract classes.

JAVA EXAMPLE / SCENARIO

```
abstract class Exporter {
       protected final String destination;
       Exporter(String d) { destination = d; }
       abstract void export();
   }
   interface Printable {
       default void print() { System.out.println("Ready"); }
   }
```

DRAW THE IDEA



Watch out: An interface default method does not replace every use of shared state.

INTERVIEWER FOLLOW-UP - ANSWERED

When would composition be safer than extending AbstractExporter?

Composition is safer when an object needs an exporter without really being an exporter, or when you want to swap implementations, mock dependencies, and avoid inherited state coupling.

Original source: Core Java Zero to Hero, page 16 · Q009 · Interview study edition

Q010 Can you instantiate an abstract class?

20-SECOND ANSWER

You cannot instantiate an abstract class directly. You can instantiate a concrete or anonymous subclass.

WHY IT WORKS · EASY EXPLANATION

The anonymous class provides the missing implementation. The created object's class is that concrete anonymous subclass. Task is just the reference type. The abstract superclass constructor still participates in construction.

JAVA EXAMPLE / SCENARIO

```
abstract class Task {
       abstract void run();
   }
   // Task t = new Task(); // Compile-time error
   Task t = new Task() {
       @Override void run() { System.out.println("Done"); }
   };
   t.run(); // Done
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
abstract class Task {	You cannot instantiate an abstract class directly.	The anonymous class provides the missing implementation.

Watch out: An abstract class may have constructors and may contain no abstract methods.

INTERVIEWER FOLLOW-UP - ANSWERED

Can an abstract class have a final concrete method?

Yes. An abstract class may define a final concrete method. Subclasses inherit that method but cannot override it.

Original source: Core Java Zero to Hero, page 17 · Q010 · Interview study edition

Q011 How are overloading and overriding resolved?

20-SECOND ANSWER

Overloading selects a signature at compile time; overriding selects an instance implementation at runtime.

WHY IT WORKS · EASY EXPLANATION

a is declared Animal, so show(Animal) is selected. The receiver object is a Dog, so the overridden speak runs. The two decisions happen at different stages.

JAVA EXAMPLE / SCENARIO

```
class Animal { String speak() { return "Animal"; } }
   class Dog extends Animal {
       @Override String speak() { return "Dog"; }
   }
   static void show(Animal a) { System.out.println("Animal overload"); }
   static void show(Dog d) { System.out.println("Dog overload"); }
   Animal a = new Dog();
   show(a);                        // Animal overload
   System.out.println(a.speak());  // Dog
```

DRAW THE IDEA

START / CASE
class Animal { String speak() { return...

JAVA RULE
Overloading selects a signature at compile time;...

RESULT
a is declared Animal, so show(Animal) is selected.

Watch out: A private method is not overridden by a same-named subclass method.

INTERVIEWER FOLLOW-UP - ANSWERED

How does overload resolution treat a null argument?

The compiler chooses the most specific applicable overload; null matches reference types, not primitives. If unrelated overloads are equally specific, the call is ambiguous.

Original source: Core Java Zero to Hero, page 17 · Q011 · Interview study edition

Q012 Can static methods and fields be polymorphic?

20-SECOND ANSWER

Static methods are hidden, and fields are selected by the reference's declared type. Neither uses instance-method dispatch.

WHY IT WORKS · EASY EXPLANATION

The declared Parent type selects the field and static method even though the object is Child. Use class names for static calls to make this selection explicit.

JAVA EXAMPLE / SCENARIO

```
class Parent {
       int value = 1;
       static String who() { return "Parent"; }
   }
   class Child extends Parent {
       int value = 2;
       static String who() { return "Child"; }
   }
   Parent p = new Child();
   System.out.println(p.value); // 1
   System.out.println(p.who()); // Parent; prefer Parent.who()
```

DRAW THE IDEA

START / CASE
class Parent {

JAVA RULE
Static methods are hidden, and fields are selected by...

RESULT
The declared Parent type selects the field and static...

Watch out: Calling a static method through an instance can misleadingly resemble virtual dispatch.

INTERVIEWER FOLLOW-UP - ANSWERED

Can a subclass change an instance method into a static method?

No. A subclass cannot hide an inherited instance method with a static method having the same overriding signature; the compiler rejects it.

Original source: Core Java Zero to Hero, page 18 · Q012 · Interview study edition

Q013 What is the order of construction and initialization?

20-SECOND ANSWER

Fields first receive defaults. In the Java 8-21 construction model, superclass construction precedes subclass initializers and constructor body.

WHY IT WORKS · EASY EXPLANATION

Base's constructor dispatches to Child.show before Child's name initializer runs. The field still contains its default null. Avoid calling overridable methods from constructors. Class-level static initialization is a separate stage.

JAVA EXAMPLE / SCENARIO

```
class Base {
       Base() { show(); }
       void show() {}
   }
   class Child extends Base {
       String name = "ready";
       @Override void show() { System.out.println(name); }
   }
   new Child(); // Prints null
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
class Base {	Fields first receive defaults.	Base's constructor dispatches to Child.show before...

Watch out: Avoid publishing this or invoking overridable behavior during construction.

INTERVIEWER FOLLOW-UP - ANSWERED

When can reading a compile-time constant avoid class initialization?

Reading a static final compile-time constant such as a literal int or String may use an inlined value, without triggering initialization of its declaring class.

Original source: Core Java Zero to Hero, page 18 · Q013 · Interview study edition

Q014 How are conflicting interface default methods resolved?

20-SECOND ANSWER

An implementing class must resolve matching defaults inherited from unrelated interfaces.

WHY IT WORKS · EASY EXPLANATION

Without Report's override, the compiler cannot choose between A and B. A.super.label explicitly delegates to A's default. A concrete inherited class implementation normally wins over an interface default.

JAVA EXAMPLE / SCENARIO

```
interface A { default String label() { return "A"; } }
   interface B { default String label() { return "B"; } }
   class Report implements A, B {
       @Override public String label() {
           return A.super.label();
       }
   }
   System.out.println(new Report().label()); // A
```

DRAW THE IDEA

START / CASE
interface A { default String label() { return...

JAVA RULE
An implementing class must resolve matching defaults...

RESULT
Without Report's override, the compiler cannot choose...

Watch out: Interfaces do not inherit static methods as instance behavior.

INTERVIEWER FOLLOW-UP - ANSWERED

Can an interface provide a default implementation of equals(Object)?

No. An interface default method cannot override a public method of Object such as equals(Object), hashCode(), or toString(). Implement it in a class instead.

Original source: Core Java Zero to Hero, page 19 · Q014 · Interview study edition

Q015 When should composition replace inheritance?

20-SECOND ANSWER

Use inheritance for a valid is-a relationship; use composition to reuse or replace behavior through a collaborator.

WHY IT WORKS · EASY EXPLANATION

RetryService has a Sender instead of claiming to be a particular HTTP sender. A test can supply a fake Sender. Retry logic can evolve independently of the transport implementation.

JAVA EXAMPLE / SCENARIO

```
interface Sender { void send(String text); }
   class RetryService {
       private final Sender sender;
       RetryService(Sender s) { sender = s; }
       void send(String text) { sender.send(text); }
   }
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
interface Sender { void send(String text); }	Use inheritance for a valid is-a relationship; use...	RetryService has a Sender instead of claiming to be a...

Watch out: Sharing a few methods is insufficient evidence of a valid subtype relationship.

INTERVIEWER FOLLOW-UP - ANSWERED

What parent guarantee would a subclass that rejects add() violate?

It violates the Liskov Substitution Principle if the parent contract promises add() support. A caller relying on that promise cannot safely substitute the subclass.

Original source: Core Java Zero to Hero, page 19 · Q015 · Interview study edition

Q016 How do SOLID principles translate into Java interview answers?

20-SECOND ANSWER

SOLID keeps responsibilities focused, extensions isolated, subtypes substitutable, interfaces small, and policy dependent on abstractions.

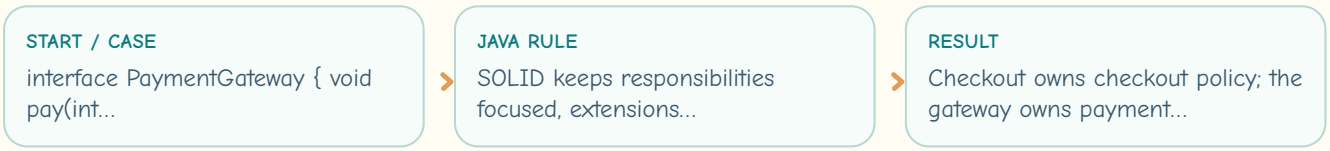
WHY IT WORKS · EASY EXPLANATION

Checkout owns checkout policy; the gateway owns payment behavior (SRP). Add a UPI implementation without rewriting Checkout (OCP). Every gateway must honor pay's contract (LSP). The interface exposes only needed operations (ISP). Checkout depends on the interface (DIP).

JAVA EXAMPLE / SCENARIO

```
interface PaymentGateway { void pay(int rupees); }
   class Checkout {
       private final PaymentGateway gateway;
       Checkout(PaymentGateway g) { gateway = g; }
       void complete() { gateway.pay(500); }
   }
```

DRAW THE IDEA



Watch out: More interfaces and more classes do not automatically improve the design.

INTERVIEWER FOLLOW-UP - ANSWERED

Which principle fails if a gateway sometimes silently discards payments?

Primarily Liskov Substitution: the implementation breaks the PaymentGateway contract. It can also violate the application's expected error-reporting policy.

Original source: Core Java Zero to Hero, page 20 · Q016 · Interview study edition

MODULE 03 / 21

03

Equality, copying and immutability

Equal objects must have equal hash codes. Equal hash codes do not guarantee equal objects.

8 interview questions · clear examples · follow-up answers

Q017 What are the equals and hashCode contracts?

20-SECOND ANSWER

Equal objects must have equal hash codes. Equal hash codes do not guarantee equal objects.

WHY IT WORKS · EASY EXPLANATION

Both methods use the same stable value field. A HashMap can therefore find a mapping with an equivalent new Id. equals must also be reflexive, symmetric, transitive, consistent, and false for null.

JAVA EXAMPLE / SCENARIO

```
final class Id {
       private final int value;
       Id(int v) { value = v; }
       @Override public boolean equals(Object o) {
           return o instanceof Id && value == ((Id) o).value;
       }
       @Override public int hashCode() { return Integer.hashCode(value); }
   }
   System.out.println(new Id(7).equals(new Id(7))); // true
```

DRAW THE IDEA

START / CASE
final class Id {

JAVA RULE
Equal objects must have equal hash codes.

RESULT
Both methods use the same stable value field.

Watch out: A different hash code proves inequality; an equal hash code does not prove equality.

INTERVIEWER FOLLOW-UP - ANSWERED

What breaks if you override equals but inherit Object.hashCode()?

Two objects that compare equal could have different identity-based hash codes. HashMap and HashSet may then fail to treat them as the same key, violating the contract.

Original source: Core Java Zero to Hero, page 21 · Q017 · Interview study edition

Q018 Why are mutable keys dangerous in a HashMap?

20-SECOND ANSWER

Changing a map key's equality or hash fields after insertion can make its entry unreachable through normal lookup.

WHY IT WORKS · EASY EXPLANATION

Insertion uses the old hash. Lookup uses the new hash, which may search a different bucket or fail the stored-hash check. A final reference would still allow key.id to change. Use an immutable identity key.

JAVA EXAMPLE / SCENARIO

```
// MutableKey uses id in equals and hashCode.
   MutableKey key = new MutableKey(1);
   Map<MutableKey, String> map = new HashMap<>();
   map.put(key, "Sai");
   key.id = 2;
   System.out.println(map.get(key)); // Can be null
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
MutableKey key = new MutableKey(1);	Changing a map key's equality or hash fields after...	Insertion uses the old hash.

Watch out: final Customer key prevents reassignment, not mutation of key.id.

INTERVIEWER FOLLOW-UP - ANSWERED

Would a TreeMap key mutation also cause problems?

Yes. Changing a comparison field after insertion can corrupt the tree's expected ordering; lookups, removal and uniqueness checks may behave incorrectly. Use stable keys.

Original source: Core Java Zero to Hero, page 22 · Q018 · Interview study edition

Q019 How do you make a Java class immutable?

20-SECOND ANSWER

An immutable class prevents observable state changes and controls mutable inputs, outputs, and subclassing.

WHY IT WORKS · EASY EXPLANATION

Changing input after construction does not change values because the constructor copied the structure. Callers cannot modify the returned list. String elements are immutable; mutable elements would need their own copy policy.

JAVA EXAMPLE / SCENARIO

```
final class Names {
       private final List<String> values;
       Names(List<String> input) {
           values = Collections.unmodifiableList(new ArrayList<>(input));
       }
       List<String> values() { return values; }
   }
```

DRAW THE IDEA

START / CASE
final class Names {

JAVA RULE
An immutable class prevents observable state changes...

RESULT
Changing input after construction does not change...

Watch out: An unmodifiable list of mutable elements does not make the entire object graph immutable.

INTERVIEWER FOLLOW-UP - ANSWERED

What would you do when an immutable object contains Date?

Make defensive copies on input and output, such as new Date(input.getTime()). Prefer immutable java.time types when the API permits.

Original source: Core Java Zero to Hero, page 22 · Q019 · Interview study edition

Q020 How do final, finally and finalize differ?

20-SECOND ANSWER

final restricts changes; finally runs cleanup control flow; finalize is an obsolete, unreliable cleanup mechanism.

WHY IT WORKS · EASY EXPLANATION

final prevents reassigning names, not changing its list. finally prints cleanup when normal try control flow exits. Use try-with-resources for owned closeable resources rather than relying on finalize.

JAVA EXAMPLE / SCENARIO

```
final List<String> names = new ArrayList<>();
   names.add("Sai");         // Allowed: object is mutable
   // names = new ArrayList<>(); // Compile-time error
   try {
       System.out.println("work");
   } finally {
       System.out.println("cleanup");
   }
```

DRAW THE IDEA

START / CASE
final List<String> names = new ArrayList<>();

JAVA RULE
final restricts changes; finally runs cleanup control...

RESULT
final prevents reassigning names, not changing its...

Watch out: finally is not guaranteed during forced process termination.

INTERVIEWER FOLLOW-UP - ANSWERED

Does a final field make every operation on its object thread safe?

No. final prevents reassignment of that field; it does not synchronize mutations of the referenced object or make its methods atomic.

Original source: Core Java Zero to Hero, page 23 · Q020 · Interview study edition

Q021 What is the difference between shallow and deep copying?

20-SECOND ANSWER

A shallow copy duplicates the outer container; a deep copy duplicates the mutable elements that must be independently owned.

WHY IT WORKS · EASY EXPLANATION

The lists differ, but their first elements reference the same StringBuilder. To isolate this element, create a new StringBuilder from its text while copying. Immutable elements can safely be shared.

JAVA EXAMPLE / SCENARIO

```
List<StringBuilder> original = new ArrayList<>();
   original.add(new StringBuilder("A"));
   List<StringBuilder> copy = new ArrayList<>(original);
   copy.get(0).append("B");
   System.out.println(original.get(0)); // AB
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
List<StringBuilder> original = new...	A shallow copy duplicates the outer container; a deep...	The lists differ, but their first elements reference...

Watch out: A recursive deep copy must account for cycles and shared identity if they matter.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you copy an object that contains a mutable list of mutable items?

Copy the outer collection and each mutable element, including nested mutable fields where ownership requires it. Define how shared references and cycles should be treated.

Original source: Core Java Zero to Hero, page 23 · Q021 · Interview study edition

Q022 Why is equals difficult across an inheritance hierarchy?

20-SECOND ANSWER

Adding equality-relevant state in a subclass can break symmetry or transitivity across the hierarchy.

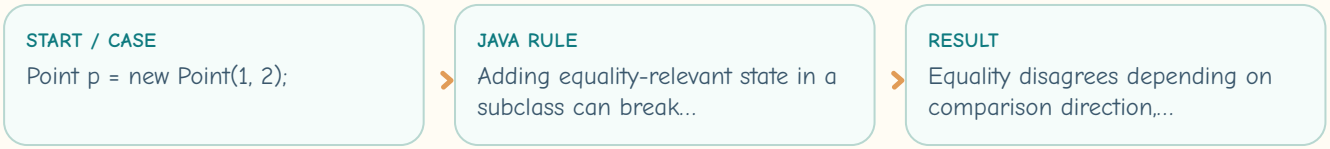
WHY IT WORKS · EASY EXPLANATION

Equality disagrees depending on comparison direction, violating symmetry. A strict same-class policy avoids that comparison, but changes the domain contract. Final value classes or composition often make the policy simpler.

JAVA EXAMPLE / SCENARIO

```
// Assume Point accepts any Point subtype with the same x/y.
   Point p = new Point(1, 2);
   ColoredPoint c = new ColoredPoint(1, 2, "red");
   // p.equals(c) -> true
   // c.equals(p) -> false if it requires matching color
```

DRAW THE IDEA



Watch out: There is no universal getClass versus instanceof rule independent of the domain.

INTERVIEWER FOLLOW-UP - ANSWERED

Why do records simplify this design choice?

Records are final and generate component-based equals/hashCode, reducing inheritance-related symmetry problems. Mutable components still need defensive-copy rules.

Original source: Core Java Zero to Hero, page 24 · Q022 · Interview study edition

Q023 How are identity, value equality and ordering different?

20-SECOND ANSWER

== compares reference identity, equals compares domain values, and comparison zero means equivalence in an ordering.

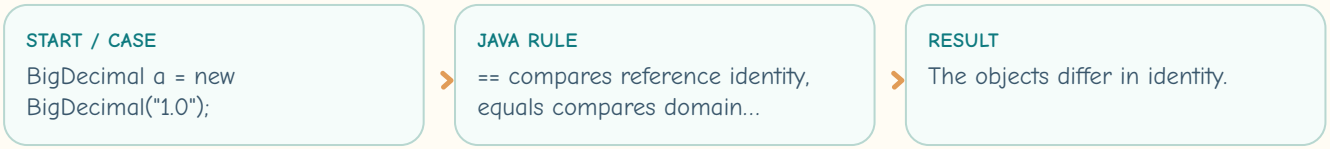
WHY IT WORKS · EASY EXPLANATION

The objects differ in identity. BigDecimal.equals includes scale, so they differ as values under that contract. compareTo compares numeric magnitude, so they occupy the same natural-order position in a TreeSet.

JAVA EXAMPLE / SCENARIO

```
BigDecimal a = new BigDecimal("1.0");
   BigDecimal b = new BigDecimal("1.00");
   System.out.println(a == b);           // false
   System.out.println(a.equals(b));      // false
   System.out.println(a.compareTo(b));   // 0
```

DRAW THE IDEA



Watch out: Objects.compare(a,b,cmp) and Objects.equals(a,b) answer different questions.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens if a TreeSet comparator compares employees only by salary?

Employees with equal salaries compare as zero and TreeSet keeps only one of them. Add a unique tie-breaker, such as employee ID, when distinct employees must remain.

Original source: Core Java Zero to Hero, page 24 · Q023 · Interview study edition

Q024 When are immutable objects useful in concurrent code?

20-SECOND ANSWER

Immutable snapshots simplify concurrent reads; publishing or replacing them still needs a clear coordination policy.

WHY IT WORKS · EASY EXPLANATION

Readers obtain a complete snapshot through the volatile reference. Config's value never changes afterward. Two writers doing read-modify-write replacement can still lose updates unless a lock or atomic update coordinates them.

JAVA EXAMPLE / SCENARIO

```
final class Config {
       final int timeout;
       Config(int t) { timeout = t; }
   }
   volatile Config current = new Config(1000);
   // Writer:
   current = new Config(2000);
   // Reader:
   Config snapshot = current;
```

DRAW THE IDEA

START / CASE
final class Config {

JAVA RULE
Immutable snapshots simplify concurrent reads;...

RESULT
Readers obtain a complete snapshot through the volatile...

Watch out: Immutability of a snapshot does not make a read-modify-write replacement atomic.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you prevent two config writers from losing each other's updates?

Use one immutable Config value in AtomicReference and update it with compareAndSet/ updateAndGet, or guard the full read-modify-write with one lock. Retry functions should be side-effect-free.

Original source: Core Java Zero to Hero, page 25 · Q024 · Interview study edition

MODULE 04 / 21

04

Strings, numbers and arrays

String literals are interned, but use equals for text comparison regardless of pooling.

8 interview questions · clear examples · follow-up answers

Q025 How does the string pool affect == and equals?

20-SECOND ANSWER

String literals are interned, but use equals for text comparison regardless of pooling.

WHY IT WORKS · EASY EXPLANATION

a and the compile-time constant b use the pooled string. c is a distinct object with identical text. Pooling explains identity; equals expresses the application's text comparison.

JAVA EXAMPLE / SCENARIO

```
String a = "java";
   String b = "ja" + "va";
   String c = new String("java");
   System.out.println(a == b);      // true
   System.out.println(a == c);      // false
   System.out.println(a.equals(c)); // true
```

DRAW THE IDEA

START / CASE
String a = "java";

JAVA RULE
String literals are interned, but use equals for text...

RESULT
a and the compile-time constant b use the pooled...

Watch out: Do not infer business equality from incidental string interning.

INTERVIEWER FOLLOW-UP - ANSWERED

What changes if one operand of concatenation is a non-final variable?

Concatenation may happen at runtime, producing a different String object rather than the same interned compile-time constant. Compare text with equals(), not ==.

Original source: Core Java Zero to Hero, page 26 · Q025 · Interview study edition

Q026 Why is String immutable?

20-SECOND ANSWER

String is immutable: its methods produce a result rather than modifying its existing value.

WHY IT WORKS · EASY EXPLANATION

The first call's returned value is ignored. The second assignment changes the local reference to the uppercase result. The original string remains unchanged, so its hash and value stay stable when shared.

JAVA EXAMPLE / SCENARIO

```
String s = "java";
   s.toUpperCase(Locale.ROOT);
   System.out.println(s); // java
   s = s.toUpperCase(Locale.ROOT);
   System.out.println(s); // JAVA
```

DRAW THE IDEA

START / CASE
String s = "java";

JAVA RULE
String is immutable: its methods produce a result...

RESULT
The first call's returned value is ignored.

Watch out: Immutability does not mean that every string operation creates a fresh object.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is a char[] sometimes preferred for a password?

A char[] can be overwritten after use; an immutable String cannot be cleared in place. This reduces some exposure, but copies, logging and runtime memory still need care.

Original source: Core Java Zero to Hero, page 27 · Q026 · Interview study edition

Q027 When should you use StringBuilder or StringBuffer?

20-SECOND ANSWER

Use StringBuilder for mutable text built within one thread. StringBuffer synchronizes methods but not every multi-step workflow.

WHY IT WORKS · EASY EXPLANATION

One builder accumulates characters before producing a String. Repeatedly replacing a growing String in a loop can cause extra copying. A thread-local builder needs no StringBuffer synchronization.

JAVA EXAMPLE / SCENARIO

```
StringBuilder out = new StringBuilder();
   for (int i = 1; i <= 3; i++) {
       out.append(i).append(' ');
   }
   System.out.println(out.toString()); // 1 2 3 (trailing space)
```

DRAW THE IDEA

START / CASE
StringBuilder out = new StringBuilder();

JAVA RULE
Use StringBuilder for mutable text built within one...

RESULT
One builder accumulates characters before producing a...

Watch out: Synchronization on individual StringBuffer calls does not make every multi-call algorithm atomic.

INTERVIEWER FOLLOW-UP - ANSWERED

Can a shared StringBuilder be safe if all access uses one external lock?

Yes, if every read and write uses the same lock and the builder does not escape that policy. Individual uncoordinated accesses remain unsafe.

Original source: Core Java Zero to Hero, page 27 · Q027 · Interview study edition

Q028 What is the difference between a char and a Unicode code point?

20-SECOND ANSWER

A char is a UTF-16 code unit. Some Unicode code points need two chars, and a displayed character can contain several code points.

WHY IT WORKS · EASY EXPLANATION

The supplementary emoji uses a surrogate pair, so length counts two code units. codePointCount counts one code point. Combining marks and joined emoji require grapheme-aware handling for displayed-character operations.

JAVA EXAMPLE / SCENARIO

```
String emoji = new String(Character.toChars(0x1F600));
   System.out.println(emoji.length()); // 2
   System.out.println(emoji.codePointCount(0, emoji.length())); // 1
```

DRAW THE IDEA

START / CASE
String emoji = new...

JAVA RULE
A char is a UTF-16 code unit.

RESULT
The supplementary emoji uses a surrogate pair, so...

Watch out: A char-based frequency map can split a supplementary character.

INTERVIEWER FOLLOW-UP - ANSWERED

Would reversing code points preserve every user-visible grapheme?

No. A grapheme can contain several code points (combining marks or joined emoji). Use Unicode grapheme segmentation, for example BreakIterator with its documented limitations.

Original source: Core Java Zero to Hero, page 28 · Q028 · Interview study edition

Q029 Why should money usually avoid double?

20-SECOND ANSWER

Use decimal arithmetic with an explicit rounding policy for money; binary double cannot represent many decimal fractions exactly.

WHY IT WORKS · EASY EXPLANATION

The double result reflects binary approximation. Decimal string construction preserves the intended decimal amount. Choose currency scale and rounding at the business-defined boundary.

JAVA EXAMPLE / SCENARIO

```
System.out.println(0.1 + 0.2); // 0.30000000000000004
   BigDecimal total = new BigDecimal("0.10")
           .add(new BigDecimal("0.20"));
   System.out.println(total); // 0.30
```

DRAW THE IDEA

START / CASE
System.out.println(0.1 + 0.2); //...

JAVA RULE
Use decimal arithmetic with an explicit rounding policy...

RESULT
The double result reflects binary approximation.

Watch out: BigDecimal operations return new values; add does not mutate the receiver.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can divide throw ArithmeticException even when the divisor is nonzero?

Some exact decimal quotients never terminate, such as 1 / 3. Supply an explicit scale and RoundingMode, or a MathContext, for the domain.

Original source: Core Java Zero to Hero, page 28 · Q029 · Interview study edition

Q030 Why do BigDecimal.equals and compareTo give different results?

20-SECOND ANSWER

BigDecimal.equals includes scale; compareTo compares numeric value only.

WHY IT WORKS · EASY EXPLANATION

HashSet uses equals, so scale makes these values distinct. TreeSet uses natural comparison, so numeric equality collapses them. Normalize decimals according to your domain before using them as identity keys.

JAVA EXAMPLE / SCENARIO

```
BigDecimal a = new BigDecimal("2.0");
   BigDecimal b = new BigDecimal("2.00");
   Set<BigDecimal> hash = new HashSet<>(Arrays.asList(a, b));
   Set<BigDecimal> tree = new TreeSet<>(Arrays.asList(a, b));
   System.out.println(hash.size()); // 2
   System.out.println(tree.size()); // 1
```

DRAW THE IDEA

START / CASE
BigDecimal a = new BigDecimal("2.0");

JAVA RULE
BigDecimal.equals includes scale; compareTo compares...

RESULT
HashSet uses equals, so scale makes these values...

Watch out: stripTrailingZeros can produce a negative scale, so do not assume scale is always nonnegative.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you store a price key consistently?

Choose one canonical representation, such as fixed currency scale and currency code, before storing the key. Avoid treating 2.0 and 2.00 as different product prices accidentally.

Original source: Core Java Zero to Hero, page 29 · Q030 · Interview study edition

Q031 Why can array covariance throw ArrayStoreException?

20-SECOND ANSWER

Reference arrays allow covariance but check stores at runtime; an incompatible element causes ArrayStoreException.

WHY IT WORKS · EASY EXPLANATION

The variable type accepts Object, but the actual array accepts only String. Generics reject the analogous unsafe List assignment at compile time through invariance.

JAVA EXAMPLE / SCENARIO

```
Object[] values = new String[1];
   values[0] = "Java"; // Allowed
   // values[0] = 42; // ArrayStoreException
   // List<Object> xs = new ArrayList<String>(); // Compile error
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
Object[] values = new String[1];	Reference arrays allow covariance but check stores at...	The variable type accepts Object, but the actual array...

Watch out: List<String> cannot be assigned to List<Object>.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can new List<String>[10] not be created directly?

Generic component types are erased and not reifiable, so Java cannot check array stores for List<String> at runtime. Prefer List<List<String>> or a carefully managed List<?>[].

Original source: Core Java Zero to Hero, page 29 · Q031 · Interview study edition

Q032 How do arrays and lists differ in copying and conversion?

20-SECOND ANSWER

Arrays.asList creates a fixed-size view over a reference array; copying it into ArrayList creates independently resizable structure.

WHY IT WORKS · EASY EXPLANATION

set changes the backing array. add would change its fixed length and is rejected. The ArrayList copy has its own structure. Arrays.asList(new int[]{1,2}) creates one list element containing the primitive array.

JAVA EXAMPLE / SCENARIO

```
String[] a = {"A", "B"};
   List<String> view = Arrays.asList(a);
   view.set(0, "X");
   System.out.println(a[0]); // X
   // view.add("C");        // UnsupportedOperationException
   List<String> copy = new ArrayList<>(view);
   copy.add("C");           // Allowed
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
String[] a = {"A", "B"};	Arrays.asList creates a fixed-size view over a...	set changes the backing array. add would change its...

Watch out: Fixed size and unmodifiable describe different properties.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you get a mutable list from an int[]?

Use Arrays.stream(intArray).boxed().collect(Collectors.toCollection(ArrayList::new)); Arrays.asList(intArray) gives a single element containing the whole primitive array.

Original source: Core Java Zero to Hero, page 30 · Q032 · Interview study edition

MODULE 05 / 21

05

Exceptions and resource ownership

Checked exceptions must be caught or declared; unchecked exceptions have no such compile-time obligation.

8 interview questions · clear examples · follow-up answers

Q033 How do checked and unchecked exceptions differ?

20-SECOND ANSWER

Checked exceptions must be caught or declared; unchecked exceptions have no such compile-time obligation.

WHY IT WORKS · EASY EXPLANATION

Calling read requires handling or declaring IOException. Calling validate does not require a catch declaration. Exception category describes compiler obligations, not whether recovery is possible.

JAVA EXAMPLE / SCENARIO

```
static void read() throws IOException {
       throw new IOException("File unavailable");
   }
   static void validate(int age) {
       if (age < 0) throw new IllegalArgumentException("age");
   }
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
static void read() throws IOException {	Checked exceptions must be caught or declared;...	Calling read requires handling or declaring IOException.

Watch out: RuntimeException is still an Exception, while Error is a separate Throwable branch.

INTERVIEWER FOLLOW-UP - ANSWERED

What should a repository layer expose if callers cannot use a vendor-specific checked exception?

Translate it at the repository boundary to a stable domain/data-access exception with the original cause preserved. Document whether callers may retry or must reject.

Original source: Core Java Zero to Hero, page 31 · Q033 · Interview study edition

Q034 What are the disadvantages of checked exceptions?

20-SECOND ANSWER

Checked exceptions make failures explicit, but can spread implementation coupling and add wrapping boilerplate.

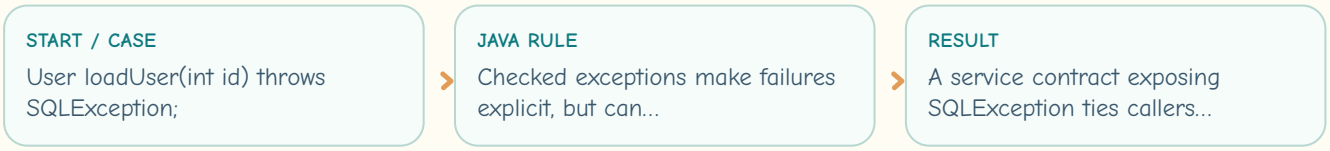
WHY IT WORKS · EASY EXPLANATION

A service contract exposing SQLException ties callers to a database detail. Translation can preserve the cause while exposing a stable domain contract. Keep checked exceptions when callers have a meaningful recovery responsibility.

JAVA EXAMPLE / SCENARIO

```
// JDBC detail leaks into every caller's signature:
   User loadUser(int id) throws SQLException;
   // An alternative boundary can translate to a domain failure:
   // throw new UserLoadException("Cannot load " + id, cause);
```

DRAW THE IDEA



Watch out: Checked exceptions are not inherently bad; the recovery contract determines their value.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you handle IOException inside a stream mapping operation?

Catch IOException inside the mapping function and wrap it in an unchecked exception with its cause, or move the I/O outside the stream. Translate it back at a suitable boundary.

Original source: Core Java Zero to Hero, page 32 · Q034 · Interview study edition

Q035 What is the difference between throw and throws?

20-SECOND ANSWER

throw raises an exception object; throws declares failures a method can propagate.

WHY IT WORKS · EASY EXPLANATION

The throws clause alone performs no action. The throw statement changes control flow immediately. An overriding method cannot broaden checked failures beyond the parent's permitted contract.

JAVA EXAMPLE / SCENARIO

```
static void load() throws IOException { // Declaration
       throw new IOException("Missing file"); // Action
   }
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
static void load() throws IOException { //...	throw raises an exception object; throws declares...	The throws clause alone performs no action.

Watch out: A method can declare an unchecked exception for documentation without forcing callers to catch it.

INTERVIEWER FOLLOW-UP - ANSWERED

Can an override narrow a checked exception from Exception to IOException?

Yes. An override may declare the same, a narrower checked exception such as IOException, or none, as long as the superclass declares a compatible checked type.

Original source: Core Java Zero to Hero, page 32 · Q035 · Interview study edition

Q036 What happens when both try and finally return or throw?

20-SECOND ANSWER

An abrupt return or throw in finally replaces the pending return or exception from try.

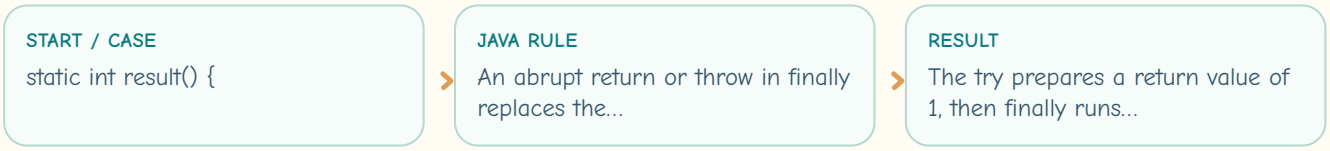
WHY IT WORKS · EASY EXPLANATION

The try prepares a return value of 1, then finally runs before returning. Its return replaces the pending result. Avoid returns in finally because they can also hide failures.

JAVA EXAMPLE / SCENARIO

```
static int result() {
       try { return 1; }
       finally { return 2; }
   }
   System.out.println(result()); // 2
```

DRAW THE IDEA



Watch out: Avoid control-flow-changing finally blocks.

INTERVIEWER FOLLOW-UP - ANSWERED

What if try throws IOException and finally throws IllegalStateException?

IllegalStateException from finally replaces the pending IOException, which is normally lost unless explicitly preserved. Avoid throwing from finally; use try-with-resources for suppression.

Original source: Core Java Zero to Hero, page 33 · Q036 · Interview study edition

Q037 How does try-with-resources close multiple resources?

20-SECOND ANSWER

Try-with-resources closes owned resources in reverse declaration order and preserves body failures through suppression.

WHY IT WORKS · EASY EXPLANATION

The later-created output closes first. If the body fails and close also fails, the body exception remains primary and close exceptions are available through getSuppressed. Resources must implement AutoCloseable.

JAVA EXAMPLE / SCENARIO

```
try (InputStream in = Files.newInputStream(source);
        OutputStream out = Files.newOutputStream(target)) {
       // Read and write bytes.
   }
   // Close order: out, then in
```

DRAW THE IDEA

START / CASE
try (InputStream in =...

JAVA RULE
Try-with-resources closes owned resources in reverse...

RESULT
The later-created output closes first.

Watch out: Check getSuppressed() when diagnosing a body failure followed by failed cleanup.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens if creating the second resource fails?

Already-created first resource is closed even if construction of the second resource throws. The original construction failure stays primary and closing failures are suppressed.

Original source: Core Java Zero to Hero, page 33 · Q037 · Interview study edition

Q038 How should you wrap an exception without losing useful information?

20-SECOND ANSWER

Wrap with meaningful context and keep the original cause so diagnosis remains possible.

WHY IT WORKS · EASY EXPLANATION

The outer message identifies the operation. The cause retains the lower-level stack trace and failure reason. This sketch assumes ImportException has a constructor accepting a message and cause.

JAVA EXAMPLE / SCENARIO

```
try {
       importBatch(batchId);
   } catch (IOException cause) {
       throw new ImportException("Failed batch " + batchId, cause);
   }
```

DRAW THE IDEA

START / CASE
try {

JAVA RULE
Wrap with meaningful context and keep the original...

RESULT
The outer message identifies the operation.

Watch out: A message-only wrapper loses the original stack and cause.

INTERVIEWER FOLLOW-UP - ANSWERED

Which exceptions should you avoid catching as part of an ordinary retry loop?

Avoid blindly catching fatal JVM errors (such as OutOfMemoryError), cancellation, and stable validation or business errors. Retry only documented transient failures with bounded attempts.

Original source: Core Java Zero to Hero, page 34 · Q038 · Interview study edition

Q039 Why should catch blocks be ordered from specific to general?

20-SECOND ANSWER

Order catch blocks from specific to general; otherwise a broader catch makes the narrower one unreachable.

WHY IT WORKS · EASY EXPLANATION

A missing file matches the first catch. Other I/O failures match the second. Reversing these catches is a compile-time error because IOException already covers FileNotFoundException.

JAVA EXAMPLE / SCENARIO

```
try {
       readFile(); // Declares IOException
   } catch (FileNotFoundException e) {
       System.out.println("Missing file");
   } catch (IOException e) {
       System.out.println("Other I/O failure");
   }
```

DRAW THE IDEA

START / CASE
try {

JAVA RULE
Order catch blocks from specific to general; otherwise...

RESULT
A missing file matches the first catch.

Watch out: catch (IOException | FileNotFoundException e) is invalid because one is a subtype of the other.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can a multi-catch parameter not be reassigned?

A multi-catch variable has an implicitly final binding because its type represents several alternatives. Reassignment would undermine type safety in that combined catch.

Original source: Core Java Zero to Hero, page 34 · Q039 · Interview study edition

Q040 What should you do with InterruptedException?

20-SECOND ANSWER

Propagate interruption or restore the interrupt flag and stop the current operation when propagation is unsuitable.

WHY IT WORKS · EASY EXPLANATION

Blocking take reacts to interruption by throwing InterruptedException and clearing the status. Restoring the flag preserves cancellation information for outer code. Returning stops this operation instead of silently continuing.

JAVA EXAMPLE / SCENARIO

```
try {
        queue.take();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return;
    }
```

DRAW THE IDEA

START / CASE
try {

JAVA RULE
Propagate interruption or restore the interrupt flag...

RESULT
Blocking take reacts to interruption by throwing...

Watch out: Restoring the flag and immediately continuing the same blocking loop can create a busy failure loop.

INTERVIEWER FOLLOW-UP - ANSWERED

Why does Future.cancel(true) not guarantee that a task stops?

cancel(true) requests interruption if running, but code may ignore it or be blocked in uninterruptible work. Check task termination and resource cleanup separately.

Original source: Core Java Zero to Hero, page 35 · Q040 · Interview study edition

MODULE 06 / 21

06

Generics, nested classes and annotations

Generics are invariant: List<Dog> is not a List<Animal>, even when Dog extends Animal.

8 interview questions · clear examples · follow-up answers

Q041 Why are generic collections invariant?

20-SECOND ANSWER

Generics are invariant: List<Dog> is not a List<Animal>, even when Dog extends Animal.

WHY IT WORKS · EASY EXPLANATION

If unrestricted assignment were allowed, a Cat could enter a list promised to contain Dogs. The extends wildcard permits reading Animal values, but blocks adding arbitrary Animal instances.

JAVA EXAMPLE / SCENARIO

```
List<Dog> dogs = new ArrayList<>();
   dogs.add(new Dog());
   // List<Animal> animals = dogs; // Compile-time error
   List<? extends Animal> animals = dogs;
   Animal first = animals.get(0); // Assumes a non-empty list
   // animals.add(new Cat());     // Compile-time error
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
List<Dog> dogs = new ArrayList<>();	Generics are invariant: List<Dog> is not a...	If unrestricted assignment were allowed, a Cat could...

Watch out: Inheritance between element types does not imply inheritance between their generic containers.

INTERVIEWER FOLLOW-UP - ANSWERED

Which value can you always add to a List<? extends Animal>?

You may add null to a List<? extends Animal> when null is supported. You cannot safely add a particular Animal because its actual element subtype is unknown.

Original source: Core Java Zero to Hero, page 36 · Q041 · Interview study edition

Q042 What does PECS mean and when do you use it?

20-SECOND ANSWER

PECS means producer extends, consumer super: use wildcards according to whether you read or write values.

WHY IT WORKS · EASY EXPLANATION

src produces values that can be read as T. dst accepts T values even if it is a list of a supertype. This allows integers to be copied safely into a Number list.

JAVA EXAMPLE / SCENARIO

```
static <T> void copy(List<? super T> dst,
                     List<? extends T> src) {
      for (T value : src) dst.add(value);
  }
  List<Integer> src = Arrays.asList(1, 2);
  List<Number> dst = new ArrayList<>();
  copy(dst, src);
```

DRAW THE IDEA

START / CASE
static <T> void copy(List<? super T> dst,

JAVA RULE
PECS means producer extends, consumer super: use...

RESULT
src produces values that can be read as T. dst accepts...

Watch out: Reading from ? super T yields Object because the actual list may hold a wider type.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is List<?> different from raw List?

List<?> retains generic type checking: reading gives Object, and arbitrary additions are blocked. Raw List suppresses checks and can introduce heap pollution.

Original source: Core Java Zero to Hero, page 37 · Q042 · Interview study edition

Q043 What is type erasure?

20-SECOND ANSWER

Type erasure removes most generic type arguments from runtime object types while the compiler inserts checks and casts.

WHY IT WORKS · EASY EXPLANATION

Both objects have runtime class ArrayList. Their generic arguments enforce source-level safety, not different runtime classes. Some generic signatures remain available as metadata; erasure does not mean every type detail disappears.

JAVA EXAMPLE / SCENARIO

```
List<String> a = new ArrayList<>();
   List<Integer> b = new ArrayList<>();
   System.out.println(a.getClass() == b.getClass()); // true
```

DRAW THE IDEA

START / CASE
List<String> a = new ArrayList<>();

JAVA RULE
Type erasure removes most generic type arguments from...

RESULT
Both objects have runtime class ArrayList.

Watch out: Erasure does not mean all generic declaration metadata disappears.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can overloads f(List<String>) and f(List<Integer>) clash?

Both erase to the same JVM parameter type List, causing a name clash. Use different method names or distinct parameter types.

Original source: Core Java Zero to Hero, page 37 · Q043 · Interview study edition

Q044 Why can you not use new T() or instanceof List<String>?

20-SECOND ANSWER

An erased type parameter does not provide a concrete constructor or a runtime test for parameterized element types.

WHY IT WORKS · EASY EXPLANATION

The Supplier supplies the construction decision that new T cannot make. List<?> can test the runtime list type, but the erased list object cannot prove that all elements are String.

JAVA EXAMPLE / SCENARIO

```
static <T> T create(Supplier<T> factory) {
       return factory.get();
   }
   StringBuilder b = create(StringBuilder::new);
   Object value = new ArrayList<String>();
   System.out.println(value instanceof List<?>); // true
   // value instanceof List<String> // Not permitted here
```

DRAW THE IDEA

START / CASE
static <T> T create(Supplier<T> factory) {

JAVA RULE
An erased type parameter does not provide a concrete...

RESULT
The Supplier supplies the construction decision that...

Watch out: An unchecked cast does not validate every element of a collection.

INTERVIEWER FOLLOW-UP - ANSWERED

When would a Class<T> token be useful?

A Class<T> token lets you validate, cast or construct types at runtime despite generic erasure, for example T value = type.cast(raw).

Original source: Core Java Zero to Hero, page 38 · Q044 · Interview study edition

Q045 What causes heap pollution?

20-SECOND ANSWER

Heap pollution occurs when a parameterized reference reaches an object containing incompatible values, often through raw types.

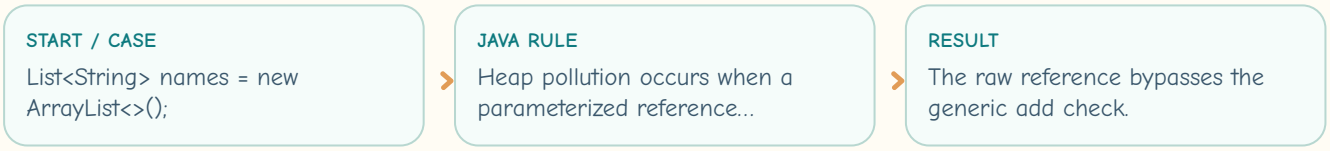
WHY IT WORKS · EASY EXPLANATION

The raw reference bypasses the generic add check. Reading through List<String> inserts a String cast, which fails for Integer. Treat unchecked warnings as points requiring review.

JAVA EXAMPLE / SCENARIO

```
List<String> names = new ArrayList<>();
   List raw = names; // Unchecked/raw-type warning
   raw.add(42);     // Pollutes the list
   // String name = names.get(0); // ClassCastException
```

DRAW THE IDEA



Watch out: @SuppressWarnings and @SafeVarargs are promises about safety, not runtime enforcement.

INTERVIEWER FOLLOW-UP - ANSWERED

Why should a generic varargs method avoid writing incompatible elements into its parameter array?

Generic varargs expose an array whose runtime component type may be less specific than the generic promise. Storing the wrong element can cause heap pollution and later ClassCastException.

Original source: Core Java Zero to Hero, page 38 · Q045 · Interview study edition

Q046 How do static nested classes and inner classes differ?

20-SECOND ANSWER

A static nested class needs no enclosing instance; an inner class belongs to an enclosing object.

WHY IT WORKS · EASY EXPLANATION

Nested can be constructed directly. Inner is constructed relative to outer and can access its instance state. This enclosing relationship can unintentionally retain an outer object.

JAVA EXAMPLE / SCENARIO

```
class Outer {
       static class Nested {}
       class Inner {}
   }
   Outer.Nested n = new Outer.Nested();
   Outer outer = new Outer();
   Outer.Inner i = outer.new Inner();
```

DRAW THE IDEA

START / CASE
class Outer {

JAVA RULE
A static nested class needs no enclosing instance; an...

RESULT
Nested can be constructed directly.

Watch out: An unnecessary enclosing reference can extend an object's lifetime.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is a static nested node class often suitable inside a data structure?

A static nested node has no implicit reference to the enclosing instance, reducing accidental retention and unnecessary per-node state.

Original source: Core Java Zero to Hero, page 39 · Q046 · Interview study edition

Q047 What does effectively final mean for a lambda capture?

20-SECOND ANSWER

A captured local variable must be final or effectively final; the referenced object's contents can still change.

WHY IT WORKS · EASY EXPLANATION

The reference out stays assigned to the same object, so capture is legal. The lambda changes that object's contents. Effective finality does not make the list immutable or thread safe.

JAVA EXAMPLE / SCENARIO

```
List<String> out = new ArrayList<>();
   Runnable r = () -> out.add("A");
   r.run();
   System.out.println(out); // [A]
   // out = new ArrayList<>(); // Would invalidate the capture
```

DRAW THE IDEA

START / CASE
List<String> out = new ArrayList<>();

JAVA RULE
A captured local variable must be final or effectively...

RESULT
The reference out stays assigned to the same object, so...

Watch out: Compiling a capture does not make the referenced mutable object thread safe.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can an incremented local int not be captured directly?

Captured local variables must be final or effectively final. Use an AtomicInteger for genuine shared mutation or compute a new stream result rather than incrementing a local.

Original source: Core Java Zero to Hero, page 39 · Q047 · Interview study edition

Q048 How do annotation retention and targets work?

20-SECOND ANSWER

Retention controls how long annotation metadata survives; target controls where the annotation can appear.

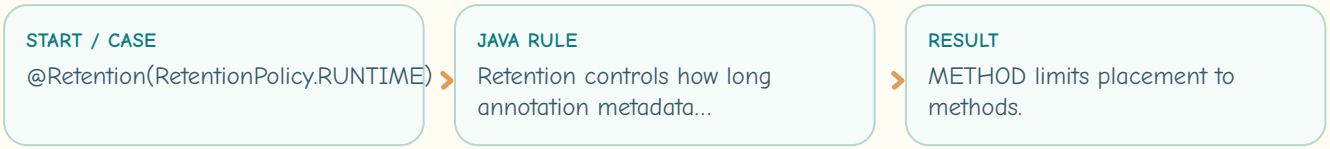
WHY IT WORKS · EASY EXPLANATION

METHOD limits placement to methods. RUNTIME keeps the annotation available to reflection. SOURCE would discard it during compilation; CLASS retains it in the class file without ordinary runtime reflection access.

JAVA EXAMPLE / SCENARIO

```
@Retention(RetentionPolicy.RUNTIME)
   @Target(ElementType.METHOD)
   @interface Audited {}
   class Service { @Audited public void save() {} }
   boolean marked = Service.class.getMethod("save")
           .isAnnotationPresent(Audited.class); // true
```

DRAW THE IDEA



Watch out: @Override aids compiler checking and does not require reflective runtime retention.

INTERVIEWER FOLLOW-UP - ANSWERED

When would you choose compile-time processing over runtime reflection?

Use annotation processing when you need validation or generated code at build time, avoiding runtime reflection cost and catching mistakes before deployment.

Original source: Core Java Zero to Hero, page 40 · Q048 · Interview study edition

MODULE 07 / 21

07

Lists, sets, queues and ordering

ArrayList is usually the default list; LinkedList helps specific iterator-local operations but has costly indexed access and extra nodes.

8 interview questions · clear examples · follow-up answers

Q049 How do ArrayList and LinkedList compare?

20-SECOND ANSWER

ArrayList is usually the default list; LinkedList helps specific iterator-local operations but has costly indexed access and extra nodes.

WHY IT WORKS · EASY EXPLANATION

ArrayList uses a resizable array, giving fast indexed reads and good locality. LinkedList must walk to an index. Repeated get(i) over all LinkedList positions can turn a traversal into quadratic work.

JAVA EXAMPLE / SCENARIO

```
List<Integer> values = new ArrayList<>();
   values.add(10);
   values.add(20);
   System.out.println(values.get(1)); // 20: constant-time access
   // On LinkedList, get(i) must traverse nodes.
```

DRAW THE IDEA

START / CASE
List<Integer> values = new ArrayList<>();

JAVA RULE
ArrayList is usually the default list; LinkedList helps...

RESULT
ArrayList uses a resizable array, giving fast indexed...

Watch out: LinkedList middle insertion is not constant time when you first locate the index.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is ArrayDeque usually a better queue than LinkedList?

ArrayDeque is usually faster and has less node allocation for stack/queue operations. It disallows null and is not thread-safe.

Original source: Core Java Zero to Hero, page 41 · Q049 · Interview study edition

Q050 How do HashSet, LinkedHashSet and TreeSet differ?

20-SECOND ANSWER

HashSet has no encounter-order guarantee, LinkedHashSet preserves insertion order, and TreeSet uses sorted comparator order.

WHY IT WORKS · EASY EXPLANATION

Both remove duplicates. LinkedHashSet retains each value's first insertion position. TreeSet sorts values and uses comparison zero to decide whether an element is already present.

JAVA EXAMPLE / SCENARIO

```
List<Integer> input = Arrays.asList(3, 1, 3, 2);
   System.out.println(new LinkedHashSet<>(input)); // [3, 1, 2]
   System.out.println(new TreeSet<>(input));       // [1, 2, 3]
```

DRAW THE IDEA

START / CASE
List<Integer> input = Arrays.asList(3, 1, 3,...

JAVA RULE
HashSet has no encounter-order guarantee, LinkedHashSet...

RESULT
Both remove duplicates.

Watch out: TreeSet uniqueness depends on comparison zero, not a separate equals check.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens when a key's ordering field changes after insertion?

TreeSet/TreeMap maintain search structure using the original ordering. Mutation may make contains(), remove() and navigation inconsistent; use immutable sort keys.

Original source: Core Java Zero to Hero, page 42 · Q050 · Interview study edition

Q051 How do Comparable and Comparator differ?

20-SECOND ANSWER

Comparable defines a type's natural order; Comparator supplies an external ordering that can vary by use case.

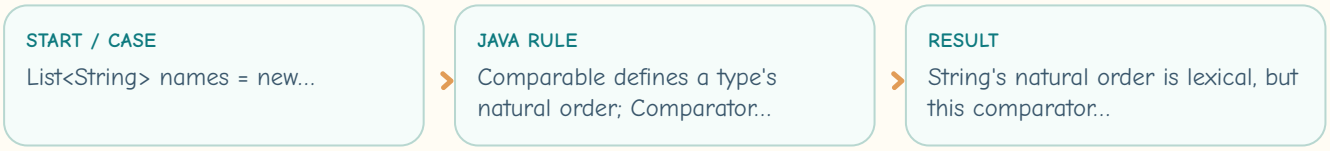
WHY IT WORKS · EASY EXPLANATION

String's natural order is lexical, but this comparator first orders by length. A lexical tie breaker makes equal-length ordering explicit without changing String's own implementation.

JAVA EXAMPLE / SCENARIO

```
List<String> names = new ArrayList<>(Arrays.asList("Sai", "A", "Bo"));
   names.sort(Comparator.comparingInt(String::length)
                        .thenComparing(Comparator.naturalOrder()));
   System.out.println(names); // [A, Bo, Sai]
```

DRAW THE IDEA



Watch out: Reversing an entire chained comparator also reverses its tie breakers.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you sort salary descending and name ascending?

Use Comparator.comparing(Employee::salary).reversed().thenComparing(Employee::name), adding a final unique key if deterministic total ordering is required.

Original source: Core Java Zero to Hero, page 42 · Q051 · Interview study edition

Q052 What is a fail-fast iterator?

20-SECOND ANSWER

A fail-fast iterator may detect unsupported structural changes; it is a bug detector, not a thread-safety guarantee.

WHY IT WORKS · EASY EXPLANATION

iterator.remove coordinates with that iterator's expected modification state. Calling xs.remove directly during traversal can invalidate it and cause ConcurrentModificationException, even in one thread.

JAVA EXAMPLE / SCENARIO

```
List<Integer> xs = new ArrayList<>(Arrays.asList(1, 2, 3));
   Iterator<Integer> it = xs.iterator();
   while (it.hasNext()) {
       if (it.next() % 2 == 0) it.remove();
   }
   System.out.println(xs); // [1, 3]
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
List<Integer> xs = new...	A fail-fast iterator may detect unsupported structural...	iterator.remove coordinates with that iterator's...

Watch out: Absence of ConcurrentModificationException does not prove concurrent access is safe.

INTERVIEWER FOLLOW-UP - ANSWERED

How does a ConcurrentHashMap iterator differ?

ConcurrentHashMap iterators are weakly consistent: they do not throw ordinary fail-fast ConcurrentModificationException and may reflect some updates, but not a full snapshot.

Original source: Core Java Zero to Hero, page 43 · Q052 · Interview study edition

Q053 How do immutable collections differ from unmodifiable views?

20-SECOND ANSWER

An unmodifiable view blocks changes through the view; a structural immutable snapshot also isolates later backing-list changes.

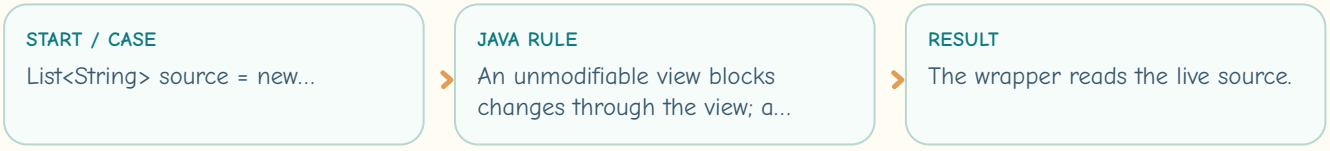
WHY IT WORKS · EASY EXPLANATION

The wrapper reads the live source. List.copyOf preserves the accepted structure as a snapshot. Neither approach deep-copies mutable elements; here String values are immutable.

JAVA EXAMPLE / SCENARIO

```
List<String> source = new ArrayList<>(Arrays.asList("A"));
   List<String> view = Collections.unmodifiableList(source);
   List<String> snapshot = List.copyOf(source); // Java 10+
   source.add("B");
   System.out.println(view);     // [A, B]
   System.out.println(snapshot); // [A]
```

DRAW THE IDEA



Watch out: List.copyOf does not recursively copy mutable element objects.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you expose a safe snapshot when elements themselves are mutable?

Copy the collection and defensively copy each mutable element, or expose immutable DTO snapshots. A shallow copy alone leaves mutable objects shared.

Original source: Core Java Zero to Hero, page 43 · Q053 · Interview study edition

Q054 What is the difference between a queue, a deque and a priority queue?

20-SECOND ANSWER

A queue usually models FIFO, a deque supports both ends, and a priority queue removes the highest-priority element by its ordering.

WHY IT WORKS · EASY EXPLANATION

FIFO returns the oldest inserted value. The min-priority queue returns the smallest value. Its iterator is not guaranteed to visit all values in sorted order.

JAVA EXAMPLE / SCENARIO

```
Deque<Integer> fifo = new ArrayDeque<>();
   fifo.offer(3); fifo.offer(1);
   System.out.println(fifo.poll()); // 3
   Queue<Integer> priority = new PriorityQueue<>();
   priority.offer(3); priority.offer(1);
   System.out.println(priority.poll()); // 1
```

DRAW THE IDEA

START / CASE
Deque<Integer> fifo = new ArrayDeque<>();

JAVA RULE
A queue usually models FIFO, a deque supports both...

RESULT
FIFO returns the oldest inserted value.

Watch out: PriorityQueue iteration order is not a sorted result.

INTERVIEWER FOLLOW-UP - ANSWERED

How would a bounded BlockingQueue change producer behavior?

A bounded BlockingQueue blocks, times out or rejects when full, depending on put/offer policy. That creates backpressure instead of unlimited memory growth.

Original source: Core Java Zero to Hero, page 44 · Q054 · Interview study edition

Q055 What risks come with subList and other collection views?

20-SECOND ANSWER

subList is a backed view, so changes can affect its parent and parent structural changes can invalidate the view.

WHY IT WORKS · EASY EXPLANATION

The view shares the parent's storage and ownership. The copy has independent list structure. Do not keep a view while making unsupported structural changes to its parent.

JAVA EXAMPLE / SCENARIO

```
List<String> all = new ArrayList<>(Arrays.asList("A", "B", "C"));
   List<String> view = all.subList(1, 3);
   view.set(0, "X");
   System.out.println(all); // [A, X, C]
   List<String> copy = new ArrayList<>(view);
```

DRAW THE IDEA

START / CASE
List<String> all = new...

JAVA RULE
subList is a backed view, so changes can affect its...

RESULT
The view shares the parent's storage and ownership.

Watch out: A copy of a view still shares its mutable element objects.

INTERVIEWER FOLLOW-UP - ANSWERED

Would you return a live view from a public API without documenting it?

Only if the API explicitly promises a live view and defines mutation, concurrency and lifecycle behavior. Otherwise return a snapshot to prevent surprising changes.

Original source: Core Java Zero to Hero, page 44 · Q055 · Interview study edition

Q056 How do you choose a collection under interview constraints?

20-SECOND ANSWER

Choose a collection from required ordering, access, duplicates, mutation, concurrency, and memory limits.

WHY IT WORKS · EASY EXPLANATION

The bounded min-heap keeps the two largest values, 7 and 9. It uses O(k) space and O(n log k) time. Sorting all inputs would use more work when k is small; duplicate policy must also be stated.

JAVA EXAMPLE / SCENARIO

```
int k = 2;
   PriorityQueue<Integer> top = new PriorityQueue<>();
   for (int n : new int[]{7, 2, 9, 4}) {
       top.offer(n);
       if (top.size() > k) top.poll();
   }
   System.out.println(top.peek()); // 7: second-largest value
```

DRAW THE IDEA

START / CASE
int k = 2;

JAVA RULE
Choose a collection from required ordering, access,...

RESULT
The bounded min-heap keeps the two largest values, 7...

Watch out: One collection does not optimize every workload.

INTERVIEWER FOLLOW-UP - ANSWERED

What collection would support floor/ceiling queries on timestamps?

Use NavigableMap, commonly TreeMap<Instant,Value>, with floorEntry(), ceilingEntry() and range views. Use a concurrent alternative when needed.

Original source: Core Java Zero to Hero, page 45 · Q056 · Interview study edition

MODULE 08 / 21

08

HashMap and map design

HashMap uses a hash to select a bucket and equals to identify the matching key within candidates.

8 interview questions · clear examples · follow-up answers

Q057 How does HashMap put and get work?

20-SECOND ANSWER

HashMap uses a hash to select a bucket and equals to identify the matching key within candidates.

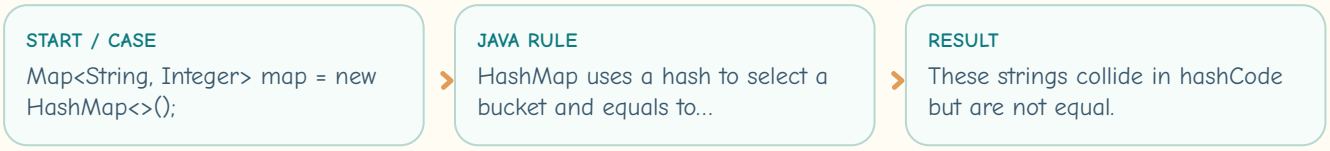
WHY IT WORKS · EASY EXPLANATION

These strings collide in hashCode but are not equal. The map keeps separate entries and distinguishes them through equality. A collision is normal; an inconsistent equals/hashCode contract is a defect.

JAVA EXAMPLE / SCENARIO

```
Map<String, Integer> map = new HashMap<>();
   System.out.println("Aa".hashCode() == "BB".hashCode()); // true
   map.put("Aa", 1);
   map.put("BB", 2);
   System.out.println(map.get("Aa")); // 1
   System.out.println(map.size());    // 2
```

DRAW THE IDEA



Watch out: A hash collision does not mean the map overwrites an unequal key.

INTERVIEWER FOLLOW-UP - ANSWERED

Why are capacity values powers of two in the implementation?

Power-of-two capacity lets HashMap choose a bucket with (capacity - 1) & hash and supports efficient resizing. This is an implementation detail, not an API guarantee.

Original source: Core Java Zero to Hero, page 46 · Q057 · Interview study edition

Q058 What changed in HashMap collision handling in Java 8?

20-SECOND ANSWER

Java 8 HashMap can treeify heavily collided buckets, but resizing and implementation thresholds affect when it does so.

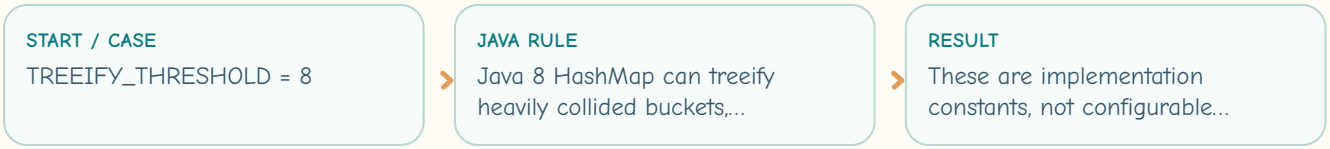
WHY IT WORKS · EASY EXPLANATION

These are implementation constants, not configurable API guarantees or legal variable declarations here. A threshold of 8 should not be memorized as an unconditional 'the eighth put creates a tree' rule. Table size and insertion path matter.

JAVA EXAMPLE / SCENARIO

```
// OpenJDK HashMap implementation constants:
   TREEIFY_THRESHOLD = 8
   UNTREEIFY_THRESHOLD = 6
   MIN_TREEIFY_CAPACITY = 64
   // A small table generally resizes instead of treeifying.
```

DRAW THE IDEA



Watch out: Avoid promising that every put at exactly eight entries always creates a tree.

INTERVIEWER FOLLOW-UP - ANSWERED

Is every adversarial equal-hash lookup guaranteed logarithmic for arbitrary key types?

No. Tree bins have thresholds and comparable-key behaviors; arbitrary hostile collisions do not guarantee logarithmic behavior in every case.

Original source: Core Java Zero to Hero, page 47 · Q058 · Interview study edition

Q059 How do load factor and capacity affect resizing?

20-SECOND ANSWER

A HashMap resizes when its size exceeds the threshold determined by capacity and load factor.

WHY IT WORKS · EASY EXPLANATION

A lower load factor spends more space to reduce average bucket occupancy. Pre-sizing for expected entries can avoid repeated growth. OpenJDK rounds capacity to a power of two; constructor allocation is normally lazy.

JAVA EXAMPLE / SCENARIO

```
Map<Integer, String> map = new HashMap<>(16, 0.75f);
   // Once allocated, capacity 16 usually has threshold 12.
   for (int i = 0; i < 13; i++) map.put(i, "v");
   // The thirteenth mapping exceeds 12 and triggers growth.
```

DRAW THE IDEA

START / CASE
Map<Integer, String> map = new HashMap<>(16,...

JAVA RULE
A HashMap resizes when its size exceeds the threshold...

RESULT
A lower load factor spends more space to reduce average...

Watch out: new HashMap<>(100) does not mean exactly 100 buckets or a maximum of 100 entries.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can iteration cost depend on both size and capacity?

HashMap iteration walks its internal capacity table and entries, so excess capacity can increase iteration work even when few mappings are stored.

Original source: Core Java Zero to Hero, page 47 · Q059 · Interview study edition

Q060 Can HashMap contain null keys and values?

20-SECOND ANSWER

HashMap permits one null key and null values; get alone cannot distinguish a missing key from a null mapping.

WHY IT WORKS · EASY EXPLANATION

Both lookups return null, but containsKey identifies the stored mapping. A null key is permitted once because keys are unique. This differs from ConcurrentHashMap's no-null contract.

JAVA EXAMPLE / SCENARIO

```
Map<String, String> map = new HashMap<>();
   map.put("x", null);
   System.out.println(map.get("x"));         // null
   System.out.println(map.get("missing"));   // null
   System.out.println(map.containsKey("x")); // true
   map.put(null, "default");
```

DRAW THE IDEA

START / CASE
Map<String, String> map = new HashMap<>();

JAVA RULE
HashMap permits one null key and null values; get alone...

RESULT
Both lookups return null, but containsKey identifies...

Watch out: ConcurrentHashMap disallows null keys and null values.

INTERVIEWER FOLLOW-UP - ANSWERED

When does computeIfAbsent call its function for a key mapped to null?

It invokes the function when the key is absent or currently maps to null. A non-null existing value is returned without invoking the mapping function.

Original source: Core Java Zero to Hero, page 48 · Q060 · Interview study edition

Q061 How do HashMap, LinkedHashMap and TreeMap differ?

20-SECOND ANSWER

HashMap supports general lookup, LinkedHashMap adds encounter order, and TreeMap provides sorted keys and range operations.

WHY IT WORKS · EASY EXPLANATION

floorEntry finds the greatest key not above 15. LinkedHashMap preserves insertion order by default and can optionally track access order. HashMap supplies neither guarantee.

JAVA EXAMPLE / SCENARIO

```
NavigableMap<Integer, String> prices = new TreeMap<>();
   prices.put(10, "old"); prices.put(20, "new");
   System.out.println(prices.floorEntry(15)); // 10=old
   Map<String, Integer> ordered = new LinkedHashMap<>();
   ordered.put("B", 2); ordered.put("A", 1);
   System.out.println(ordered.keySet()); // [B, A]
```

DRAW THE IDEA

START / CASE
NavigableMap<Integer, String> prices = new...

JAVA RULE
HashMap supports general lookup, LinkedHashMap adds...

RESULT
floorEntry finds the greatest key not above 15.

Watch out: TreeMap comparison determines key equivalence.

INTERVIEWER FOLLOW-UP - ANSWERED

What changes when accessOrder is true in LinkedHashMap?

An access (such as get) moves an entry to the end of access order, enabling LRU eviction. Iteration order now reflects use, not only insertion.

Original source: Core Java Zero to Hero, page 48 · Q061 · Interview study edition

Q062 How do putIfAbsent, computeIfAbsent and merge differ?

20-SECOND ANSWER

putIfAbsent supplies a value, computeIfAbsent derives a missing value, and merge combines an incoming value with an existing one.

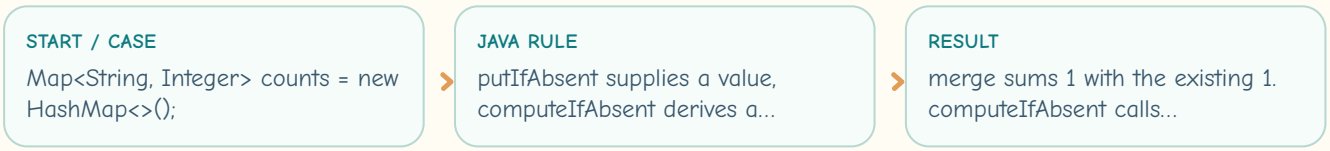
WHY IT WORKS · EASY EXPLANATION

merge sums 1 with the existing 1. computeIfAbsent calls its function because kafka has no non-null mapping. Atomicity depends on the implementation; HashMap does not make these concurrent operations safe.

JAVA EXAMPLE / SCENARIO

```
Map<String, Integer> counts = new HashMap<>();
   counts.putIfAbsent("java", 1);
   counts.merge("java", 1, Integer::sum);
   counts.computeIfAbsent("kafka", k -> k.length());
   System.out.println(counts.get("java"));  // 2
   System.out.println(counts.get("kafka")); // 5
```

DRAW THE IDEA



Watch out: Do not perform recursive updates on the same map inside a remapping function.

INTERVIEWER FOLLOW-UP - ANSWERED

Why should a concurrent remapping function be short and side-effect conscious?

It can execute under map coordination or be retried depending on API. Keep it short and avoid external side effects, blocking calls and recursive updates.

Original source: Core Java Zero to Hero, page 49 · Q062 · Interview study edition

Q063 Why is HashMap unsafe for concurrent updates?

20-SECOND ANSWER

HashMap has no concurrent-update safety; visibility and compound operations need external coordination or a concurrent alternative.

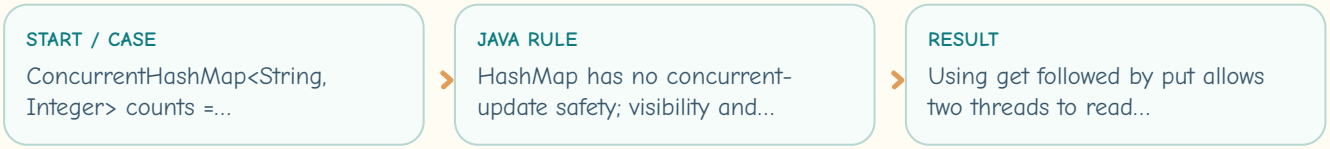
WHY IT WORKS · EASY EXPLANATION

Using get followed by put allows two threads to read the same old value and overwrite one increment. ConcurrentHashMap.compute coordinates one key's update, unlike the same sequence on a plain map.

JAVA EXAMPLE / SCENARIO

```
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();
   counts.put("java", 0);
   counts.compute("java", (k, v) -> v + 1);
   // Atomic per-key update; repeating from two tasks preserves both updates.
```

DRAW THE IDEA



Watch out: Safe concurrent reading requires a properly published map that is not being unsafely mutated.

INTERVIEWER FOLLOW-UP - ANSWERED

Would Collections.synchronizedMap make an entire iteration safe automatically?

No. Synchronize on the map around the complete traversal and use its documented iterator discipline, or take a safe snapshot first.

Original source: Core Java Zero to Hero, page 49 · Q063 · Interview study edition

Q064 How would you implement a bounded LRU cache in core Java?

20-SECOND ANSWER

An access-order LinkedHashMap can implement size-bounded LRU eviction; concurrent use needs synchronization.

WHY IT WORKS · EASY EXPLANATION

get(1) makes key 1 recently accessed. Inserting 3 exceeds the limit, so the least recently used key 2 is removed. Reads can modify access order and must be included in any locking policy.

JAVA EXAMPLE / SCENARIO

```
Map<Integer, String> cache = new LinkedHashMap<Integer, String>(16, 0.75f, true) {
       @Override protected boolean removeEldestEntry(Map.Entry<Integer, String> e) {
           return size() > 2;
       }
   };
   cache.put(1, "A"); cache.put(2, "B"); cache.get(1);
   cache.put(3, "C");
   System.out.println(cache.keySet()); // [1, 3]
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
Map<Integer, String> cache = new...	An access-order LinkedHashMap can implement...	get(1) makes key 1 recently accessed.

Watch out: A basic LRU exercise does not implement TTL, request coalescing or concurrent cache guarantees.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you stop several threads from loading the same missing key?

Use atomic per-key loading, such as ConcurrentHashMap.computeIfAbsent when appropriate, or cache a shared Future per key. Define failure, expiry and duplicate-load policy.

Original source: Core Java Zero to Hero, page 50 · Q064 · Interview study edition

MODULE 09 / 21

09

Concurrent collections

ConcurrentHashMap provides coordinated concurrent operations; synchronizedMap serializes operations through its wrapper's lock.

8 interview questions · clear examples · follow-up answers

Q065 How does ConcurrentHashMap differ from synchronizedMap?

20-SECOND ANSWER

ConcurrentHashMap provides coordinated concurrent operations; synchronizedMap serializes operations through its wrapper's lock.

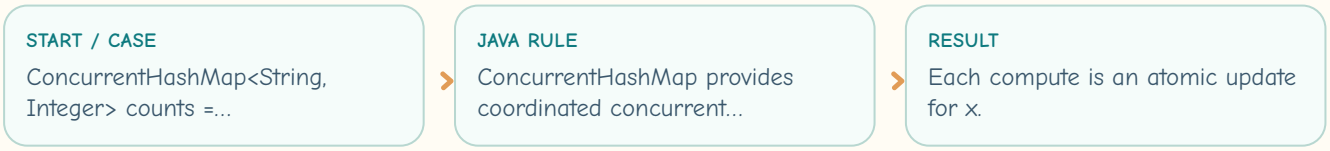
WHY IT WORKS · EASY EXPLANATION

Each compute is an atomic update for x. With synchronizedMap, multi-call check-then-update logic still needs an outer synchronized block on the wrapper. Its iteration also requires documented external synchronization.

JAVA EXAMPLE / SCENARIO

```
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();
   counts.compute("x", (k, v) -> v == null ? 1 : v + 1);
   counts.compute("x", (k, v) -> v == null ? 1 : v + 1);
   System.out.println(counts.get("x")); // 2
```

DRAW THE IDEA



Watch out: Java 8+ ConcurrentHashMap should not be explained as a fixed array of the old Java 7 segments.

INTERVIEWER FOLLOW-UP - ANSWERED

Which option is simplest for a small map with one compound invariant?

A single synchronized block or lock around the whole map operation is often simplest. It protects the compound invariant without complex per-entry coordination.

Original source: Core Java Zero to Hero, page 51 · Q065 · Interview study edition

Q066 What does a weakly consistent iterator guarantee?

20-SECOND ANSWER

A weakly consistent iterator tolerates concurrent changes without promising a whole-collection snapshot.

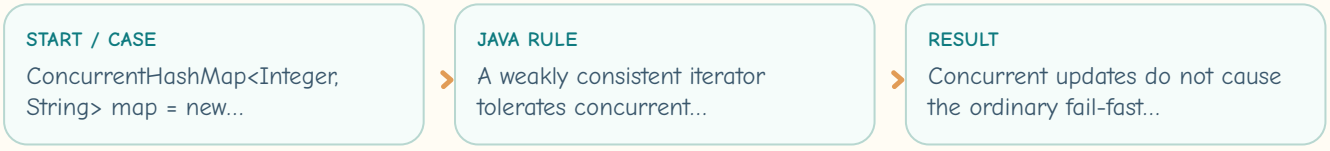
WHY IT WORKS · EASY EXPLANATION

Concurrent updates do not cause the ordinary fail-fast behavior of a HashMap iterator. Traversal can observe some changes without presenting every entry from one single instant.

JAVA EXAMPLE / SCENARIO

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<>();
   map.put(1, "A");
   Iterator<Integer> it = map.keySet().iterator();
   map.put(2, "B");
   while (it.hasNext()) System.out.println(it.next());
   // Do not promise whether this iteration observes key 2.
```

DRAW THE IDEA



Watch out: Weak consistency does not mean arbitrary memory corruption or a stable snapshot.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you obtain a transactionally consistent total across entries?

Use one lock/transaction covering the entries or an immutable snapshot published atomically. Iterating a concurrent map alone is not a transactional snapshot.

Original source: Core Java Zero to Hero, page 52 · Q066 · Interview study edition

Q067 Why does ConcurrentHashMap reject null?

20-SECOND ANSWER

ConcurrentHashMap forbids null keys and values so null retrieval has an unambiguous absence meaning.

WHY IT WORKS · EASY EXPLANATION

In a changing map, a separate containsKey check could observe a different state from get. Rejecting null values makes the individual get result easier to interpret.

JAVA EXAMPLE / SCENARIO

```
ConcurrentHashMap<String, String> map = new ConcurrentHashMap<>();
   System.out.println(map.get("x")); // null: no observed mapping
   // map.put("x", null);           // NullPointerException
   // map.put(null, "A");           // NullPointerException
```

DRAW THE IDEA

START / CASE
ConcurrentHashMap<String, String> map = new...

JAVA RULE
ConcurrentHashMap forbids null keys and values so null...

RESULT
In a changing map, a separate containsKey check could...

Watch out: An absent observation can become outdated immediately after the operation.

INTERVIEWER FOLLOW-UP - ANSWERED

Does a non-null get result imply the entire map is in one consistent snapshot?

No. The returned mapping is safely obtained, but other keys may change before or after it. A global consistent snapshot needs another mechanism.

Original source: Core Java Zero to Hero, page 52 · Q067 · Interview study edition

Q068 When is CopyOnWriteArrayList appropriate?

20-SECOND ANSWER

CopyOnWriteArrayList suits frequent reads and rare writes; each iterator traverses a structural snapshot.

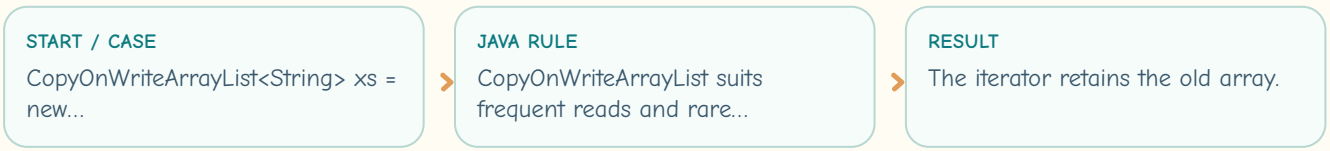
WHY IT WORKS · EASY EXPLANATION

The iterator retains the old array. The write creates a new array containing A and B for later readers. Copying every write is expensive, and mutable element state is not frozen.

JAVA EXAMPLE / SCENARIO

```
CopyOnWriteArrayList<String> xs = new CopyOnWriteArrayList<>();
   xs.add("A");
   Iterator<String> it = xs.iterator();
   xs.add("B");
   while (it.hasNext()) System.out.println(it.next()); // A only
```

DRAW THE IDEA



Watch out: It is a poor default for a large list with frequent writes.

INTERVIEWER FOLLOW-UP - ANSWERED

Can mutating a listener object still require synchronization?

Yes. CopyOnWriteArrayList protects its array structure, not the mutable listener object's fields. Synchronize those fields or make the listener immutable.

Original source: Core Java Zero to Hero, page 53 · Q068 · Interview study edition

Q069 How does BlockingQueue help producer-consumer code?

20-SECOND ANSWER

A bounded BlockingQueue coordinates producer-consumer handoff and applies backpressure when full.

WHY IT WORKS · EASY EXPLANATION

put waits for capacity; take waits for an item. offer without a timeout returns immediately. A real worker loop must also define interruption and termination rather than wait forever after producers finish.

JAVA EXAMPLE / SCENARIO

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(1);
   queue.put("A");
   System.out.println(queue.offer("B")); // false: full
   System.out.println(queue.take());     // A
   queue.put("B");                       // Now space exists
```

DRAW THE IDEA

START / CASE
BlockingQueue<String> queue = new...

JAVA RULE
A bounded BlockingQueue coordinates producer-consumer...

RESULT
put waits for capacity; take waits for an item. offer...

Watch out: An unbounded queue can hide overload until memory is exhausted.

INTERVIEWER FOLLOW-UP - ANSWERED

How does a poison-pill protocol change with several consumers?

Use one poison marker per consumer or a documented shutdown/coordinator protocol; one marker may stop only one consumer and leave others waiting.

Original source: Core Java Zero to Hero, page 53 · Q069 · Interview study edition

Q070 Should you use AtomicLong or LongAdder for counters?

20-SECOND ANSWER

AtomicLong fits exact atomic counter operations; LongAdder fits highly contended statistics whose concurrent sum may be approximate.

WHY IT WORKS · EASY EXPLANATION

AtomicLong exposes a single atomic current value and compare-and-set. LongAdder spreads contention across internal cells. Its sum is not an atomic snapshot of updates occurring concurrently.

JAVA EXAMPLE / SCENARIO

```
AtomicLong exact = new AtomicLong();
   System.out.println(exact.incrementAndGet()); // 1
   LongAdder metric = new LongAdder();
   metric.increment(); metric.increment();
   System.out.println(metric.sum()); // 2 when no updates overlap
```

DRAW THE IDEA

START / CASE
AtomicLong exact = new AtomicLong();

JAVA RULE
AtomicLong fits exact atomic counter operations;...

RESULT
AtomicLong exposes a single atomic current value and...

Watch out: LongAdder is unsuitable for reserving unique sequential identifiers with sum().

INTERVIEWER FOLLOW-UP - ANSWERED

How would you enforce a strict maximum number of admitted requests?

Acquire a semaphore permit or an atomic admission token before accepting, release it exactly once on terminal completion, and define rejection behavior.

Original source: Core Java Zero to Hero, page 54 · Q070 · Interview study edition

Q071 What are the limits of computeIfAbsent for cache loading?

20-SECOND ANSWER

computeIfAbsent is useful for short atomic initialization, but slow loading needs explicit execution, failure, and cache-lifecycle policies.

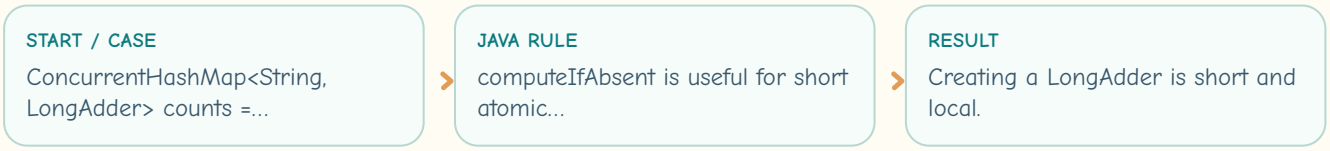
WHY IT WORKS · EASY EXPLANATION

Creating a LongAdder is short and local. A remote call can hold up competing map updates and should not be hidden inside a long mapping computation. Failed loads and eviction require separate decisions.

JAVA EXAMPLE / SCENARIO

```
ConcurrentHashMap<String, LongAdder> counts = new ConcurrentHashMap<>();
   counts.computeIfAbsent("java", k -> new LongAdder()).increment();
   // For slow I/O, reserve an in-flight future atomically,
   // then perform the load outside the map computation.
```

DRAW THE IDEA



Watch out: Do not equate one atomic insertion with an end-to-end exactly-once external call.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens when the loader throws and the next request retries?

A failed computeIfAbsent does not establish a value; a later caller may invoke the loader again. Ensure loaders are idempotent and handle repeated failure appropriately.

Original source: Core Java Zero to Hero, page 54 · Q071 · Interview study edition

Q072 Can a concurrent collection protect mutable values?

20-SECOND ANSWER

A concurrent map protects its mappings, not arbitrary mutable objects stored as values.

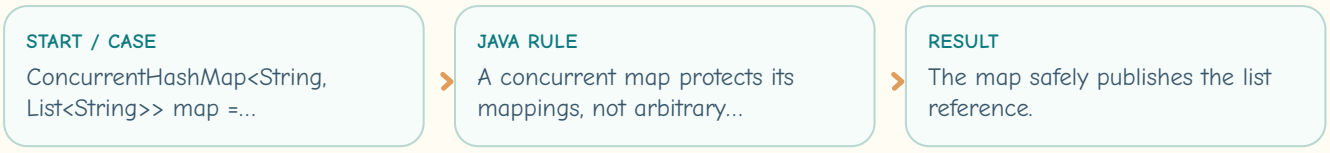
WHY IT WORKS · EASY EXPLANATION

The map safely publishes the list reference. ArrayList's internal array and size are still mutable without synchronization. Replacing values with immutable snapshots is another possible design.

JAVA EXAMPLE / SCENARIO

```
ConcurrentHashMap<String, List<String>> map = new ConcurrentHashMap<>();
   map.put("team", new ArrayList<>());
   // Concurrent map.get("team").add(...) is still unsafe.
   // Choose a concurrent value collection or protect every access.
```

DRAW THE IDEA



Watch out: Replacing one map value atomically differs from mutating a value in place.

INTERVIEWER FOLLOW-UP - ANSWERED

Would map.compute protect list readers that access the list outside compute?

No. compute coordinates the map entry update, but readers later mutating or reading a shared ArrayList require their own thread-safe protocol.

Original source: Core Java Zero to Hero, page 55 · Q072 · Interview study edition

MODULE 10 / 21

10

Lambdas, functional interfaces and Optional

A functional interface has one abstract method contract; default and static methods do not count toward that total.

8 interview questions · clear examples · follow-up answers

Q073 What makes an interface functional?

20-SECOND ANSWER

A functional interface has one abstract method contract; default and static methods do not count toward that total.

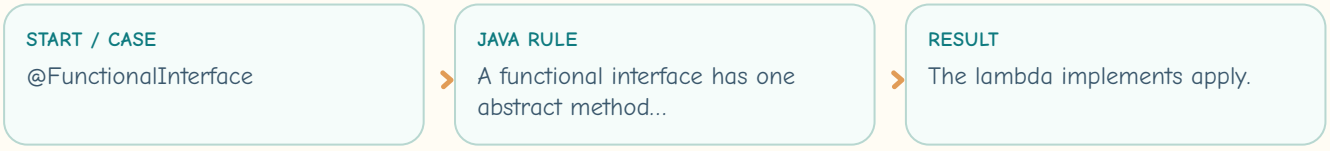
WHY IT WORKS · EASY EXPLANATION

The lambda implements apply. The default method already has an implementation, so it creates no additional abstract obligation. @FunctionalInterface asks the compiler to check the intended contract.

JAVA EXAMPLE / SCENARIO

```
@FunctionalInterface
   interface Calculator {
       int apply(int a, int b);
       default String name() { return "calculator"; }
   }
   Calculator add = (a, b) -> a + b;
   System.out.println(add.apply(2, 3)); // 5
```

DRAW THE IDEA



Watch out: Two separately named abstract methods normally prevent lambda targeting.

INTERVIEWER FOLLOW-UP - ANSWERED

Can a functional interface have several default methods?

Yes. Only one abstract method counts toward the functional-interface requirement; it may also contain multiple default and static methods.

Original source: Core Java Zero to Hero, page 56 · Q073 · Interview study edition

Q074 How do Predicate, Function, Consumer and Supplier differ?

20-SECOND ANSWER

Predicate tests, Function transforms, Consumer performs an action, and Supplier produces a value.

WHY IT WORKS · EASY EXPLANATION

Match the interface to the operation's input and output. Predicate returns boolean, Function returns a transformed result, Consumer returns no result, and Supplier takes no argument.

JAVA EXAMPLE / SCENARIO

```
Predicate<String> valid = s -> !s.isEmpty();
   Function<String, Integer> length = String::length;
   Consumer<String> print = System.out::println;
   Supplier<String> defaultName = () -> "Sai";
   System.out.println(valid.test("Java"));   // true
   System.out.println(length.apply("Java")); // 4
   print.accept(defaultName.get());          // Sai
```

DRAW THE IDEA

START / CASE
Predicate<String> valid = s -> ! s.isEmpty();

JAVA RULE
Predicate tests, Function transforms, Consumer performs...

RESULT
Match the interface to the operation's input and output.

Watch out: Consumer represents an action, so using it in a parallel pipeline can expose shared-state races.

INTERVIEWER FOLLOW-UP - ANSWERED

When does a BiFunction fit better than Function?

Use BiFunction<A,B,R> when the computation naturally needs two inputs, such as combining old and new values in a merge operation.

Original source: Core Java Zero to Hero, page 57 · Q074 · Interview study edition

Q075 How are lambdas different from anonymous classes?

20-SECOND ANSWER

A lambda uses the enclosing meaning of this; an anonymous class has its own this and class body.

WHY IT WORKS · EASY EXPLANATION

The lambda does not introduce a new enclosing-object identity. An anonymous class can declare fields and additional methods and refers to its own instance through this.

JAVA EXAMPLE / SCENARIO

```
class Demo {
       void show() {
           Runnable lambda = () -> System.out.println(this.getClass().getSimpleName());
           lambda.run(); // Demo
           // In an anonymous Runnable, this refers to that Runnable.
       }
   }
```

DRAW THE IDEA

START / CASE
class Demo {

JAVA RULE
A lambda uses the enclosing meaning of this; an...

RESULT
The lambda does not introduce a new enclosing-object...

Watch out: Do not use a lambda object as an identity-sensitive lock.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you express recursive behavior without assuming a lambda can name itself?

Use a named helper method, a class implementing the functional interface, or a holder assigned before use with care; avoid self-reference to an uninitialized local lambda.

Original source: Core Java Zero to Hero, page 57 · Q075 · Interview study edition

Q076 When can you use a method reference?

20-SECOND ANSWER

Use a method reference when an existing method or constructor matches the target functional signature.

WHY IT WORKS · EASY EXPLANATION

String::length uses the input String as the receiver. System.out::println binds an existing receiver. ArrayList::new invokes a constructor when the supplier is called.

JAVA EXAMPLE / SCENARIO

```
Function<String, Integer> length = String::length;
   Consumer<String> print = System.out::println;
   Supplier<List<String>> create = ArrayList::new;
   print.accept("Length: " + length.apply("Java")); // Length: 4
```

DRAW THE IDEA

START / CASE
Function<String, Integer> length =...

JAVA RULE
Use a method reference when an existing method or...

RESULT
String::length uses the input String as the receiver.

Watch out: A bound null receiver can fail when creating the method reference.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can a lambda defer a receiver lookup differently from a bound method reference?

A bound reference such as service::load captures one receiver now; a lambda () -> service.load() can read a changing field each time, subject to capture rules.

Original source: Core Java Zero to Hero, page 58 · Q076 · Interview study edition

Q077 What problem does Optional solve?

20-SECOND ANSWER

Optional makes an absent result explicit and encourages callers to choose a deliberate absence policy.

WHY IT WORKS · EASY EXPLANATION

The caller supplies a default or acts only when present. Optional.of rejects null; ofNullable can represent a nullable input. Avoid get without checking or otherwise guaranteeing presence.

JAVA EXAMPLE / SCENARIO

```
Optional<String> name = Optional.empty();
   System.out.println(name.orElse("Guest")); // Guest
   Optional<String> present = Optional.of("Sai");
   present.ifPresent(System.out::println);  // Sai
```

DRAW THE IDEA

START / CASE
Optional<String> name = Optional.empty();

JAVA RULE
Optional makes an absent result explicit and encourages...

RESULT
The caller supplies a default or acts only when present.

Watch out: Optional.get without establishing presence recreates an unsafe access pattern.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is an empty list usually clearer than Optional<List<T>> for a normal collection query?

An empty list naturally means zero results and avoids nested absence checks. Reserve Optional for meaningfully absent single values or distinct states.

Original source: Core Java Zero to Hero, page 58 · Q077 · Interview study edition

Q078 How do orElse and orElseGet differ?

20-SECOND ANSWER

orElse evaluates its argument eagerly; orElseGet calls its supplier only when the Optional is empty.

WHY IT WORKS · EASY EXPLANATION

Java evaluates method arguments before invoking orElse, even when its value is unused. orElseGet receives a deferred function and invokes it only for absence.

JAVA EXAMPLE / SCENARIO

```
static String fallback() {
       System.out.println("loading");
       return "Guest";
   }
   Optional<String> name = Optional.of("Sai");
   name.orElse(fallback());          // Prints loading
   name.orElseGet(() -> fallback()); // Does not call fallback
```

DRAW THE IDEA

START / CASE
static String fallback() {

JAVA RULE
orElse evaluates its argument eagerly; orElseGet calls...

RESULT
Java evaluates method arguments before invoking orElse,...

Watch out: Passing a lambda to orElseGet does not execute its body immediately.

INTERVIEWER FOLLOW-UP - ANSWERED

How does orElseThrow express a missing required value?

orElseThrow() signals that the required value is missing by throwing NoSuchElementException; the supplier overload gives a domain-specific error with details.

Original source: Core Java Zero to Hero, page 59 · Q078 · Interview study edition

Q079 How do Optional.map and flatMap differ?

20-SECOND ANSWER

Optional.map transforms a contained value; flatMap avoids nesting when the transformation already returns Optional.

WHY IT WORKS · EASY EXPLANATION

map wraps the transformation result. A function returning Optional would yield Optional<Optional<T>> with map. flatMap uses the returned Optional directly; its mapper must not return null.

JAVA EXAMPLE / SCENARIO

```
Optional<String> name = Optional.of("Sai");
   Optional<Integer> length = name.map(String::length);
   Optional<String> result = name.flatMap(s -> Optional.of(s.toUpperCase(Locale.ROOT)));
   System.out.println(length.get()); // 3
   System.out.println(result.get()); // SAI
```

DRAW THE IDEA

START / CASE
Optional<String> name = Optional.of("Sai");

JAVA RULE
Optional.map transforms a contained value; flatMap...

RESULT
map wraps the transformation result.

Watch out: Optional<Optional<Address>> often indicates that map was used where flatMap was intended.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you distinguish a missing user from a missing address when the API needs separate errors?

Model the stages separately: Optional<User> for absent user, then validate the address and throw a different domain error when it is missing.

Original source: Core Java Zero to Hero, page 59 · Q079 · Interview study edition

Q080 How should functions compose and handle failures?

20-SECOND ANSWER

Function composition states processing order; exceptions still need an explicit failure contract.

WHY IT WORKS · EASY EXPLANATION

andThen runs parse before doubleIt. compose reverses which function is applied first. Chaining does not automatically recover from malformed input or checked failures.

JAVA EXAMPLE / SCENARIO

```
Function<String, Integer> parse = Integer::parseInt;
   Function<Integer, Integer> doubleIt = n -> n * 2;
   System.out.println(parse.andThen(doubleIt).apply("21")); // 42
   // Applying it to "bad" throws NumberFormatException.
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
Function<String, Integer> parse =...	Function composition states processing order;...	andThen runs parse before doubleIt. compose reverses...

Watch out: A function pipeline does not automatically add retries, isolation or transactions.

INTERVIEWER FOLLOW-UP - ANSWERED

When would a Result<T,E> type be clearer than throwing during a batch import?

Use Result<T,E> when batch rows should carry expected failures as data so processing continues, while exceptional infrastructure failures still have distinct handling.

Original source: Core Java Zero to Hero, page 60 · Q080 · Interview study edition

MODULE 11 / 21

11

Streams fundamentals

A collection stores values; a stream is a one-use processing pipeline whose intermediate operations are normally lazy.

8 interview questions · clear examples · follow-up answers

Q081 How is a stream different from a collection?

20-SECOND ANSWER

A collection stores values; a stream is a one-use processing pipeline whose intermediate operations are normally lazy.

WHY IT WORKS · EASY EXPLANATION

filter constructs the pipeline. count triggers evaluation and consumes the stream. The collection remains reusable, so create a new stream for another traversal.

JAVA EXAMPLE / SCENARIO

```
List<String> xs = Arrays.asList("A", "BB");
   Stream<String> stream = xs.stream().filter(s -> s.length() > 1);
   System.out.println(stream.count()); // 1
   // stream.count(); // IllegalStateException: already consumed
   System.out.println(xs.size());      // 2: source still exists
```

DRAW THE IDEA

START / CASE
List<String> xs = Arrays.asList("A", "BB");

JAVA RULE
A collection stores values; a stream is a one-use...

RESULT
filter constructs the pipeline. count triggers...

Watch out: A stream pipeline alone does not create a background thread.

INTERVIEWER FOLLOW-UP - ANSWERED

Which terminal operations can finish on an infinite stream?

Short-circuit terminals like findFirst(), anyMatch() or limit(n).count() can terminate if enough elements arrive; sum(), count() on an unbounded source cannot.

Original source: Core Java Zero to Hero, page 61 · Q081 · Interview study edition

Q082 What is the difference between map and flatMap?

20-SECOND ANSWER

map transforms each element into one result; flatMap expands each element and flattens the resulting streams.

WHY IT WORKS · EASY EXPLANATION

Each inner list contributes a stream. flatMap combines their elements into one stream of Integer. map(List::stream) would instead produce a stream containing streams.

JAVA EXAMPLE / SCENARIO

```
List<List<Integer>> groups = Arrays.asList(Arrays.asList(1, 2), Arrays.asList(3));
   List<Integer> all = groups.stream().flatMap(List::stream)
           .collect(Collectors.toList());
   System.out.println(all); // [1, 2, 3]
```

DRAW THE IDEA

START / CASE
List<List<Integer>> groups =...

JAVA RULE
map transforms each element into one result; flatMap...

RESULT
Each inner list contributes a stream. flatMap combines...

Watch out: map(List::stream) produces a stream of streams rather than flattening it.

INTERVIEWER FOLLOW-UP - ANSWERED

When is mapMulti an alternative to flatMap on Java 16+?

mapMulti can emit zero, one or many downstream values without creating a Stream per input. It is often useful for simple expansion or filtering plus mapping.

Original source: Core Java Zero to Hero, page 62 · Q082 · Interview study edition

Q083 How do filter, distinct, sorted and limit affect a pipeline?

20-SECOND ANSWER

Pipeline order changes meaning: filter selects, distinct removes duplicates, sorted orders, and limit takes a prefix.

WHY IT WORKS · EASY EXPLANATION

Filtering leaves 3,2,2; distinct leaves 3,2; sorting gives 2,3; limit returns 2. Applying limit before sorting would select a different subset.

JAVA EXAMPLE / SCENARIO

```
List<Integer> xs = Arrays.asList(3, 2, 2, 1);
   List<Integer> result = xs.stream().filter(n -> n > 1)
           .distinct().sorted().limit(1).collect(Collectors.toList());
   System.out.println(result); // [2]
```

DRAW THE IDEA

START / CASE
List<Integer> xs = Arrays.asList(3, 2, 2, 1);

JAVA RULE
Pipeline order changes meaning: filter selects,...

RESULT
Filtering leaves 3,2,2; distinct leaves 3,2; sorting...

Watch out: Sorting an unbounded infinite stream cannot produce a complete sorted result.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can sorted().limit(k) be more expensive than maintaining a heap?

sorted() generally orders all n items (O(n log n)), while a size-k heap can keep only the best k in O(n log k) time and O(k) space.

Original source: Core Java Zero to Hero, page 62 · Q083 · Interview study edition

Q084 When should reduce be used instead of collect?

20-SECOND ANSWER

Use reduce for a value combined by an associative operation; use collect for mutable result containers.

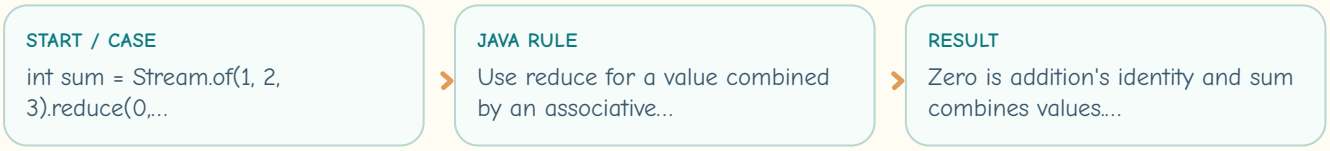
WHY IT WORKS · EASY EXPLANATION

Zero is addition's identity and sum combines values. collect owns container creation and merging. Mutating one shared list inside reduce can violate parallel reduction requirements.

JAVA EXAMPLE / SCENARIO

```
int sum = Stream.of(1, 2, 3).reduce(0, Integer::sum);
   List<Integer> copy = Stream.of(1, 2, 3).collect(Collectors.toList());
   System.out.println(sum);  // 6
   System.out.println(copy); // [1, 2, 3]
```

DRAW THE IDEA



Watch out: A parallel combiner must agree with the accumulation semantics.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is subtraction unsuitable as a parallel associative reduction?

Parallel reduction can regroup operands. Subtraction is not associative: (a-b)-c differs from a-(b-c), so it is not a valid general combiner.

Original source: Core Java Zero to Hero, page 63 · Q084 · Interview study edition

Q085 How do findFirst and findAny differ?

20-SECOND ANSWER

findFirst respects encounter order when defined; findAny may return any matching element.

WHY IT WORKS · EASY EXPLANATION

The ordered list gives findFirst a defined first match. findAny can avoid preserving that choice. Neither operation invents ordering for an unordered source.

JAVA EXAMPLE / SCENARIO

```
List<Integer> xs = Arrays.asList(4, 2, 6);
   System.out.println(xs.stream().filter(n -> n % 2 == 0).findFirst().get()); // 4
   Optional<Integer> any = xs.parallelStream().filter(n -> n % 2 == 0).findAny();
   // any can contain 4, 2, or 6; do not promise one output.
```

DRAW THE IDEA

START / CASE
List<Integer> xs = Arrays.asList(4, 2, 6);

JAVA RULE
findFirst respects encounter order when defined;...

RESULT
The ordered list gives findFirst a defined first match....

Watch out: Calling parallel does not erase an existing encounter order.

INTERVIEWER FOLLOW-UP - ANSWERED

When could unordered() improve a pipeline without changing the required result?

If encounter order is not part of the result, unordered() may relax coordination in distinct(), limit() or parallel operations. Never use it when order matters.

Original source: Core Java Zero to Hero, page 63 · Q085 · Interview study edition

Q086 Why is modifying the source during stream processing risky?

20-SECOND ANSWER

Stream functions should not interfere with the source or race through shared mutable output.

WHY IT WORKS · EASY EXPLANATION

The collector coordinates partial containers. Using parallel forEach with ArrayList.add on one shared output list would race. Modifying the source during traversal also violates the intended stream contract.

JAVA EXAMPLE / SCENARIO

```
List<Integer> xs = Arrays.asList(1, 2, 3);
   List<Integer> doubled = xs.parallelStream().map(n -> n * 2)
           .collect(Collectors.toList());
   System.out.println(doubled); // [2, 4, 6]
```

DRAW THE IDEA

START / CASE
List<Integer> xs = Arrays.asList(1, 2, 3);

JAVA RULE
Stream functions should not interfere with the source...

RESULT
The collector coordinates partial containers.

Watch out: forEachOrdered preserves encounter-order actions but does not justify interfering with the source.

INTERVIEWER FOLLOW-UP - ANSWERED

What documented property lets a concurrent source support different behavior?

The Spliterator characteristics and documented concurrent collection semantics indicate whether traversal can tolerate concurrent updates; do not guess from a source name.

Original source: Core Java Zero to Hero, page 64 · Q086 · Interview study edition

Q087 Can you depend on peek for essential side effects?

20-SECOND ANSWER

peek is for observation, not required business side effects: evaluation can skip stages when the terminal result is already known.

WHY IT WORKS · EASY EXPLANATION

A sized source lets count determine the result without visiting each element. Do not put audit writes, payments, or other essential actions in peek. Use an explicit operation with a defined execution contract.

JAVA EXAMPLE / SCENARIO

```
long count = Arrays.asList("A", "B").stream()
           .peek(System.out::println).count();
   System.out.println(count); // 2
   // On some JDKs, A and B need not be printed.
```

DRAW THE IDEA

START / CASE
long count = Arrays.asList("A", "B").stream()

JAVA RULE
peek is for observation, not required business side...

RESULT
A sized source lets count determine the result without...

Watch out: A terminal operation does not guarantee every intermediate action executes for every source element.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you guarantee that each imported row receives an audit action?

Use an explicit terminal traversal or stream-to-collection processing whose completion and error policy you control. peek() is not a reliable audit-event contract.

Original source: Core Java Zero to Hero, page 64 · Q087 · Interview study edition

Q088 What is the difference between Stream.toList and Collectors.toList?

20-SECOND ANSWER

Stream.toList returns an unmodifiable list; Collectors.toList promises no specific mutability; toCollection can request ArrayList.

WHY IT WORKS · EASY EXPLANATION

Choose toCollection when callers require a mutable list. An unmodifiable structure does not make its elements immutable. Stream.toList is unavailable in Java 8.

JAVA EXAMPLE / SCENARIO

```
List<Integer> fixed = Stream.of(1, 2).toList(); // Java 16+
   // fixed.add(3); // UnsupportedOperationException
   List<Integer> mutable = Stream.of(1, 2)
           .collect(Collectors.toCollection(ArrayList::new));
   mutable.add(3);
   System.out.println(mutable); // [1, 2, 3]
```

DRAW THE IDEA

START / CASE
List<Integer> fixed = Stream.of(1,...

JAVA RULE
Stream.toList returns an unmodifiable list;...

RESULT
Choose toCollection when callers require a mutable list.

Watch out: Replacing collect(toList()) with toList() can change downstream mutability expectations.

INTERVIEWER FOLLOW-UP - ANSWERED

How is Collectors.toUnmodifiableList different in its treatment of nulls?

toUnmodifiableList rejects null elements and returns an unmodifiable result; Collectors.toList has no comparable immutability guarantee and generally accepts null.

Original source: Core Java Zero to Hero, page 65 · Q088 · Interview study edition

MODULE 12 / 21

12

Stream recipes and date-time

groupingBy forms groups for arbitrary keys; partitioningBy forms true and false groups for a predicate.

8 interview questions · clear examples · follow-up answers

Q089 How do groupingBy and partitioningBy differ?

20-SECOND ANSWER

groupingBy forms groups for arbitrary keys; partitioningBy forms true and false groups for a predicate.

WHY IT WORKS · EASY EXPLANATION

The predicate creates exactly two logical partitions. groupingBy can instead group by department, first letter, or another key and optionally aggregate each group's count or total.

JAVA EXAMPLE / SCENARIO

```
List<Integer> xs = Arrays.asList(1, 2, 3, 4);
   Map<Boolean, List<Integer>> parts = xs.stream()
           .collect(Collectors.partitioningBy(n -> n % 2 == 0));
   System.out.println(parts.get(true));  // [2, 4]
   System.out.println(parts.get(false)); // [1, 3]
```

DRAW THE IDEA

START / CASE
List<Integer> xs = Arrays.asList(1, 2, 3, 4);

JAVA RULE
groupingBy forms groups for arbitrary keys;...

RESULT
The predicate creates exactly two logical partitions....

Watch out: The default groupingBy map does not promise sorted or encounter-order keys.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you obtain department totals in a TreeMap?

Use groupingBy(Employee::department, TreeMap::new, summingLong(Employee::salary)) so department keys are ordered and totals aggregate correctly.

Original source: Core Java Zero to Hero, page 66 · Q089 · Interview study edition

Q090 How do you handle duplicate keys in Collectors.toMap?

20-SECOND ANSWER

Collectors.toMap needs a deliberate merge policy when two input elements produce equal keys.

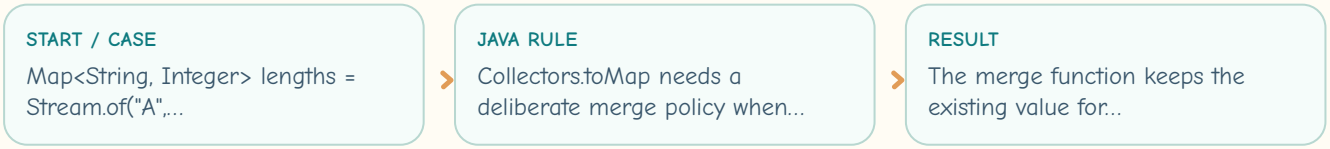
WHY IT WORKS · EASY EXPLANATION

The merge function keeps the existing value for duplicate A. The two-argument toMap would reject this duplicate key. Other domains may need sum, newest-value selection, or explicit rejection.

JAVA EXAMPLE / SCENARIO

```
Map<String, Integer> lengths = Stream.of("A", "A", "BB")
          .collect(Collectors.toMap(Function.identity(), String::length,
                                    (oldValue, newValue) -> oldValue));
   System.out.println(lengths.get("A")); // 1
```

DRAW THE IDEA



Watch out: Keeping the first element has meaning only if encounter order and the required policy agree.

INTERVIEWER FOLLOW-UP - ANSWERED

What should happen when two duplicate records share the same update timestamp?

Define a deterministic tie rule, such as an authoritative sequence, ID, or explicit conflict status; timestamps alone cannot safely choose the latest record.

Original source: Core Java Zero to Hero, page 67 · Q090 · Interview study edition

Q091 How would you find the second-highest distinct salary?

20-SECOND ANSWER

Remove duplicates before descending order, skip one, and represent a missing second distinct salary explicitly.

WHY IT WORKS · EASY EXPLANATION

The distinct descending values are 200,150,100. Skipping one yields 150. Empty or single-distinct-value input returns Optional.empty. For large data, consider a one-pass top-two algorithm.

JAVA EXAMPLE / SCENARIO

```
Optional<Integer> second = Stream.of(100, 200, 200, 150)
          .distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst();
   System.out.println(second.get()); // 150
```

DRAW THE IDEA

START / CASE
Optional<Integer> second = Stream.of(100,...

JAVA RULE
Remove duplicates before descending order, skip one,...

RESULT
The distinct descending values are 200,150,100.

Watch out: For BigDecimal, distinct uses equals, so different scales require an explicit policy.

INTERVIEWER FOLLOW-UP - ANSWERED

How does the solution change when ties should share a rank?

Use dense_rank-style logic where equal salaries share a rank; distinct() before ordering is one option if only unique salary levels matter.

Original source: Core Java Zero to Hero, page 67 · Q091 · Interview study edition

Q092 When does parallelStream help and when does it hurt?

20-SECOND ANSWER

Parallel streams can help sufficiently large, independent CPU work; overhead, blocking, and shared-pool contention can make them slower.

WHY IT WORKS · EASY EXPLANATION

The reduction is independent and associative for this bounded sum. It illustrates a suitable computation shape, not a guarantee of speedup. Remote I/O and small input need different performance reasoning.

JAVA EXAMPLE / SCENARIO

```
long sum = LongStream.rangeClosed(1, 1_000_000)
           .parallel().sum();
   System.out.println(sum); // 500000500000
   // Measure against sequential sum under representative load.
```

DRAW THE IDEA

START / CASE
long sum = LongStream.rangeClosed(1, 1_000_000)

JAVA RULE
Parallel streams can help sufficiently large,...

RESULT
The reduction is independent and associative for this...

Watch out: Parallel execution does not change the requirement for non-interference.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can nested parallel streams compete for the same workers?

Nested parallel streams may compete for shared fork-join resources, causing contention and oversubscription. Parallelize at a deliberate outer boundary.

Original source: Core Java Zero to Hero, page 68 · Q092 · Interview study edition

Q093 What requirements does a custom collector need to satisfy?

20-SECOND ANSWER

A collector needs consistent supplier, accumulator, combiner, and finisher behavior, with valid identity and associativity.

WHY IT WORKS · EASY EXPLANATION

Each partition gets its own list, adds values, then combines lists by appending. Claiming CONCURRENT would require a container and accumulation policy safe for shared concurrent calls.

JAVA EXAMPLE / SCENARIO

```
List<Integer> out = Stream.of(1, 2, 3).parallel().collect(
          ArrayList::new, List::add, List::addAll);
   System.out.println(out); // [1, 2, 3]
```

DRAW THE IDEA

START / CASE
List<Integer> out = Stream.of(1, 2,...

JAVA RULE
A collector needs consistent supplier, accumulator,...

RESULT
Each partition gets its own list, adds values, then...

Watch out: Marking an ArrayList accumulator CONCURRENT does not make concurrent adds safe.

INTERVIEWER FOLLOW-UP - ANSWERED

When can a concurrent collector actually share one result container?

Only when the collector has the CONCURRENT characteristic and its container supports safe concurrent accumulation; source ordering constraints still matter.

Original source: Core Java Zero to Hero, page 68 · Q093 · Interview study edition

Q094 How do Instant, LocalDateTime and ZonedDateTime differ?

20-SECOND ANSWER

Instant identifies a timeline point; LocalDateTime is a wall-clock value; ZonedDateTime includes regional time-zone rules.

WHY IT WORKS · EASY EXPLANATION

09:00 in India is 03:30 UTC. The local date-time alone does not specify an instant until a zone or offset is chosen. Regional zones also apply daylight-saving rules where relevant.

JAVA EXAMPLE / SCENARIO

```
LocalDateTime local = LocalDateTime.of(2026, 10, 8, 9, 0);
   ZonedDateTime india = local.atZone(ZoneId.of("Asia/Kolkata"));
   System.out.println(india.toInstant()); // 2026-10-08T03:30:00Z
```

DRAW THE IDEA

START / CASE
LocalDateTime local = LocalDateTime.of(2026,...

JAVA RULE
Instant identifies a timeline point; LocalDateTime is a...

RESULT
09:00 in India is 03:30 UTC.

Watch out: A system default zone can differ between a developer laptop and production.

INTERVIEWER FOLLOW-UP - ANSWERED

How can a daylight-saving overlap map one local time to two offsets?

During the autumn clock change, one local clock time can correspond to two instants with different offsets. Specify which offset or overlap policy to use.

Original source: Core Java Zero to Hero, page 69 · Q094 · Interview study edition

Q095 How do Duration and Period differ?

20-SECOND ANSWER

Duration models time-based amounts; Period models calendar-based years, months, and days.

WHY IT WORKS · EASY EXPLANATION

This date crosses a daylight-saving clock advance. One calendar day keeps noon; 24 elapsed hours reaches 13:00. Choose the amount according to the business rule.

JAVA EXAMPLE / SCENARIO

```
ZonedDateTime t = ZonedDateTime.of(2024, 3, 9, 12, 0, 0, 0,
                                   ZoneId.of("America/New_York"));
   System.out.println(t.plus(Period.ofDays(1)).getHour());   // 12
   System.out.println(t.plus(Duration.ofHours(24)).getHour()); // 13
```

DRAW THE IDEA

START / CASE
ZonedDateTime t = ZonedDateTime.of(2024, 3,...

JAVA RULE
Duration models time-based amounts; Period models...

RESULT
This date crosses a daylight-saving clock advance.

Watch out: A month is not a fixed number of seconds.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you model a subscription billed on the first day of each month?

Store the subscription's ZoneId and calendar rule, then calculate the next first-of-month local date in that zone. Convert to Instant only when executing.

Original source: Core Java Zero to Hero, page 69 · Q095 · Interview study edition

Q096 Is DateTimeFormatter thread safe?

20-SECOND ANSWER

DateTimeFormatter is immutable and thread safe; make pattern, locale, zone, and parsing strictness explicit.

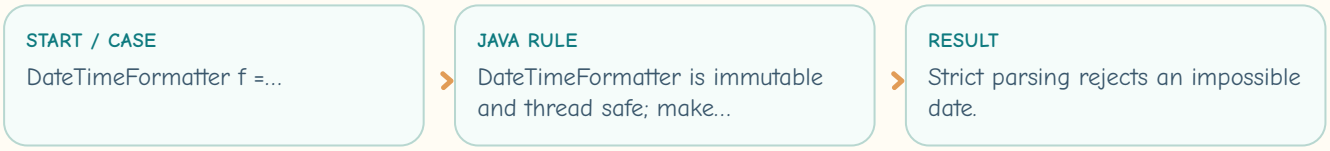
WHY IT WORKS · EASY EXPLANATION

Strict parsing rejects an impossible date. The same formatter can be safely shared. uuuu represents the proleptic year; yyyy is year-of-era and has different strict-resolution implications.

JAVA EXAMPLE / SCENARIO

```
DateTimeFormatter f = DateTimeFormatter.ofPattern("uuuu-MM-dd")
          .withResolverStyle(ResolverStyle.STRICT);
   System.out.println(LocalDate.parse("2024-02-29", f)); // 2024-02-29
   // LocalDate.parse("2023-02-29", f); // DateTimeParseException
```

DRAW THE IDEA



Watch out: Using a shared SimpleDateFormat without protection can corrupt concurrent parsing or formatting.

INTERVIEWER FOLLOW-UP - ANSWERED

Why should an application inject Clock rather than call now() everywhere?

Injecting Clock makes tests deterministic, removes environment dependence and lets every calculation in a workflow use a consistent notion of now.

Original source: Core Java Zero to Hero, page 70 · Q096 · Interview study edition

MODULE 13 / 21

13

Threads and cooperative cancellation

start launches a new thread; calling run directly executes the method on the current thread.

8 interview questions · clear examples · follow-up answers

Q097 What is the difference between start and run?

20-SECOND ANSWER

start launches a new thread; calling run directly executes the method on the current thread.

WHY IT WORKS · EASY EXPLANATION

run is an ordinary method call. start schedules a distinct thread to invoke run. A Thread object can be started only once; calling start again throws IllegalThreadStateException.

JAVA EXAMPLE / SCENARIO

```
Runnable task = () -> System.out.println(Thread.currentThread().getName());
   Thread worker = new Thread(task, "worker");
   worker.run();   // Current caller's name
   worker.start(); // worker
   worker.join();
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
Runnable task = () ->...	start launches a new thread; calling run directly...	run is an ordinary method call. start schedules a...

Watch out: Calling start twice throws IllegalThreadStateException.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is an Executor preferable to manually creating one platform thread per request?

Executors manage task queues, worker reuse, admission, failures and lifecycle centrally. Creating a platform thread per request is harder to bound under load.

Original source: Core Java Zero to Hero, page 71 · Q097 · Interview study edition

Q098 What are Java thread states?

20-SECOND ANSWER

Java thread states describe lifecycle and waiting categories, not exact CPU scheduler states.

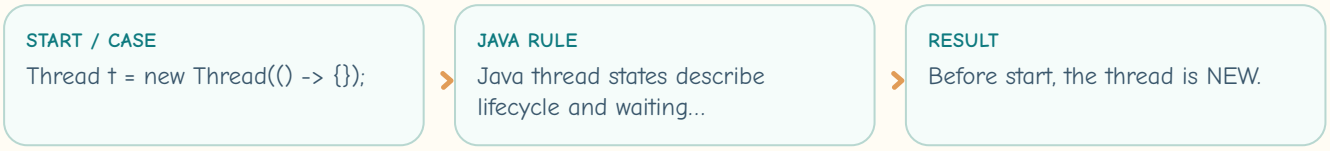
WHY IT WORKS · EASY EXPLANATION

Before start, the thread is NEW. After completion and join, it is TERMINATED. RUNNABLE can include executing or waiting for OS scheduling; BLOCKED specifically refers to waiting to enter a monitor.

JAVA EXAMPLE / SCENARIO

```
Thread t = new Thread(() -> {});
   System.out.println(t.getState()); // NEW
    t.start();
   t.join();
   System.out.println(t.getState()); // TERMINATED
   // Other states: RUNNABLE, BLOCKED, WAITING, TIMED_WAITING
```

DRAW THE IDEA



Watch out: Waiting for a ReentrantLock should not automatically be labeled monitor BLOCKED.

INTERVIEWER FOLLOW-UP - ANSWERED

What state would a thread often show during Thread.sleep?

TIMED_WAITING is typical because sleep waits for a bounded duration. It does not release a monitor already held.

Original source: Core Java Zero to Hero, page 72 · Q098 · Interview study edition

Q099 How do sleep, wait and join differ?

20-SECOND ANSWER

sleep pauses without releasing held monitors; wait releases its monitor; join waits for another thread to finish.

WHY IT WORKS · EASY EXPLANATION

wait requires owning the target monitor and reacquires it before returning. sleep has no monitor-ownership requirement. These blocking calls can throw InterruptedException.

JAVA EXAMPLE / SCENARIO

```
synchronized (lock) {
       Thread.sleep(100); // Still owns lock
       while (!ready) lock.wait(); // Releases lock while waiting
   }
   worker.join(); // Waits for worker termination
```

DRAW THE IDEA

START / CASE
synchronized (lock) {

JAVA RULE
sleep pauses without releasing held monitors; wait...

RESULT
wait requires owning the target monitor and reacquires...

Watch out: wait releases only the target monitor, not every lock the thread holds.

INTERVIEWER FOLLOW-UP - ANSWERED

What visibility guarantee follows a successful join of a completed thread?

Actions of the terminated worker happen-before the joining thread's subsequent actions after a successful join, allowing visibility of its completed result.

Original source: Core Java Zero to Hero, page 72 · Q099 · Interview study edition

Q100 Why must wait be placed inside a condition loop?

20-SECOND ANSWER

Always recheck the waiting condition in a loop because wakeups do not guarantee the condition is true.

WHY IT WORKS · EASY EXPLANATION

Spurious wakeups are allowed, and another thread may consume the available state before this waiter reacquires the monitor. The loop makes the condition the authority rather than the notification.

JAVA EXAMPLE / SCENARIO

```
synchronized (lock) {
       while (!ready) lock.wait();
       consumeData();
   }
```

DRAW THE IDEA

START / CASE
synchronized (lock) {

JAVA RULE
Always recheck the waiting condition in a loop because...

RESULT
Spurious wakeups are allowed, and another thread may...

Watch out: if (!ready) wait() checks the condition only once.

INTERVIEWER FOLLOW-UP - ANSWERED

Why must the notifying thread update ready while holding the same monitor?

The predicate update and wait check must use the same monitor policy to avoid missed conditions and to make the updated state visible when the waiter reacquires it.

Original source: Core Java Zero to Hero, page 73 · Q100 · Interview study edition

Q101 How do notify and notifyAll differ?

20-SECOND ANSWER

notify wakes one arbitrary waiter; notifyAll wakes all waiters so each can recheck its own condition.

WHY IT WORKS · EASY EXPLANATION

The notifier still owns lock until it exits the synchronized block. Waiters then compete to reacquire it. With several conditions on one monitor, notify might wake a thread whose condition is still false.

JAVA EXAMPLE / SCENARIO

```
synchronized (lock) {
       ready = true;
       lock.notifyAll();
   }
```

DRAW THE IDEA

START / CASE
synchronized (lock) {

JAVA RULE
notify wakes one arbitrary waiter; notifyAll wakes all...

RESULT
The notifier still owns lock until it exits the...

Watch out: Waking a waiter for the wrong predicate can cause missed progress with notify.

INTERVIEWER FOLLOW-UP - ANSWERED

Does notifyAll guarantee fairness among the awakened threads?

No. notifyAll wakes eligible waiters to compete for the lock, but the scheduler chooses which reacquires it first; fairness is not promised.

Original source: Core Java Zero to Hero, page 73 · Q101 · Interview study edition

Q102 What does interrupt actually do?

20-SECOND ANSWER

interrupt requests cooperative cancellation; it does not forcibly terminate a thread.

WHY IT WORKS · EASY EXPLANATION

The worker observes the flag between work units. Some blocking APIs instead throw InterruptedException. Non-interruptible operations need their own deadline or resource-closing policy.

JAVA EXAMPLE / SCENARIO

```
while (!Thread.currentThread().isInterrupted()) {
       doOneSmallUnit();
   }
   // Another thread can call worker.interrupt().
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
while...	interrupt requests cooperative cancellation; it does...	The worker observes the flag between work units.

Watch out: interrupt is not a safe forced kill.

INTERVIEWER FOLLOW-UP - ANSWERED

How do you combine interruption with closing a resource to stop blocking I/O?

Combine a cancellation signal/interruption with closing the channel/socket or using the API's cancellation mechanism; separately verify the operation actually ends.

Original source: Core Java Zero to Hero, page 74 · Q102 · Interview study edition

Q103 What are daemon threads?

20-SECOND ANSWER

Daemon threads do not keep the JVM alive when all non-daemon threads have ended.

WHY IT WORKS · EASY EXPLANATION

The JVM can terminate while the sampler is still running. Essential writes or commits need an explicit lifecycle with completion and shutdown handling; daemon status is unsuitable as a completion guarantee.

JAVA EXAMPLE / SCENARIO

```
Thread sampler = new Thread(() -> sampleMetrics());
   sampler.setDaemon(true); // Must be set before start
   sampler.start();
```

DRAW THE IDEA

START / CASE
Thread sampler = new Thread(() ->...

JAVA RULE
Daemon threads do not keep the JVM alive when all...

RESULT
The JVM can terminate while the sampler is still running.

Watch out: Daemon does not mean lower priority or automatic safe shutdown.

INTERVIEWER FOLLOW-UP - ANSWERED

Are virtual threads daemon threads?

Yes. Virtual threads are daemon threads; a running virtual thread alone does not keep the JVM alive. Wait for required work explicitly.

Original source: Core Java Zero to Hero, page 74 · Q103 · Interview study edition

Q104 How do deadlock, livelock and starvation differ?

20-SECOND ANSWER

Deadlock is a circular wait; livelock is repeated action without progress; starvation is one task repeatedly denied progress.

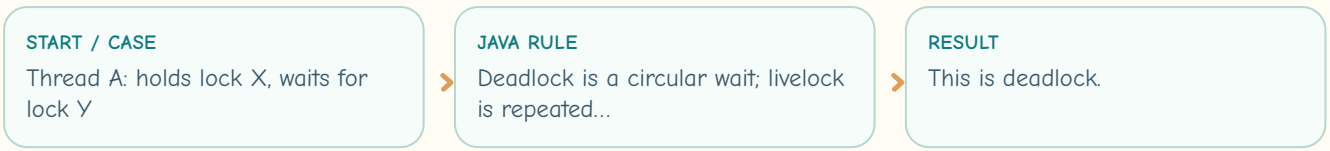
WHY IT WORKS · EASY EXPLANATION

This is deadlock. If both repeatedly release and retry in collision, they may livelock. If one worker keeps losing access while others progress, it is starving. The remedies depend on the failure type.

JAVA EXAMPLE / SCENARIO

```
Thread A: holds lock X, waits for lock Y
   Thread B: holds lock Y, waits for lock X
   Result: circular wait; neither can progress
```

DRAW THE IDEA



Watch out: More threads or a longer timeout does not fix a cyclic lock dependency.

INTERVIEWER FOLLOW-UP - ANSWERED

What evidence would you gather from a thread dump?

Collect repeated dumps, thread names, states, stack traces, lock owners and wait targets, alongside CPU and queue/dependency metrics.

Original source: Core Java Zero to Hero, page 75 · Q104 · Interview study edition

MODULE 14 / 21

14

Java Memory Model

A data race has conflicting shared accesses, including a write, without the required happens-before ordering.

8 interview questions · clear examples · follow-up answers

Q105 What is a data race?

20-SECOND ANSWER

A data race has conflicting shared accesses, including a write, without the required happens-before ordering.

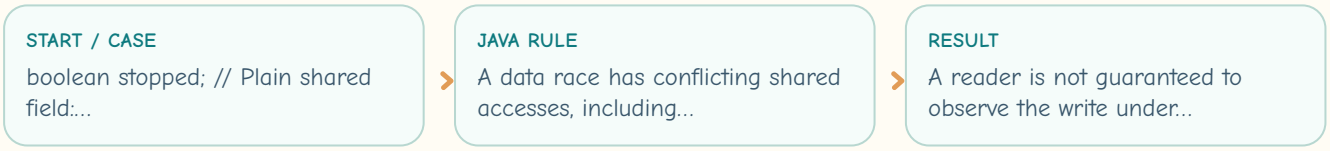
WHY IT WORKS · EASY EXPLANATION

A reader is not guaranteed to observe the write under this pattern. Timing or Thread.sleep does not establish the missing ordering. Other compound state may require more than a volatile flag.

JAVA EXAMPLE / SCENARIO

```
boolean stopped; // Plain shared field: unsafe flag pattern
   // Thread A: stopped = true;
   // Thread B: while (!stopped) { work(); }
   // Repair visibility with volatile or shared locking.
```

DRAW THE IDEA



Watch out: A race can exist even when a quick test appears to work.

INTERVIEWER FOLLOW-UP - ANSWERED

What is the difference between a data race and a higher-level race condition?

A data race is unsynchronized conflicting memory access. A higher-level race condition can also happen with individually synchronized operations that violate a multi-step business rule.

Original source: Core Java Zero to Hero, page 76 · Q105 · Interview study edition

Q106 What does happens-before mean?

20-SECOND ANSWER

Happens-before is a Java Memory Model ordering that guarantees visibility of earlier actions to later actions.

WHY IT WORKS · EASY EXPLANATION

The start rule orders the earlier write before the child's actions, so this prints 42 when no other writes intervene. join also makes the terminated thread's actions visible to the joining thread.

JAVA EXAMPLE / SCENARIO

```
int[] data = {0};
   data[0] = 42;
   Thread child = new Thread(() -> System.out.println(data[0]));
   child.start(); // Writes before start are visible to child
   child.join();
```

DRAW THE IDEA

START / CASE
int[] data = {0};

JAVA RULE
Happens-before is a Java Memory Model ordering that...

RESULT
The start rule orders the earlier write before the...

Watch out: sleep or yield creates no happens-before edge for a shared data update.

INTERVIEWER FOLLOW-UP - ANSWERED

What guarantee does submitting a task to an ExecutorService provide?

ExecutorService specifies that actions before submitting a task happen-before actions in that task. Successful Future.get supplies the return handoff to the caller.

Original source: Core Java Zero to Hero, page 77 · Q106 · Interview study edition

Q107 What does volatile guarantee?

20-SECOND ANSWER

volatile provides visibility and ordering for reads and writes, but does not make compound updates atomic.

WHY IT WORKS · EASY EXPLANATION

The volatile flag suits a stop signal. An increment combines several steps, so competing threads can overwrite updates. Use an atomic counter or a lock when every increment must count.

JAVA EXAMPLE / SCENARIO

```
volatile boolean stopped;
   // Writer: stopped = true;
   // Reader: while (!stopped) { doOneUnit(); }
   // volatile int count; count++; is still read + add + write.
```

DRAW THE IDEA

START / CASE
volatile boolean stopped;

JAVA RULE
volatile provides visibility and ordering for reads and...

RESULT
The volatile flag suits a stop signal.

Watch out: volatile int count; count++; still consists of a read, calculation and write.

INTERVIEWER FOLLOW-UP - ANSWERED

When is volatile Config enough and when is compare-and-set needed?

A volatile Config works for whole immutable replacements. Use compareAndSet or locking if multiple writers must compose changes without lost updates.

Original source: Core Java Zero to Hero, page 77 · Q107 · Interview study edition

Q108 How does synchronized supply both exclusion and visibility?

20-SECOND ANSWER

synchronized gives mutual exclusion and visibility when all conflicting accesses follow the same monitor policy.

WHY IT WORKS · EASY EXPLANATION

Only one thread can execute these critical sections at a time. Unlocking happens-before a subsequent successful lock on the same monitor. Reading balance without this policy would break the protection.

JAVA EXAMPLE / SCENARIO

```
private int balance;
   private final Object lock = new Object();
   void deposit(int amount) {
       synchronized (lock) { balance += amount; }
   }
   int balance() {
       synchronized (lock) { return balance; }
   }
```

DRAW THE IDEA

START / CASE
private int balance;

JAVA RULE
synchronized gives mutual exclusion and visibility when...

RESULT
Only one thread can execute these critical sections at...

Watch out: A synchronized writer and an unsynchronized reader still need an appropriate visibility policy.

INTERVIEWER FOLLOW-UP - ANSWERED

What monitor does a static synchronized method acquire?

The monitor associated with the defining Class object, for example MyService.class, rather than any particular instance monitor.

Original source: Core Java Zero to Hero, page 78 · Q108 · Interview study edition

Q109 How do atomic variables use compare-and-set?

20-SECOND ANSWER

Compare-and-set changes a value only if it still matches the expected value, enabling coordinated atomic updates.

WHY IT WORKS · EASY EXPLANATION

The first update matches 5 and installs 6. The second expected value is stale, so no change occurs. Retried update functions must avoid non-repeatable side effects.

JAVA EXAMPLE / SCENARIO

```
AtomicInteger n = new AtomicInteger(5);
   System.out.println(n.compareAndSet(5, 6)); // true
   System.out.println(n.compareAndSet(5, 7)); // false
   System.out.println(n.get());              // 6
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
AtomicInteger n = new AtomicInteger(5);	Compare-and-set changes a value only if it still...	The first update matches 5 and installs 6.

Watch out: Atomicity of one field does not automatically protect an invariant spanning two fields.

INTERVIEWER FOLLOW-UP - ANSWERED

How could an AtomicReference hold one immutable state containing both fields?

Store a record or immutable State object containing both fields in AtomicReference<State>; update the whole state atomically rather than separate fields.

Original source: Core Java Zero to Hero, page 78 · Q109 · Interview study edition

Q110 What is safe publication?

20-SECOND ANSWER

Safe publication ensures another thread sees a correctly constructed object's initialized state.

WHY IT WORKS · EASY EXPLANATION

Construct first, then publish through volatile. A reader observing that assignment sees the preceding construction writes. Later mutable state changes still require their own synchronization.

JAVA EXAMPLE / SCENARIO

```
volatile Config config;
   // Writer:
   Config built = new Config(2000);
   config = built;
   // Reader:
   Config snapshot = config;
   if (snapshot != null) use(snapshot);
```

DRAW THE IDEA

START / CASE
volatile Config config;

JAVA RULE
Safe publication ensures another thread sees a...

RESULT
Construct first, then publish through volatile.

Watch out: Publishing this from a constructor can expose an incompletely initialized object.

INTERVIEWER FOLLOW-UP - ANSWERED

Why does handing an object through BlockingQueue help visibility?

BlockingQueue provides a documented memory handoff: actions before put happen-before actions after a different thread takes the element. Later mutation still needs rules.

Original source: Core Java Zero to Hero, page 79 · Q110 · Interview study edition

Q111 Why does double-checked locking require volatile?

20-SECOND ANSWER

Conventional double-checked locking needs volatile for safe publication; a static holder is often simpler.

WHY IT WORKS · EASY EXPLANATION

Class initialization safely publishes the holder's instance when first used. For the conventional double-check pattern, a volatile instance reference supplies the necessary ordering. Neither approach makes mutable service methods automatically thread safe.

JAVA EXAMPLE / SCENARIO

```
class Service {
       private static class Holder {
           static final Service INSTANCE = new Service();
       }
       static Service instance() { return Holder.INSTANCE; }
   }
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
class Service {	Conventional double-checked locking needs volatile for...	Class initialization safely publishes the holder's...

Watch out: Synchronizing only the construction path does not protect readers that bypass that lock.

INTERVIEWER FOLLOW-UP - ANSWERED

How does class initialization make the initialization-on-demand holder idiom safe?

JVM class initialization serializes initialization of the holder class and makes its initialized static field safely available to users.

Original source: Core Java Zero to Hero, page 79 · Q111 · Interview study edition

Q112 What are ABA and false sharing?

20-SECOND ANSWER

ABA is an unnoticed value change away and back; false sharing is contention between independent data sharing a cache line.

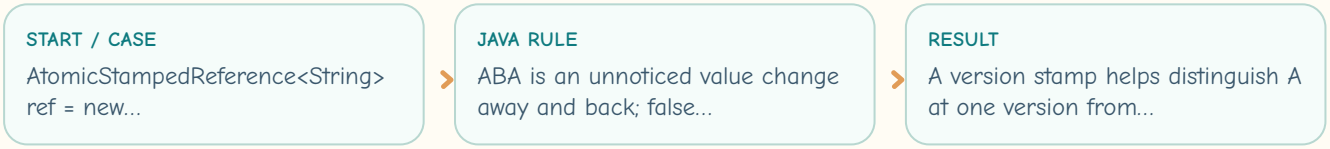
WHY IT WORKS · EASY EXPLANATION

A version stamp helps distinguish A at one version from A after intermediate changes, when every relevant update changes the stamp. False sharing is a separate performance problem, not this logical identity issue.

JAVA EXAMPLE / SCENARIO

```
AtomicStampedReference<String> ref = new AtomicStampedReference<>("A", 0);
   int[] stamp = new int[1];
   String old = ref.get(stamp);
   boolean changed = ref.compareAndSet(old, "B", stamp[0], stamp[0] + 1);
```

DRAW THE IDEA



Watch out: Not every compare-and-set algorithm is harmed by ABA; analyze what history matters.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you verify suspected false sharing rather than assuming it?

Measure with JMH, hardware counters or profilers while controlling contention and cache layout, then compare padded/separated counters under the same workload.

Original source: Core Java Zero to Hero, page 80 · Q112 · Interview study edition

MODULE 15 / 21

15

Locks and coordination primitives

Use ReentrantLock when you need timed or interruptible acquisition or multiple Conditions; always release it in finally.

8 interview questions · clear examples · follow-up answers

Q113 When would you use ReentrantLock instead of synchronized?

20-SECOND ANSWER

Use ReentrantLock when you need timed or interruptible acquisition or multiple Conditions; always release it in finally.

WHY IT WORKS · EASY EXPLANATION

This call can stop waiting after a deadline and can be interrupted. synchronized does not provide this timed acquisition API. A missed unlock can block future progress indefinitely.

JAVA EXAMPLE / SCENARIO

```
ReentrantLock lock = new ReentrantLock();
   if (lock.tryLock(100, TimeUnit.MILLISECONDS)) {
       try { update(); }
       finally { lock.unlock(); }
   } else {
       handleBusy();
   }
```

DRAW THE IDEA

START / CASE
ReentrantLock lock = new ReentrantLock();

JAVA RULE
Use ReentrantLock when you need timed or interruptible...

RESULT
This call can stop waiting after a deadline and can be...

Watch out: A fair lock can reduce starvation tendencies but may lower throughput and does not guarantee fair CPU scheduling.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can tryLock with a deadline help avoid indefinite waits?

tryLock with a timeout allows the caller to abort, retry, or report overload instead of waiting forever for a contested lock.

Original source: Core Java Zero to Hero, page 81 · Q113 · Interview study edition

Q114 What does reentrancy mean?

20-SECOND ANSWER

A reentrant lock lets the owning thread acquire the same lock again without deadlocking itself.

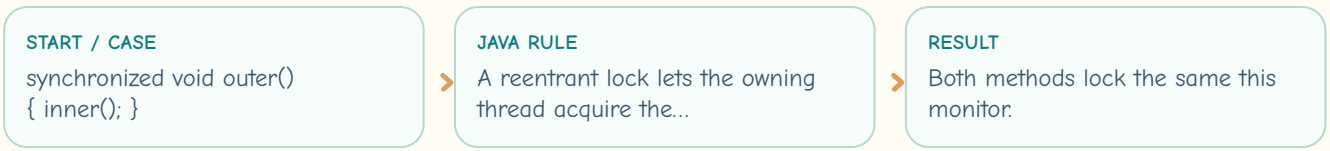
WHY IT WORKS · EASY EXPLANATION

Both methods lock the same this monitor. The owner may enter inner while still in outer. Reentrancy does not permit another thread to enter until the owner releases its acquisitions.

JAVA EXAMPLE / SCENARIO

```
synchronized void outer() { inner(); }
   synchronized void inner() { System.out.println("OK"); }
   // Calling outer on one object prints OK.
```

DRAW THE IDEA



Watch out: Reentrancy does not prevent deadlock involving two different locks.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens if an explicit lock is acquired twice and unlocked only once?

ReentrantLock's hold count remains one after only one unlock, so the current thread still owns it and other threads may remain blocked.

Original source: Core Java Zero to Hero, page 82 · Q114 · Interview study edition

Q115 How do Condition objects improve a wait/notify design?

20-SECOND ANSWER

Conditions let one lock maintain separate wait queues for different predicates.

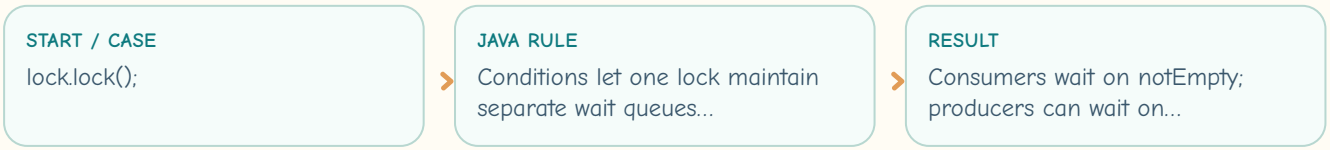
WHY IT WORKS · EASY EXPLANATION

Consumers wait on notEmpty; producers can wait on notFull. await releases and reacquires the associated lock. Keep the predicate loop and hold the lock when awaiting or signalling.

JAVA EXAMPLE / SCENARIO

```
lock.lock();
   try {
       while (queue.isEmpty()) notEmpty.await();
       Item item = queue.remove();
       notFull.signal();
   } finally { lock.unlock(); }
```

DRAW THE IDEA



Watch out: A signal is a wake-up hint to recheck state, not a stored permit.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can Semaphore release-before-acquire work differently from Condition.signal?

Semaphore permits are counted and can be released before acquisition. Condition.signal requires the related lock and does not store an unbounded signal count.

Original source: Core Java Zero to Hero, page 82 · Q115 · Interview study edition

Q116 How does a Semaphore differ from a lock?

20-SECOND ANSWER

A semaphore limits permits for concurrent work; a lock usually grants exclusive ownership of a critical section.

WHY IT WORKS · EASY EXPLANATION

At most two cooperating operations can hold permits at once. Release only a successfully acquired permit, and include failures and cancellation in cleanup. Unlike a lock, semaphore permits are not owned by the acquiring thread.

JAVA EXAMPLE / SCENARIO

```
Semaphore slots = new Semaphore(2);
   slots.acquire();
   try { callDownstream(); }
   finally { slots.release(); }
```

DRAW THE IDEA

START / CASE
Semaphore slots = new Semaphore(2);

JAVA RULE
A semaphore limits permits for concurrent work; a lock...

RESULT
At most two cooperating operations can hold permits at...

Watch out: Accidentally releasing after a failed acquire inflates the permit count.

INTERVIEWER FOLLOW-UP - ANSWERED

Does limiting concurrency also enforce a fixed requests-per-second rate?

No. Concurrency limits cap simultaneous in-flight work; rate limits cap arrivals per time interval. Throughput also depends on task duration.

Original source: Core Java Zero to Hero, page 83 · Q116 · Interview study edition

Q117 How do CountDownLatch and CyclicBarrier differ?

20-SECOND ANSWER

CountDownLatch counts down once to release waiters; CyclicBarrier repeatedly coordinates a fixed group at a meeting point.

WHY IT WORKS · EASY EXPLANATION

The latch cannot reset after reaching zero. The barrier can coordinate later rounds after a successful trip. Interruptions or failures can break a barrier and require a recovery policy.

JAVA EXAMPLE / SCENARIO

```
CountDownLatch ready = new CountDownLatch(2);
   // Each of two workers calls ready.countDown() when ready.
   ready.await();
   CyclicBarrier round = new CyclicBarrier(2);
   // Both simulation workers call round.await() each round.
```

DRAW THE IDEA

START / CASE
CountDownLatch ready = new CountDownLatch(2);

JAVA RULE
CountDownLatch counts down once to release waiters;...

RESULT
The latch cannot reset after reaching zero.

Watch out: A worker that never reaches the barrier can block the remaining parties.

INTERVIEWER FOLLOW-UP - ANSWERED

When does Phaser offer a better fit?

Phaser suits repeated phases with parties that can register or deregister dynamically; CyclicBarrier has a fixed number of parties.

Original source: Core Java Zero to Hero, page 83 · Q117 · Interview study edition

Q118 When is a ReadWriteLock useful?

20-SECOND ANSWER

A ReadWriteLock allows concurrent readers and exclusive writers; it helps only when the workload benefits from that separation.

WHY IT WORKS · EASY EXPLANATION

Readers can overlap while no writer owns the lock. All accesses must honor the same policy. Short reads or frequent writes can make this slower than a simple lock or immutable snapshots.

JAVA EXAMPLE / SCENARIO

```
ReadWriteLock rw = new ReentrantReadWriteLock();
   rw.readLock().lock();
   try { readStableState(); }
   finally { rw.readLock().unlock(); }
   rw.writeLock().lock();
   try { replaceState(); }
   finally { rw.writeLock().unlock(); }
```

DRAW THE IDEA

START / CASE
ReadWriteLock rw = new...

JAVA RULE
A ReadWriteLock allows concurrent readers and exclusive...

RESULT
Readers can overlap while no writer owns the lock.

Watch out: An access-order LRU get changes state and cannot casually use a read lock.

INTERVIEWER FOLLOW-UP - ANSWERED

Why is upgrading directly from a read lock to a write lock dangerous?

Two readers can both hold the read lock and wait to acquire the write lock, causing deadlock or stalled upgrade. Release/retry or use a supported conversion protocol.

Original source: Core Java Zero to Hero, page 84 · Q118 · Interview study edition

Q119 How does StampedLock optimistic reading work?

20-SECOND ANSWER

StampedLock optimistic reading requires validation; failed validation must retry under a real read lock.

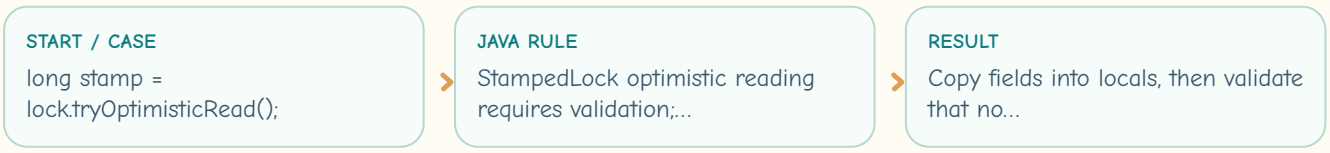
WHY IT WORKS · EASY EXPLANATION

Copy fields into locals, then validate that no intervening write invalidated the read. Do not act on unvalidated inconsistent values. StampedLock is not reentrant, and optimistic reads must be designed carefully.

JAVA EXAMPLE / SCENARIO

```
long stamp = lock.tryOptimisticRead();
   int localX = x, localY = y;
   if (!lock.validate(stamp)) {
       stamp = lock.readLock();
       try { localX = x; localY = y; }
       finally { lock.unlockRead(stamp); }
   }
```

DRAW THE IDEA



Watch out: Successful validation is necessary before accepting a multi-field snapshot.

INTERVIEWER FOLLOW-UP - ANSWERED

Why are complex traversals of concurrently mutated structures risky during an optimistic read?

Optimistic reads can see inconsistent intermediate structure while a writer mutates it. Copy simple fields, validate the stamp, and retry under a read lock if invalid.

Original source: Core Java Zero to Hero, page 84 · Q119 · Interview study edition

Q120 How would you prevent and diagnose deadlock?

20-SECOND ANSWER

Prevent deadlock through consistent lock ordering; diagnose it from blocked threads and lock ownership evidence.

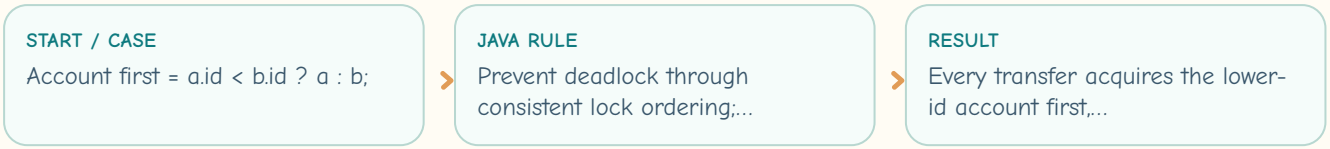
WHY IT WORKS · EASY EXPLANATION

Every transfer acquires the lower-id account first, preventing opposite ordering. Handle equal identities deliberately. Thread dumps and ThreadMXBean can identify many monitor or ownable-synchronizer deadlocks.

JAVA EXAMPLE / SCENARIO

```
// Assuming stable, unique account ids:
   Account first = a.id < b.id ? a : b;
   Account second = a.id < b.id ? b : a;
   synchronized (first) {
       synchronized (second) { transferBalances(a, b); }
   }
```

DRAW THE IDEA



Watch out: Ordering by a non-unique or changing attribute can fail to establish a consistent total order.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you handle two account objects whose ordering ids are equal?

Use a stable unique tie-breaker such as object identity ordering plus a tie lock. Equal business IDs must not produce opposite locking orders.

Original source: Core Java Zero to Hero, page 85 · Q120 · Interview study edition

MODULE 16 / 21

16

Executors, futures and overload

ExecutorService separates task submission from execution and offers results, cancellation, and lifecycle control.

8 interview questions · clear examples · follow-up answers

Q121 What problem does ExecutorService solve?

20-SECOND ANSWER

ExecutorService separates task submission from execution and offers results, cancellation, and lifecycle control.

WHY IT WORKS · EASY EXPLANATION

The executor runs the Callable and Future exposes its result. This convenience factory uses an unbounded queue, so production overload control may require an explicitly bounded ThreadPoolExecutor.

JAVA EXAMPLE / SCENARIO

```
ExecutorService pool = Executors.newFixedThreadPool(2);
   try {
       Future<Integer> result = pool.submit(() -> 21 * 2);
       System.out.println(result.get()); // 42
   } finally { pool.shutdown(); }
```

DRAW THE IDEA

START / CASE
ExecutorService pool =...

JAVA RULE
ExecutorService separates task submission from...

RESULT
The executor runs the Callable and Future exposes its...

Watch out: An executor does not automatically make the submitted task thread safe.

INTERVIEWER FOLLOW-UP - ANSWERED

Why should an application avoid creating a new pool for every request?

A pool needs an application-level owner to reuse it and shut it down. Per-request pools multiply threads, queues and leaks and lose predictable capacity controls.

Original source: Core Java Zero to Hero, page 86 · Q121 · Interview study edition

Q122 How does ThreadPoolExecutor decide whether to add a worker?

20-SECOND ANSWER

ThreadPoolExecutor tries core workers, then queueing, then extra workers up to maximum, then rejection.

WHY IT WORKS · EASY EXPLANATION

Workers are created on demand. Once core capacity is reached, tasks normally enter the queue. A full queue triggers growth toward maximum. If both are unavailable, the rejection policy runs. The counts describe a blocked-work scenario, not every timing interleaving.

JAVA EXAMPLE / SCENARIO

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
          2, 4, 30, TimeUnit.SECONDS,
          new ArrayBlockingQueue<>(10),
          new ThreadPoolExecutor.AbortPolicy());
   // Under sustained blocked work: 2 core + 10 queued + 2 extra.
```

DRAW THE IDEA

START / CASE
ThreadPoolExecutor pool = new...

JAVA RULE
ThreadPoolExecutor tries core workers, then queueing,...

RESULT
Workers are created on demand.

Watch out: maximumPoolSize does not guarantee the pool grows before tasks enter the queue.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can an unbounded queue make latency rise while active threads stay constant?

Unbounded queues accept requests faster than workers finish, making queue age and response latency grow while only the fixed worker count runs.

Original source: Core Java Zero to Hero, page 87 · Q122 · Interview study edition

Q123 How would you size a thread pool?

20-SECOND ANSWER

Size a pool from measured work, CPU capacity, blocking time, downstream limits, and admission requirements.

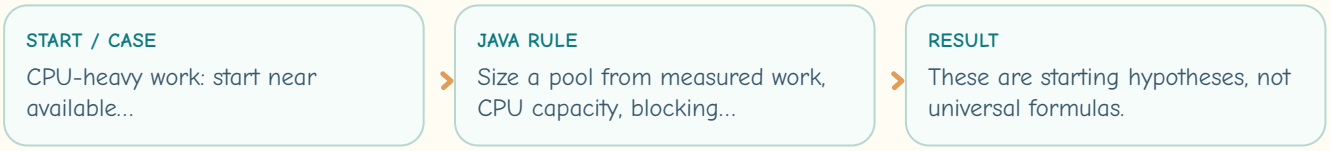
WHY IT WORKS · EASY EXPLANATION

These are starting hypotheses, not universal formulas. Measure throughput, tail latency, queue age, and resource waits, then validate under the actual container limits and dependency behavior.

JAVA EXAMPLE / SCENARIO

```
CPU-heavy work: start near available processor capacity.
   Mostly waiting work: more concurrency may help.
   Database limited to 20 connections: 200 workers cannot
   create 200 simultaneous database operations.
```

DRAW THE IDEA



Watch out: Increasing thread count can increase context switching and downstream contention.

INTERVIEWER FOLLOW-UP - ANSWERED

Which measurements would show that the queue is the bottleneck?

Measure queue depth, oldest task age, enqueue-to-start delay, active workers, completion rate, rejects and downstream wait times under load.

Original source: Core Java Zero to Hero, page 87 · Q123 · Interview study edition

Q124 How do execute and submit report task failures?

20-SECOND ANSWER

execute failures follow uncaught-exception handling; submit captures task failures in its returned Future.

WHY IT WORKS · EASY EXPLANATION

The Future retains the exception. Ignoring it can hide failure from application logic. execute does not return a Future; failures from a Runnable reach the worker's uncaught-exception path.

JAVA EXAMPLE / SCENARIO

```
Future<?> result = pool.submit(() -> {
       throw new IllegalStateException("boom");
   });
   try { result.get(); }
   catch (ExecutionException e) {
       System.out.println(e.getCause().getMessage()); // boom
   }
```

DRAW THE IDEA

START / CASE
Future<?> result = pool.submit(() -> {

JAVA RULE
execute failures follow uncaught-exception handling;...

RESULT
The Future retains the exception.

Watch out: A successful submit call means admission, not successful task completion.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you record failures for fire-and-forget tasks?

Wrap Runnable work in explicit error logging/metrics, use observed Futures, or centralize reporting in an executor hook; never assume fire-and-forget succeeds.

Original source: Core Java Zero to Hero, page 88 · Q124 · Interview study edition

Q125 How do rejection policies affect overload?

20-SECOND ANSWER

Rejection policy defines overload behavior; choose explicit failure, backpressure, or documented loss semantics.

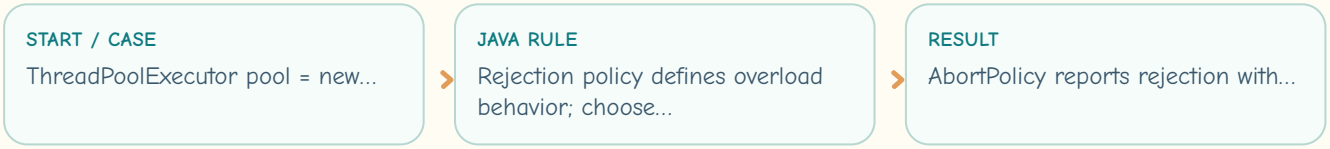
WHY IT WORKS · EASY EXPLANATION

AbortPolicy reports rejection with RejectedExecutionException. CallerRunsPolicy can slow the submitting thread, but may block a sensitive caller such as an event loop. Silent discard can lose work and leave submit-created futures incomplete.

JAVA EXAMPLE / SCENARIO

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
          1, 1, 0, TimeUnit.SECONDS, new ArrayBlockingQueue<>(1),
          new ThreadPoolExecutor.AbortPolicy());
   // One running + one queued; a further task can be rejected.
```

DRAW THE IDEA



Watch out: CallerRuns can block an event-loop or request thread and alter latency dramatically.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens to a submitted Future if its task is silently discarded?

If a handler silently discards the task, its Future can remain incomplete and callers may wait forever. Prefer explicit rejection/cancellation semantics.

Original source: Core Java Zero to Hero, page 88 · Q125 · Interview study edition

Q126 How should an ExecutorService shut down?

20-SECOND ANSWER

Shut down gracefully first, wait with a bound, then request interruption if the deadline expires.

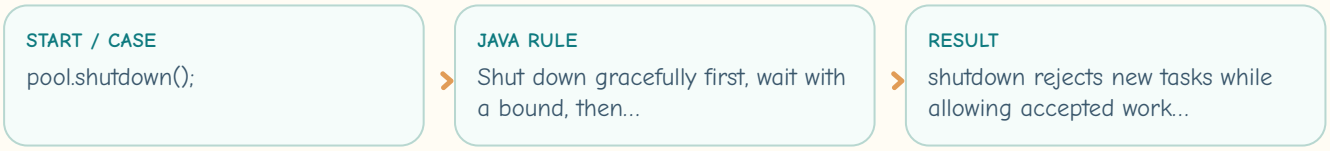
WHY IT WORKS · EASY EXPLANATION

shutdown rejects new tasks while allowing accepted work to finish. shutdownNow requests interruption and returns tasks that never started. A non-cooperative task can still continue; shutdown is not a guaranteed forced stop.

JAVA EXAMPLE / SCENARIO

```
pool.shutdown();
   try {
       if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();
   } catch (InterruptedException e) {
       pool.shutdownNow();
       Thread.currentThread().interrupt();
   }
```

DRAW THE IDEA



Watch out: shutdown returning does not mean every task has finished.

INTERVIEWER FOLLOW-UP - ANSWERED

What would you persist before terminating an important queued batch?

Persist job identity, state, progress/checkpoint and recovery intent before exit, then reconcile uncompleted tasks after restart instead of relying only on memory.

Original source: Core Java Zero to Hero, page 89 · Q126 · Interview study edition

Q127 How do Future.get timeouts and cancellation differ?

20-SECOND ANSWER

A Future.get timeout limits the caller's wait; cancel is a separate best-effort request affecting the task's result state.

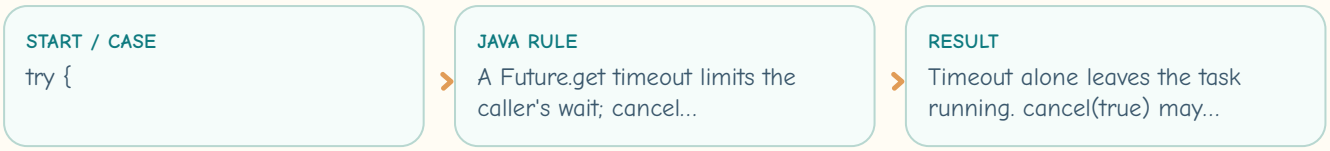
WHY IT WORKS · EASY EXPLANATION

Timeout alone leaves the task running. cancel(true) may interrupt its executing thread but cannot undo completed external work. Define deadlines and cancellation behavior for the actual operation.

JAVA EXAMPLE / SCENARIO

```
try {
       result.get(1, TimeUnit.SECONDS);
   } catch (TimeoutException e) {
       result.cancel(true); // Request interruption if supported
   }
```

DRAW THE IDEA



Watch out: A timed-out caller and a terminated task are different states.

INTERVIEWER FOLLOW-UP - ANSWERED

How would retries cause duplicate side effects if the original task keeps running?

The old task might complete after the timeout while a retry repeats the same external action. Use stable idempotency keys and an outcome lookup/reconciliation contract.

Original source: Core Java Zero to Hero, page 89 · Q127 · Interview study edition

Q128 How can a thread pool deadlock without explicit locks?

20-SECOND ANSWER

A pool can starve when every worker blocks waiting for child tasks queued to that same exhausted pool.

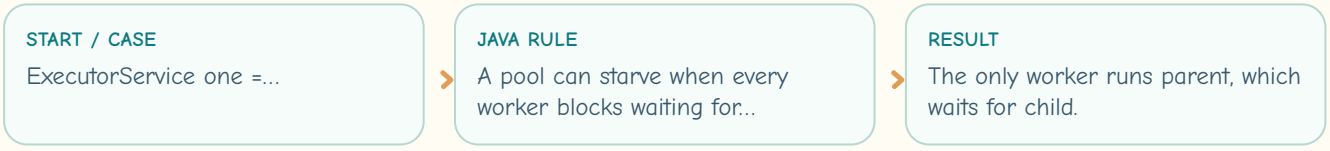
WHY IT WORKS · EASY EXPLANATION

The only worker runs parent, which waits for child. Child cannot start until that worker is free. Compose dependencies without blocking scarce workers or use a carefully separated execution design.

JAVA EXAMPLE / SCENARIO

```
ExecutorService one = Executors.newSingleThreadExecutor();
   Future<Integer> parent = one.submit(() -> {
       Future<Integer> child = one.submit(() -> 42);
       return child.get(); // No free worker to execute child
   });
   // Illustration of the defect; do not run without a bound/cleanup.
```

DRAW THE IDEA



Watch out: A queue full of runnable children cannot progress while all workers wait for them.

INTERVIEWER FOLLOW-UP - ANSWERED

How would thenCompose change this design?

thenCompose flattens a function that returns another CompletionStage, so dependency chains avoid nested futures and their error handling stays composable.

Original source: Core Java Zero to Hero, page 90 · Q128 · Interview study edition

MODULE 17 / 21

17

CompletableFuture and asynchronous composition

CompletableFuture adds result transformations, composition, combination, and recovery to Future-style results.

8 interview questions · clear examples · follow-up answers

Q129 How is CompletableFuture different from Future?

20-SECOND ANSWER

CompletableFuture adds result transformations, composition, combination, and recovery to Future-style results.

WHY IT WORKS · EASY EXPLANATION

thenApply creates a dependent stage. No manual blocking retrieval is needed between the two stages. CompletableFuture does not by itself prove the underlying supplier uses non-blocking I/O.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<Integer> answer = CompletableFuture.completedFuture(21)
          .thenApply(n -> n * 2);
   System.out.println(answer.join()); // 42
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
CompletableFuture<Integer> answer =...	CompletableFuture adds result transformations,...	thenApply creates a dependent stage.

Watch out: Returning a CompletableFuture does not prove that the underlying work is non-blocking.

INTERVIEWER FOLLOW-UP - ANSWERED

How could a blocking supplier still run behind supplyAsync?

supplyAsync schedules work on an executor but the supplier can still block on a database, socket or sleep. Async composition is not automatically nonblocking I/O.

Original source: Core Java Zero to Hero, page 91 · Q129 · Interview study edition

Q130 How do thenApply and thenCompose differ?

20-SECOND ANSWER

thenApply transforms a result; thenCompose flattens a transformation that returns another asynchronous result.

WHY IT WORKS · EASY EXPLANATION

thenCompose returns one future for the final String. Using thenApply for the same future-returning function would create a nested future. length directly transforms String into Integer.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<String> name = CompletableFuture.completedFuture("Sai");
   CompletableFuture<Integer> length = name.thenApply(String::length);
   CompletableFuture<String> greeting = name.thenCompose(
           n -> CompletableFuture.completedFuture("Hi " + n));
   System.out.println(greeting.join()); // Hi Sai
```

DRAW THE IDEA

START / CASE	JAVA RULE	RESULT
CompletableFuture<String> name =...	thenApply transforms a result; thenCompose flattens a...	thenCompose returns one future for the final String.

Watch out: Calling join inside thenApply can turn a composable dependency into a blocked worker.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you combine independent user and configuration loads?

Start two independent CompletableFutures, then combine them with thenCombine after both results are available; set deadlines and failure policy.

Original source: Core Java Zero to Hero, page 92 · Q130 · Interview study edition

Q131 How do thenCombine and allOf differ?

20-SECOND ANSWER

thenCombine merges two successful values; allOf waits for a group but returns no collected values itself.

WHY IT WORKS · EASY EXPLANATION

The combine function receives both values. allOf signals group completion with Void; obtain individual results afterward if needed. Its completion does not automatically cancel siblings after a failure.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<Integer> a = CompletableFuture.completedFuture(10);
   CompletableFuture<Integer> b = CompletableFuture.completedFuture(20);
   System.out.println(a.thenCombine(b, Integer::sum).join()); // 30
   CompletableFuture<Void> both = CompletableFuture.allOf(a, b);
   both.join();
```

DRAW THE IDEA

START / CASE
CompletableFuture<Integer> a =...

JAVA RULE
thenCombine merges two successful values; allOf waits...

RESULT
The combine function receives both values. allOf...

Watch out: allOf is not a guaranteed immediate fail-fast cancellation of the other tasks.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you keep successful outcomes even when one branch fails?

Handle each branch independently into a result containing success/error before allOf or combining. Do not let one failed branch erase other valid results.

Original source: Core Java Zero to Hero, page 92 · Q131 · Interview study edition

Q132 Where do normal and Async continuation stages execute?

20-SECOND ANSWER

Normal continuations may run on a completion or attaching thread; Async variants use their default or supplied executor.

WHY IT WORKS · EASY EXPLANATION

Supplying cpuPool makes the stage's scheduling policy explicit. A non-Async stage is not guaranteed to use the submitting thread. Default async execution commonly uses the common pool, with documented fallback behavior.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<Integer> f = CompletableFuture.completedFuture(21);
   CompletableFuture<Integer> g = f.thenApplyAsync(n -> n * 2, cpuPool);
   System.out.println(g.join()); // 42
```

DRAW THE IDEA

START / CASE
CompletableFuture<Integer> f =...

JAVA RULE
Normal continuations may run on a completion or...

RESULT
Supplying cpuPool makes the stage's scheduling policy...

Watch out: One Async suffix does not automatically move every later stage to the same executor.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you propagate request context without relying on ThreadLocal continuity?

Pass explicit immutable context through stage arguments, or use a context-aware framework, because asynchronous stages may switch threads and pooled ThreadLocals.

Original source: Core Java Zero to Hero, page 93 · Q132 · Interview study edition

Q133 How do handle, exceptionally and whenComplete differ?

20-SECOND ANSWER

handle transforms success or failure, exceptionally recovers failure, and whenComplete observes the outcome.

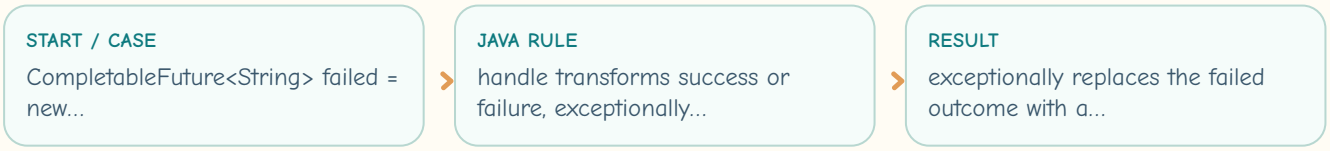
WHY IT WORKS · EASY EXPLANATION

exceptionally replaces the failed outcome with a fallback. whenComplete is normally used to observe without replacing it; an observer that throws can affect completion. handle receives both result and error for a chosen transformation.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<String> failed = new CompletableFuture<>();
   failed.completeExceptionally(new IOException("offline"));
   String value = failed.exceptionally(e -> "cached").join();
   System.out.println(value); // cached
```

DRAW THE IDEA



Watch out: Returning a default value can hide a genuine failure from downstream stages.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens if a whenComplete observer throws after a successful source stage?

The dependent whenComplete stage may complete exceptionally with the observer's error, even if its source succeeded. Keep observation callbacks safe and non-disruptive.

Original source: Core Java Zero to Hero, page 93 · Q133 · Interview study edition

Q134 How do join and get expose exceptional completion?

20-SECOND ANSWER

get reports checked interruption and ExecutionException; join reports exceptional completion through unchecked CompletionException.

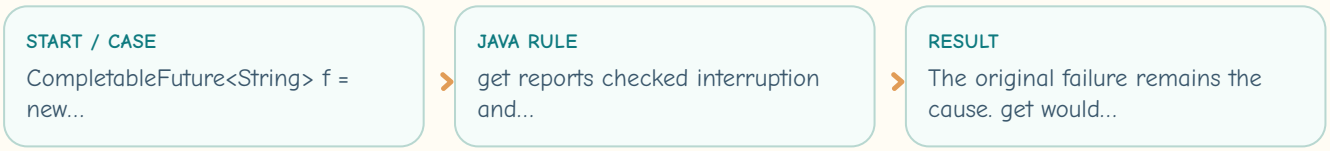
WHY IT WORKS · EASY EXPLANATION

The original failure remains the cause. get would require handling ExecutionException and InterruptedException. Cancellation has its own CancellationException behavior; join is not an interruptible-wait replacement for get.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<String> f = new CompletableFuture<>();
   f.completeExceptionally(new IOException("offline"));
   try { f.join(); }
   catch (CompletionException e) {
       System.out.println(e.getCause().getMessage()); // offline
   }
```

DRAW THE IDEA



Watch out: Using join instead of get does not make the call asynchronous.

INTERVIEWER FOLLOW-UP - ANSWERED

Which retrieval method is appropriate when interruption must stop waiting?

Use interruptible get() or timed get() and propagate/restore InterruptedException according to the caller's cancellation contract; avoid join() when checked interruption is required.

Original source: Core Java Zero to Hero, page 94 · Q134 · Interview study edition

Q135 Do orTimeout and completeOnTimeout stop the underlying work?

20-SECOND ANSWER

orTimeout and completeOnTimeout affect future completion, not automatic termination of underlying work.

WHY IT WORKS · EASY EXPLANATION

completeOnTimeout supplies a normal fallback instead. Both operate on the same future. Use downstream deadlines and task/resource ownership to control work; result completion alone is insufficient.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<String> result = new CompletableFuture<>();
   result.orTimeout(1, TimeUnit.SECONDS); // Java 9+
   // If still incomplete, result completes exceptionally.
   // A separately running external request is not stopped by this.
```

DRAW THE IDEA

START / CASE
CompletableFuture<String> result = new...

JAVA RULE
orTimeout and completeOnTimeout affect future...

RESULT
completeOnTimeout supplies a normal fallback instead.

Watch out: A timeout signal alone does not prevent duplicate external side effects from a retry.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you enforce an HTTP client deadline as well as a future deadline?

Configure client connect/read/response budgets and a total request deadline, not just Future.orTimeout. Stopping waiting does not necessarily stop a remote call.

Original source: Core Java Zero to Hero, page 94 · Q135 · Interview study edition

Q136 How would you design an asynchronous service call chain?

20-SECOND ANSWER

Compose asynchronous stages around dependencies, with explicit executors, deadlines, and failure policies.

WHY IT WORKS · EASY EXPLANATION

Profile and score may run independently. Combine waits for both values; compose flattens the asynchronous decision. This domain sketch assumes matching APIs and constructor. Decide which failures allow fallback and which require stopping the request.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<Profile> p = loadProfile(id);
   CompletableFuture<Score> s = loadScore(id);
   CompletableFuture<Decision> d = p.thenCombine(s, Request::new)
           .thenCompose(this::decide);
```

DRAW THE IDEA

START / CASE
CompletableFuture<Profile> p = loadProfile(id);

JAVA RULE
Compose asynchronous stages around dependencies, with...

RESULT
Profile and score may run independently.

Watch out: Adding supplyAsync everywhere can create queueing and context-loss problems.

INTERVIEWER FOLLOW-UP - ANSWERED

Which dependency should determine the deadline passed to later stages?

The original user/request deadline should bound later stages. Pass remaining time after prior work rather than resetting a fresh full timeout each time.

Original source: Core Java Zero to Hero, page 95 · Q136 · Interview study edition

MODULE 18 / 21

18

JVM, class loading and garbage collection

Heap holds objects conceptually; stacks hold thread frames; metaspace holds HotSpot class metadata in native memory.

8 interview questions · clear examples · follow-up answers

Q137 What is stored in the heap, stack and metaspace?

20-SECOND ANSWER

Heap holds objects conceptually; stacks hold thread frames; metaspace holds HotSpot class metadata in native memory.

WHY IT WORKS · EASY EXPLANATION

The reference and object have different roles. Optimizations can eliminate allocations, so source code is not an exact physical memory map. Thread stacks, direct buffers, and metaspace also consume memory outside the Java heap.

JAVA EXAMPLE / SCENARIO

```
void work() {
       int count = 3;                 // Local frame value
       User user = new User("Sai");  // Local reference; heap object
   }
   // User's class metadata is separate from its instance fields.
```

DRAW THE IDEA

START / CASE
void work() {

JAVA RULE
Heap holds objects conceptually; stacks hold thread...

RESULT
The reference and object have different roles.

Watch out: Metaspace is a HotSpot implementation concept, not another name for the ordinary Java object heap.

INTERVIEWER FOLLOW-UP - ANSWERED

Can reducing Java heap size increase available native memory?

Sometimes. Reducing Xmx leaves more container/process budget for stacks, code cache and native allocations, but may increase GC pressure or fail the live set.

Original source: Core Java Zero to Hero, page 96 · Q137 · Interview study edition

Q138 How do class loading, linking and initialization differ?

20-SECOND ANSWER

Loading obtains class definitions, linking verifies and prepares them, and initialization runs static initialization code.

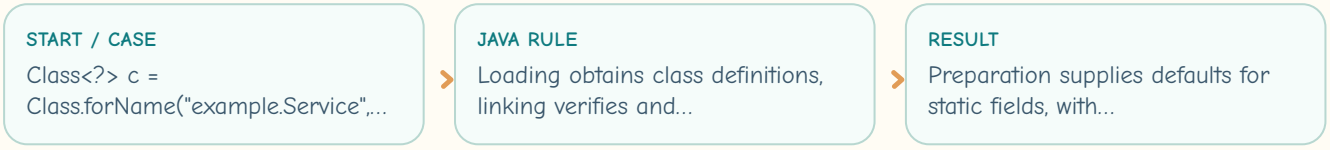
WHY IT WORKS · EASY EXPLANATION

Preparation supplies defaults for static fields, with special handling for constant values. Initialization executes static field initializers and static blocks. The example assumes the named class and loader exist.

JAVA EXAMPLE / SCENARIO

```
Class<?> c = Class.forName("example.Service", false, loader);
   // false requests loading without initialization.
   // An active use such as constructing Service can initialize it.
```

DRAW THE IDEA



Watch out: Resolution can be lazy, so do not assume every symbolic reference is resolved at one instant.

INTERVIEWER FOLLOW-UP - ANSWERED

How can a compile-time constant be used without initializing its declaring class?

A compile-time constant can be copied into the caller's bytecode. Accessing the constant alone may therefore not initialize its declaring class.

Original source: Core Java Zero to Hero, page 97 · Q138 · Interview study edition

Q139 Why can the same class name still cause a ClassCastException?

20-SECOND ANSWER

Runtime class identity includes both binary name and defining class loader.

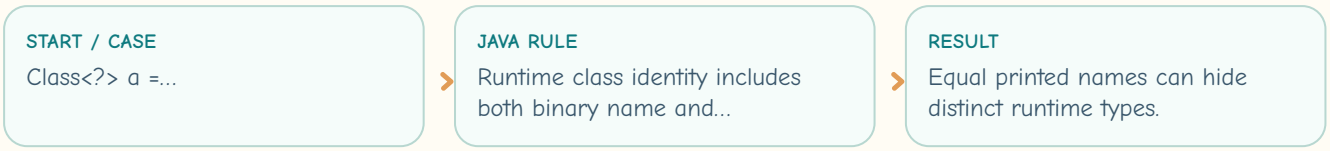
WHY IT WORKS · EASY EXPLANATION

Equal printed names can hide distinct runtime types. Parent delegation may instead make both calls return the same class. Shared plugin contracts should be loaded from an agreed common loader.

JAVA EXAMPLE / SCENARIO

```
Class<?> a = loaderA.loadClass("example.Plugin");
   Class<?> b = loaderB.loadClass("example.Plugin");
   // If independently defined by these loaders, a != b.
   // An instance of a is not automatically castable to b.
```

DRAW THE IDEA



Watch out: Matching fully qualified names is insufficient evidence of runtime type identity.

INTERVIEWER FOLLOW-UP - ANSWERED

How can static caches retain obsolete application class loaders?

Static registries, cached callbacks or long-lived thread/context-class-loader references can retain classes from an old deployment, preventing loader unloading.

Original source: Core Java Zero to Hero, page 97 · Q139 · Interview study edition

Q140 How does garbage collection decide an object is reclaimable?

20-SECOND ANSWER

Objects become collectible when no required strong-reachability path from a GC root remains; cycles alone do not prevent collection.

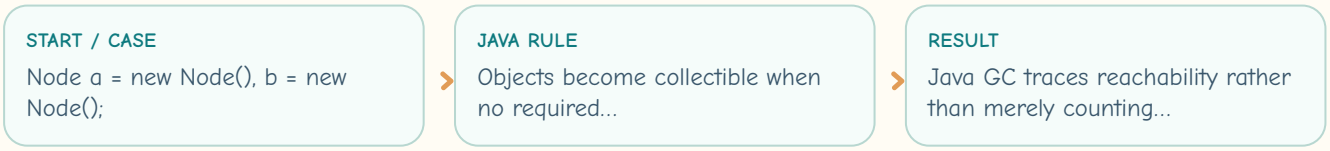
WHY IT WORKS · EASY EXPLANATION

Java GC traces reachability rather than merely counting references. Eligibility does not guarantee immediate collection. Live threads, static fields, JNI roots, and other roots can keep otherwise forgotten objects reachable.

JAVA EXAMPLE / SCENARIO

```
Node a = new Node(), b = new Node();
   a.next = b; b.next = a;
   a = null; b = null;
   // With no other live reference, the cycle is eligible for GC.
```

DRAW THE IDEA



Watch out: Setting one variable to null does not help if another root still retains the object.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can a static map create a memory leak?

A static map remains reachable as long as its defining class/loader does; entries without a bounded lifecycle retain objects even if no business user needs them.

Original source: Core Java Zero to Hero, page 98 · Q140 · Interview study edition

Q141 How do strong, soft, weak and phantom references differ?

20-SECOND ANSWER

Strong references retain objects; soft and weak references allow reclamation under different rules; phantom references support post-mortem tracking.

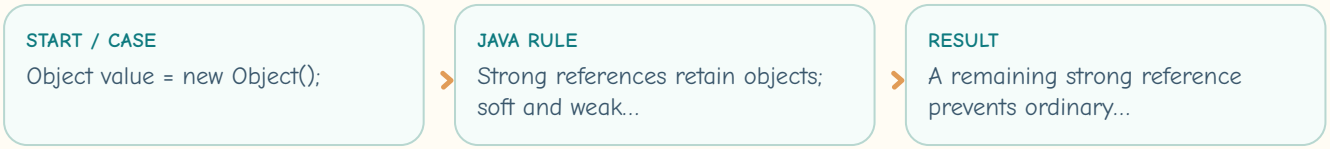
WHY IT WORKS · EASY EXPLANATION

A remaining strong reference prevents ordinary collection. WeakReference.get may later return null. Soft references are not a predictable bounded-cache policy, and phantom references expose no referent through get.

JAVA EXAMPLE / SCENARIO

```
Object value = new Object();
   WeakReference<Object> weak = new WeakReference<>(value);
   value = null;
   Object observed = weak.get();
   // observed may be non-null or null; timing is not guaranteed.
```

DRAW THE IDEA



Watch out: Weak references are not a dependable immediate cleanup schedule.

INTERVIEWER FOLLOW-UP - ANSWERED

Why should a production cache usually have an explicit capacity policy?

Capacity limits prevent unbounded retention, give predictable memory budgets and expose meaningful eviction behavior. Monitor size, hit ratio and eviction count.

Original source: Core Java Zero to Hero, page 98 · Q141 · Interview study edition

Q142 How do you choose between G1, ZGC and other collectors?

20-SECOND ANSWER

Choose a collector from measured latency, throughput, memory, and CPU goals under the supported JDK and workload.

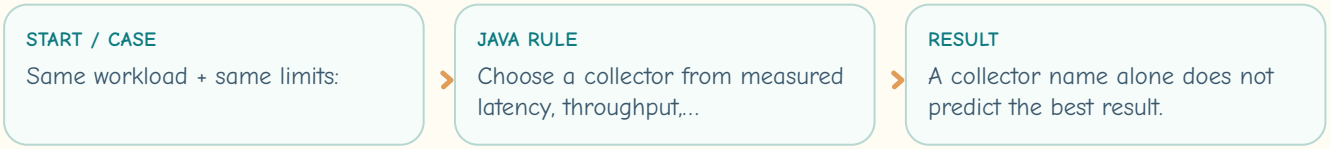
WHY IT WORKS · EASY EXPLANATION

A collector name alone does not predict the best result. Low pauses can come with different CPU or memory costs. Warm up and test realistic allocation and live-set behavior before deciding.

JAVA EXAMPLE / SCENARIO

```
Same workload + same limits:
   G1 run -> measure p99 pauses, throughput, heap use
   ZGC run -> measure p99 pauses, throughput, heap use
   Decision -> use the evidence matching service requirements
```

DRAW THE IDEA



Watch out: A pause target is a goal, not a hard real-time guarantee.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can allocation reduction matter more than changing the collector?

Short-lived allocation creates GC work. Removing unnecessary temporary objects or large payload copies can reduce pressure without collector-flag tradeoffs.

Original source: Core Java Zero to Hero, page 99 · Q142 · Interview study edition

Q143 How do you distinguish a leak from high allocation?

20-SECOND ANSWER

A leak retains objects beyond their useful lifetime; high allocation can cause pressure even when objects die promptly.

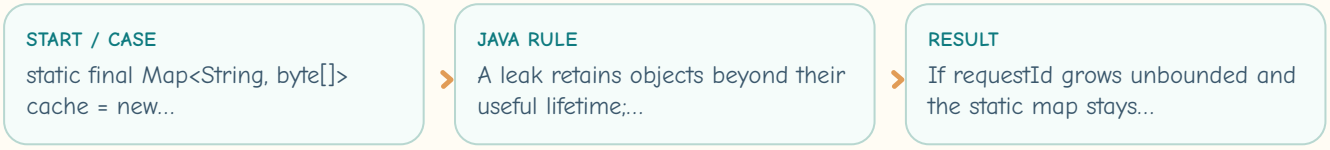
WHY IT WORKS · EASY EXPLANATION

If requestId grows unbounded and the static map stays reachable, entries remain live. Compare post-GC live-set trends and retention paths. A flat live set with frequent collection points more toward allocation pressure.

JAVA EXAMPLE / SCENARIO

```
static final Map<String, byte[]> cache = new HashMap<>();
   // Repeated unique keys without eviction keep growing live data:
   cache.put(requestId, new byte[1024]);
```

DRAW THE IDEA



Watch out: Heap used between collections is not by itself proof of a leak.

INTERVIEWER FOLLOW-UP - ANSWERED

What evidence would identify an unbounded queue retaining request payloads?

Inspect queue depth and oldest age, heap retained paths from queue nodes to request payloads, and whether producer rate exceeds consumer completion.

Original source: Core Java Zero to Hero, page 99 · Q143 · Interview study edition

Q144 How do you investigate CPU spikes, pauses and OutOfMemoryError?

20-SECOND ANSWER

Start JVM diagnosis from the symptom and evidence; OutOfMemoryError does not always mean the Java heap is too small.

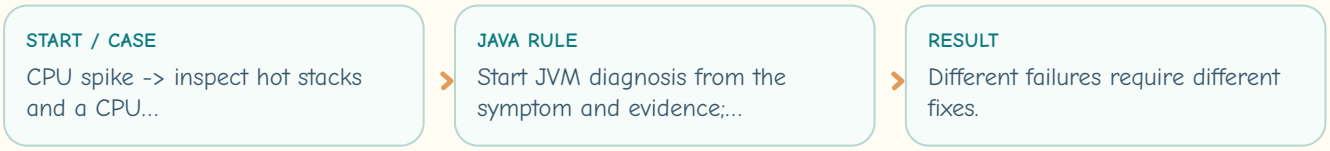
WHY IT WORKS · EASY EXPLANATION

Different failures require different fixes. Increasing -Xmx can worsen native-memory headroom. Preserve suitable thread dumps, GC logs, or memory diagnostics before changing one measured setting at a time.

JAVA EXAMPLE / SCENARIO

```
CPU spike -> inspect hot stacks and a CPU profile.
   Long pause -> correlate GC logs and safepoints.
   Heap OOME -> inspect retained objects and limits.
   Native-thread OOME -> inspect thread count and native headroom.
```

DRAW THE IDEA



Watch out: Blindly increasing -Xmx can reduce memory available for native allocations.

INTERVIEWER FOLLOW-UP - ANSWERED

Which measurements would distinguish stop-the-world pauses from slow downstream calls?

Compare GC pause timestamps with request traces, p99 latency and downstream timings. Thread dumps/JFR and CPU evidence distinguish other causes.

Original source: Core Java Zero to Hero, page 100 · Q144 · Interview study edition

MODULE 19 / 21

19

I/O, serialization, reflection and security

Byte streams handle raw bytes; readers and writers handle characters through an explicit encoding boundary.

8 interview questions · clear examples · follow-up answers

Q145 How do byte streams and character readers differ?

20-SECOND ANSWER

Byte streams handle raw bytes; readers and writers handle characters through an explicit encoding boundary.

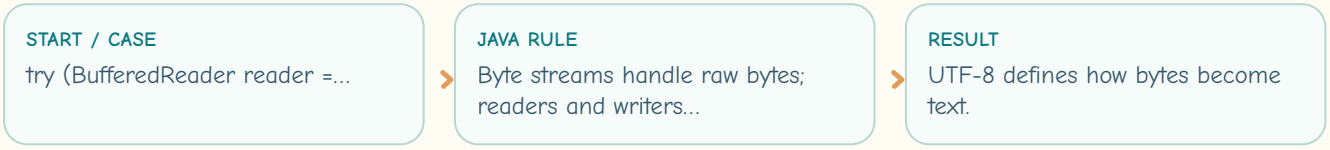
WHY IT WORKS · EASY EXPLANATION

UTF-8 defines how bytes become text. The buffered reader reduces small I/O operations and closes at the end of the block. Keep binary images or archives as bytes rather than decoding them as arbitrary text.

JAVA EXAMPLE / SCENARIO

```
try (BufferedReader reader = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
       System.out.println(reader.readLine());
   }
```

DRAW THE IDEA



Watch out: The same bytes can decode differently under different character encodings.

INTERVIEWER FOLLOW-UP - ANSWERED

How should an application handle malformed text sequences?

Choose and document a decoder policy: REPORT errors, REPLACE invalid sequences or reject input. Do not silently corrupt security-sensitive identifiers.

Original source: Core Java Zero to Hero, page 101 · Q145 · Interview study edition

Q146 How do you process a large file without loading it all?

20-SECOND ANSWER

Read large files incrementally and bound everything you retain, not just the input traversal.

WHY IT WORKS · EASY EXPLANATION

The pipeline counts lines without collecting the entire file. The stream owns a file resource and must close. An unbounded collected output, aggregation map, or work queue can still exhaust memory.

JAVA EXAMPLE / SCENARIO

```
try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) {
       long nonEmpty = lines.filter(s -> !s.isEmpty()).count();
       System.out.println(nonEmpty);
   }
```

DRAW THE IDEA

START / CASE
try (Stream<String> lines = Files.lines(path,...

JAVA RULE
Read large files incrementally and bound everything you...

RESULT
The pipeline counts lines without collecting the entire...

Watch out: Files.readAllLines retains all lines and is unsuitable for arbitrarily large input.

INTERVIEWER FOLLOW-UP - ANSWERED

When does a streaming word-count still require too much memory?

If it retains a frequency entry for every distinct token, the map can still grow without bound. Use bounded aggregation, spill-to-disk or database processing.

Original source: Core Java Zero to Hero, page 102 · Q146 · Interview study edition

Q147 What do Serializable, transient and serialVersionUID mean?

20-SECOND ANSWER

Serializable enables default object serialization; transient excludes default instance-field storage; serialVersionUID participates in compatibility checks.

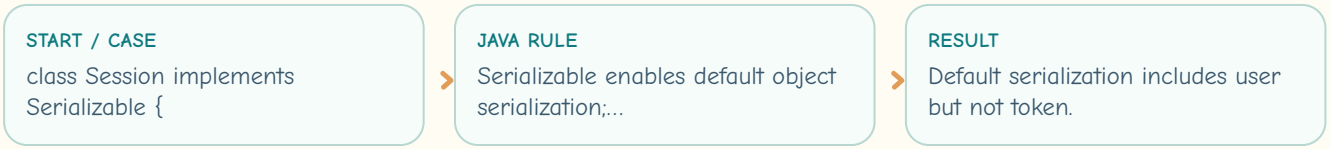
WHY IT WORKS · EASY EXPLANATION

Default serialization includes user but not token. After ordinary deserialization token receives its default null unless custom behavior supplies it. Static fields are class state, not serialized instance state. The UID is not a guarantee that all schema changes are safe.

JAVA EXAMPLE / SCENARIO

```
class Session implements Serializable {
       private static final long serialVersionUID = 1L;
       String user;
       transient String token;
   }
```

DRAW THE IDEA



Watch out: transient is not encryption or a complete protection for secrets.

INTERVIEWER FOLLOW-UP - ANSWERED

Why do static fields not belong to an individual object's serialized state?

Ordinary static fields belong to the class, not a particular instance, so default Java object serialization records instance state only.

Original source: Core Java Zero to Hero, page 102 · Q147 · Interview study edition

Q148 Why is deserializing untrusted Java objects dangerous?

20-SECOND ANSWER

Untrusted Java deserialization can execute dangerous class behavior and build excessive object graphs; prefer a controlled format and schema.

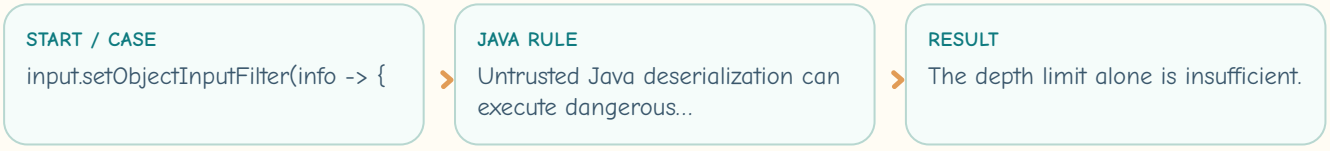
WHY IT WORKS · EASY EXPLANATION

The depth limit alone is insufficient. A full policy must constrain classes, array sizes, references, and other relevant limits. Post-read validation cannot undo behavior already executed during deserialization.

JAVA EXAMPLE / SCENARIO

```
// If serialization is unavoidable, filter before reading:
   input.setObjectInputFilter(info -> {
       if (info.depth() > 10) return ObjectInputFilter.Status.REJECTED;
       return ObjectInputFilter.Status.UNDECIDED;
   });
   // Partial illustration only: also apply a narrow class allow policy.
```

DRAW THE IDEA



Watch out: A broad package allow rule may still permit unsafe behavior inside permitted classes.

INTERVIEWER FOLLOW-UP - ANSWERED

Why must limits include graph size as well as type names?

Even allowed classes can form huge/deep graphs that exhaust resources. Limit bytes, depth, references and array sizes in addition to permitted types.

Original source: Core Java Zero to Hero, page 103 · Q148 · Interview study edition

Q149 How does deserialization interact with constructors?

20-SECOND ANSWER

Ordinary Serializable constructors are skipped during deserialization; validation must account for that path.

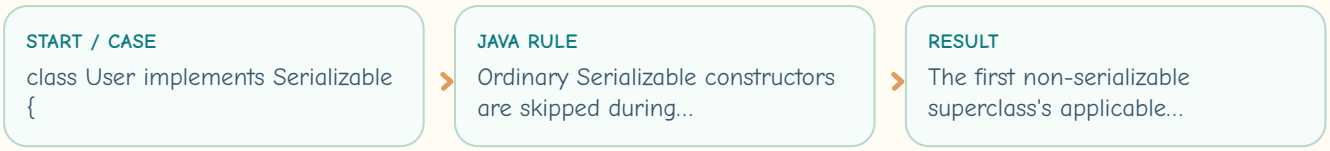
WHY IT WORKS · EASY EXPLANATION

The first non-serializable superclass's applicable no-argument constructor participates instead. Records have different rules using their canonical constructor. Use supported deserialization validation or a safer explicit data format.

JAVA EXAMPLE / SCENARIO

```
class User implements Serializable {
       final int age;
       User(int age) {
           if (age < 0) throw new IllegalArgumentException("age");
           this.age = age;
       }
   }
   // This constructor alone does not validate serialized input.
```

DRAW THE IDEA



Watch out: The constructor rules for Externalizable and records differ from ordinary Serializable classes.

INTERVIEWER FOLLOW-UP - ANSWERED

When would a serialization proxy keep value-object invariants clearer?

Use a serialization proxy when a value object needs one validated construction path and strict invariants despite historical serialization mechanisms.

Original source: Core Java Zero to Hero, page 103 · Q149 · Interview study edition

Q150 What is reflection and what are its costs?

20-SECOND ANSWER

Reflection discovers or invokes members at runtime; it trades compile-time certainty for dynamic flexibility.

WHY IT WORKS · EASY EXPLANATION

The method name and signature are resolved at runtime. Missing members or access failures become runtime errors, with module boundaries also affecting access. Cache suitable metadata and measure performance rather than assuming a fixed cost.

JAVA EXAMPLE / SCENARIO

```
Method method = String.class.getMethod("length");
   Object result = method.invoke("Java");
   System.out.println(result); // 4
```

DRAW THE IDEA

START / CASE
Method method =...

JAVA RULE
Reflection discovers or invokes members at runtime; it...

RESULT
The method name and signature are resolved at runtime.

Watch out: getDeclaredMethods includes declared methods, not a simple listing of all inherited public methods.

INTERVIEWER FOLLOW-UP - ANSWERED

How do getMethods and getDeclaredMethods differ?

getMethods returns accessible public methods including inherited ones; getDeclaredMethods returns methods declared directly in that class, including nonpublic members.

Original source: Core Java Zero to Hero, page 104 · Q150 · Interview study edition

Q151 How do annotations, reflection and dynamic proxies work together?

20-SECOND ANSWER

Runtime annotations supply metadata; reflection reads it; JDK proxies route interface calls through an InvocationHandler.

WHY IT WORKS · EASY EXPLANATION

This minimal run-only example illustrates dispatch, not a production handler. Real handlers need correct equals, hashCode, toString, delegation, and exception behavior. Annotation retention must be RUNTIME when reflection needs it.

JAVA EXAMPLE / SCENARIO

```
Runnable proxy = (Runnable) Proxy.newProxyInstance(
          Runnable.class.getClassLoader(), new Class<?>[]{Runnable.class},
          (p, method, args) -> {
              if (method.getName().equals("run")) System.out.println("Audit");
              return null;
          });
   proxy.run(); // Audit
```

DRAW THE IDEA

START / CASE
Runnable proxy = (Runnable)...

JAVA RULE
Runtime annotations supply metadata; reflection reads...

RESULT
This minimal run-only example illustrates dispatch, not...

Watch out: Self-invocation inside the target may bypass interception through the proxy.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can reflective invocation expose an InvocationTargetException wrapper?

Method.invoke wraps an exception thrown by the invoked method in InvocationTargetException. Inspect getCause() to understand the original failure.

Original source: Core Java Zero to Hero, page 104 · Q151 · Interview study edition

Q152 How would you prevent path traversal in a file API?

20-SECOND ANSWER

Reject paths escaping an allowed root and account for symlinks, real paths, and check/use races.

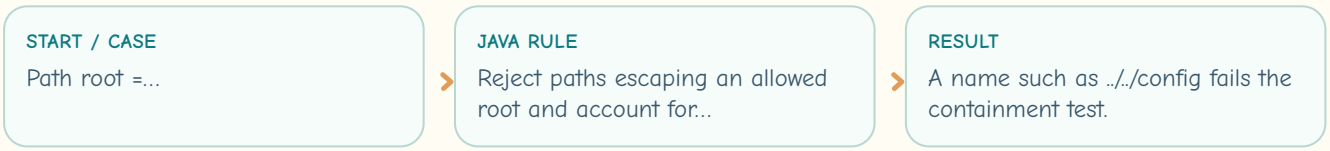
WHY IT WORKS · EASY EXPLANATION

A name such as ../../config fails the containment test. A symlink can still redirect a lexically contained path. Restrict directory ownership and names, and use appropriate secure filesystem handling for the threat model.

JAVA EXAMPLE / SCENARIO

```
Path root = Paths.get("uploads").toAbsolutePath().normalize();
   Path candidate = root.resolve(userName).normalize();
   if (!candidate.startsWith(root)) {
       throw new SecurityException("Outside upload area");
   }
   // Lexical check only: symlink-safe access requires more policy.
```

DRAW THE IDEA



Watch out: Canonical-path checks can still race with changes to the filesystem.

INTERVIEWER FOLLOW-UP - ANSWERED

Why might generated server-side filenames be safer than preserving arbitrary client paths?

Generated server paths avoid traversal, collisions and untrusted path separators. Store the original filename only as sanitized metadata when needed.

Original source: Core Java Zero to Hero, page 105 · Q152 · Interview study edition

MODULE 20 / 21

20

Modern Java for interviews

Know the feature's introduction version and compile for the deployment baseline rather than assuming every Java API exists.

8 interview questions · clear examples · follow-up answers

Q153 Which features should you know across Java 8, 17 and 21?

20-SECOND ANSWER

Know the feature's introduction version and compile for the deployment baseline rather than assuming every Java API exists.

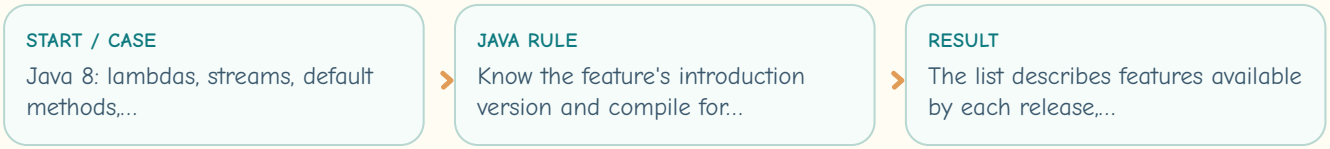
WHY IT WORKS · EASY EXPLANATION

The list describes features available by each release, not all introduced in that exact release. For example, records finalized in 16. --release checks both source language and the target platform APIs; Java 8 cannot compile records.

JAVA EXAMPLE / SCENARIO

```
Java 8:  lambdas, streams, default methods, java.time
   Java 17: records and sealed hierarchies available
   Java 21: virtual threads, record patterns, pattern switch,
            sequenced collections
   javac --release 8 Hello.java
```

DRAW THE IDEA



Watch out: Do not describe a preview feature as a universally enabled language feature.

INTERVIEWER FOLLOW-UP - ANSWERED

How would --release prevent accidental use of a newer platform API?

--release checks the target language level, standard library API availability and class-file version; -source/-target alone can still reference newer APIs.

Original source: Core Java Zero to Hero, page 106 · Q153 · Interview study edition

Q154 What do records generate and what do they not guarantee?

20-SECOND ANSWER

Records generate data-carrier members but provide only shallow immutability unless their components are controlled.

WHY IT WORKS · EASY EXPLANATION

The compact constructor copies the list structure before implicit field assignment. Records provide accessors, equals, hashCode, toString, and a canonical constructor. Mutable element objects would still need a separate copy policy.

JAVA EXAMPLE / SCENARIO

```
record Tags(List<String> values) {
       Tags { values = List.copyOf(values); }
   }
   List<String> source = new ArrayList<>(List.of("A"));
   Tags tags = new Tags(source);
   source.add("B");
   System.out.println(tags.values()); // [A]
```

DRAW THE IDEA

START / CASE
record Tags(List<String> values) {

JAVA RULE
Records generate data-carrier members but provide only...

RESULT
The compact constructor copies the list structure...

Watch out: A record containing a mutable list is not automatically deeply immutable.

INTERVIEWER FOLLOW-UP - ANSWERED

Can a record extend an arbitrary class?

No. A record implicitly extends java.lang.Record and cannot extend an arbitrary superclass. It may implement interfaces.

Original source: Core Java Zero to Hero, page 107 · Q154 · Interview study edition

Q155 What problem do sealed classes and interfaces solve?

20-SECOND ANSWER

Sealed types restrict the permitted direct subtypes so the domain's alternatives are explicit and controlled.

WHY IT WORKS · EASY EXPLANATION

The permitted alternatives describe the closed hierarchy. Records are final, satisfying the direct-subtype status rule. Ordinary direct subtypes must declare final, sealed, or non-sealed as appropriate.

JAVA EXAMPLE / SCENARIO

```
sealed interface Result permits Approved, Declined {}
   record Approved(String id) implements Result {}
   record Declined(String reason) implements Result {}
   // Java 17+: an unrelated class cannot implement Result.
```

DRAW THE IDEA

START / CASE
sealed interface Result permits Approved,...

JAVA RULE
Sealed types restrict the permitted direct subtypes so...

RESULT
The permitted alternatives describe the closed hierarchy.

Watch out: non-sealed intentionally reopens that branch for further extension.

INTERVIEWER FOLLOW-UP - ANSWERED

When is a sealed hierarchy clearer than an enum plus unrelated nullable fields?

A sealed hierarchy can encode distinct valid variants with their own fields, avoiding one enum discriminator and unrelated nullable fields.

Original source: Core Java Zero to Hero, page 107 · Q155 · Interview study edition

Q156 How do switch expressions and pattern switches differ?

20-SECOND ANSWER

A switch expression produces a value; a pattern switch can select by runtime type and needs deliberate null and exhaustiveness handling.

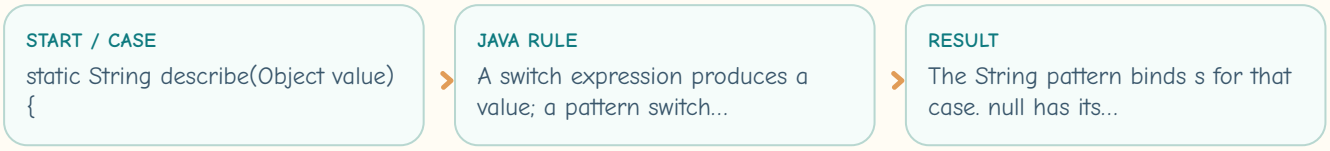
WHY IT WORKS · EASY EXPLANATION

The String pattern binds s for that case. null has its own case. Arrow branches avoid ordinary fall-through. For a block producing a switch-expression result, use yield.

JAVA EXAMPLE / SCENARIO

```
// Java 21:
   static String describe(Object value) {
       return switch (value) {
           case null -> "missing";
           case String s -> "text: " + s;
           default -> "other";
       };
   }
```

DRAW THE IDEA



Watch out: Placing a broad Object pattern before a narrower String pattern can make the narrower case dominated.

INTERVIEWER FOLLOW-UP - ANSWERED

When is case null needed?

Use case null when null is an allowed input and the switch needs an explicit outcome. Otherwise reject null according to the defined contract.

Original source: Core Java Zero to Hero, page 108 · Q156 · Interview study edition

Q157 What does var infer and where can it be used?

20-SECOND ANSWER

var infers a local variable's static type; it does not introduce dynamic typing.

WHY IT WORKS · EASY EXPLANATION

The compiler infers ArrayList<String> from the initializer. Later assignments must match that type. var is not available as an ordinary field type or method return type.

JAVA EXAMPLE / SCENARIO

```
var names = new ArrayList<String>(); // Java 10+
   names.add("Sai");
   // names = "text"; // Compile-time error
   // var missing = null; // Cannot infer a usable type
```

DRAW THE IDEA

START / CASE
var names = new ArrayList<String>(); // Java...

JAVA RULE
var infers a local variable's static type; it does not...

RESULT
The compiler infers ArrayList<String> from the...

Watch out: var value = null; cannot infer a usable local variable type.

INTERVIEWER FOLLOW-UP - ANSWERED

Why does var f = x -> x + 1 lack a target type?

A lambda needs a target functional-interface type to infer parameter and return types. var alone cannot infer the standalone lambda's functional interface.

Original source: Core Java Zero to Hero, page 108 · Q157 · Interview study edition

Q158 What are sequenced collections in Java 21?

20-SECOND ANSWER

Java 21 sequenced collections standardize first, last, and reversed-view operations for defined encounter order.

WHY IT WORKS · EASY EXPLANATION

reversed returns a reverse-ordered view, not automatically a detached copy. Defined encounter order is distinct from sorted order. Mutating operations depend on the implementation's supported contract.

JAVA EXAMPLE / SCENARIO

```
List<String> xs = new ArrayList<>(List.of("A", "B", "C"));
   System.out.println(xs.getFirst()); // A
   System.out.println(xs.getLast());  // C
   System.out.println(xs.reversed()); // [C, B, A]
```

DRAW THE IDEA

START / CASE
List<String> xs = new...

JAVA RULE
Java 21 sequenced collections standardize first, last,...

RESULT
reversed returns a reverse-ordered view, not...

Watch out: A reversed view is not necessarily an independent copy.

INTERVIEWER FOLLOW-UP - ANSWERED

How does encounter order differ from sorted order?

Encounter order is the source's defined traversal order; sorted order is an explicit comparator/ natural ordering applied by sorted(). A stream can have either, both or neither.

Original source: Core Java Zero to Hero, page 109 · Q158 · Interview study edition

Q159 What are virtual threads and when do they help?

20-SECOND ANSWER

Virtual threads make large numbers of mostly blocking tasks cheaper; they do not add CPU or downstream capacity.

WHY IT WORKS · EASY EXPLANATION

The executor starts a virtual thread for each task. For real I/O, independently bound scarce resources such as database connections. CPU-heavy work remains limited by processor capacity.

JAVA EXAMPLE / SCENARIO

```
// Java 21:
   try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
       Future<String> f = executor.submit(() -> "done");
       System.out.println(f.get()); // done
   }
```

DRAW THE IDEA

START / CASE
try (ExecutorService executor =...

JAVA RULE
Virtual threads make large numbers of mostly blocking...

RESULT
The executor starts a virtual thread for each task.

Watch out: Do not pool virtual threads to model a scarce database connection budget.

INTERVIEWER FOLLOW-UP - ANSWERED

How would a Semaphore bound 20 concurrent calls while allowing many virtual threads?

Acquire one of 20 permits before each database call and release it in finally. Many virtual threads can wait cheaply, but the database still sees at most 20 calls.

Original source: Core Java Zero to Hero, page 109 · Q159 · Interview study edition

Q160 What is virtual-thread pinning and why does the Java version matter?

20-SECOND ANSWER

Pinning is version-sensitive: Java 21 monitor-held blocking can tie up carriers; JDK 24 removes monitor-related pinning.

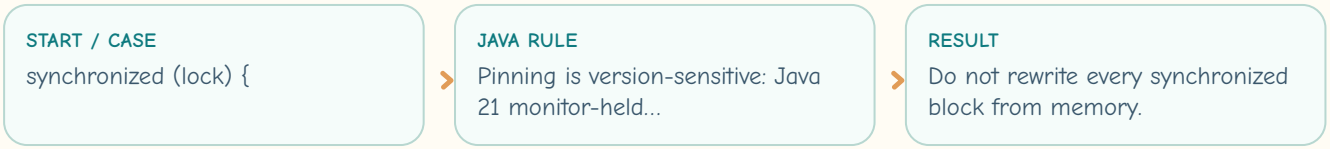
WHY IT WORKS · EASY EXPLANATION

Do not rewrite every synchronized block from memory. Profile the actual JDK and workload. The JDK 24 change does not eliminate native-interaction cases or lock contention and downstream capacity limits.

JAVA EXAMPLE / SCENARIO

```
synchronized (lock) {
       blockingCall();
   }
   // Java 21: this can pin a virtual thread's carrier.
   // JDK 24+: JEP 491 removes monitor-related pinning.
```

DRAW THE IDEA



Watch out: Changing to virtual threads does not remove downstream concurrency or memory limits.

INTERVIEWER FOLLOW-UP - ANSWERED

What evidence would distinguish pinning from a slow connection pool?

Compare JFR virtual-thread pinning events and carrier-thread occupancy with connection-pool active/wait metrics, call latency and the JVM release's pinning behavior.

Original source: Core Java Zero to Hero, page 110 · Q160 · Interview study edition

MODULE 21 / 21

21

Senior interview scenarios

Fix lost increments by making the update atomic; volatile visibility alone is insufficient.

8 interview questions · clear examples · follow-up answers

Q161 A service loses increments although count is volatile. How do you fix it?

20-SECOND ANSWER

Fix lost increments by making the update atomic; volatile visibility alone is insufficient.

WHY IT WORKS · EASY EXPLANATION

Two count++ operations can read the same old value and lose one write. Atomic increments preserve each contribution. Check the final value only after all workers complete; choose LongAdder only when its concurrent-sum semantics fit.

JAVA EXAMPLE / SCENARIO

```
AtomicInteger count = new AtomicInteger();
   // Each worker repeats:
   count.incrementAndGet();
   // After joining 100 workers doing 1,000 increments each:
   // count.get() must equal 100000.
```

DRAW THE IDEA



Watch out: Adding another volatile variable does not make the update atomic.

INTERVIEWER FOLLOW-UP - ANSWERED

Would the same fix work for a balance and ledger invariant?

Yes, but the lock or atomic design must protect both balance and ledger together; an atomic counter alone cannot guarantee they represent one transaction.

Original source: Core Java Zero to Hero, page 111 · Q161 · Interview study edition

Q162 A map contains a customer but get(customer) returns null. What do you inspect?

20-SECOND ANSWER

Check changed key fields, equality policy, and concurrent access when a present map entry cannot be retrieved.

WHY IT WORKS · EASY EXPLANATION

Changing a stored Customer's id can invalidate lookup under its original hash. Immutable CustomerId separates stable identity from mutable profile data. Rebuild affected mappings from trusted entries or records during migration.

JAVA EXAMPLE / SCENARIO

```
// Before: mutable Customer acts as a map key.
   // After: use stable identity.
   record CustomerId(String tenant, long id) {} // Java 16+
   Map<CustomerId, Customer> customers = new HashMap<>();
```

DRAW THE IDEA

START / CASE
record CustomerId(String tenant, long id) {}...

JAVA RULE
Check changed key fields, equality policy, and...

RESULT
Changing a stored Customer's id can invalidate lookup...

Watch out: Increasing HashMap capacity does not repair an equality contract.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you migrate keys without losing existing entries?

Use a planned migration: build immutable new keys, insert into a new map, resolve collisions, switch readers atomically, then retire the old map.

Original source: Core Java Zero to Hero, page 112 · Q162 · Interview study edition

Q163 An executor queue grows while CPU stays low. What does that suggest?

20-SECOND ANSWER

A growing queue with low CPU often means workers wait on I/O, locks, or resource pools rather than compute.

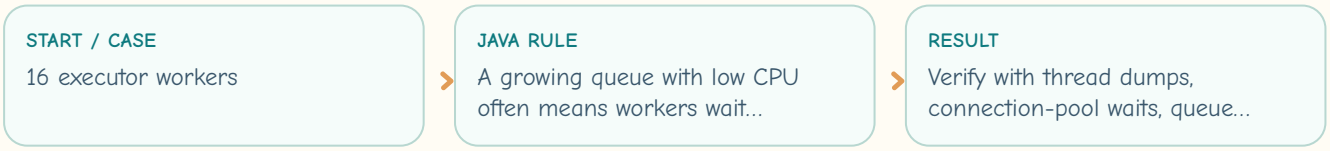
WHY IT WORKS · EASY EXPLANATION

Verify with thread dumps, connection-pool waits, queue age, and dependency latency. More workers may only create more waiting. Bound admitted work and align concurrency and deadlines with the bottleneck.

JAVA EXAMPLE / SCENARIO

```
16 executor workers
     -> 2 database connections
     -> 14 workers may wait for a connection
     -> queued requests accumulate while CPU stays low
```

DRAW THE IDEA



Watch out: Low CPU is not evidence that unlimited extra work can be admitted.

INTERVIEWER FOLLOW-UP - ANSWERED

Which request should fail first when its remaining deadline is already too short?

Reject or fail fast any request whose remaining deadline cannot cover the minimum meaningful downstream work; do not enqueue already-expired work.

Original source: Core Java Zero to Hero, page 112 · Q163 · Interview study edition

Q164 A parallel stream makes an endpoint slower. What would you change?

20-SECOND ANSWER

Measure parallel-stream overhead and contention; choose sequential processing or explicitly bounded I/O execution when appropriate.

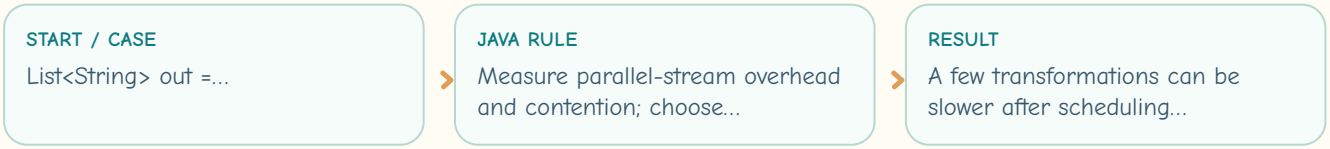
WHY IT WORKS · EASY EXPLANATION

A few transformations can be slower after scheduling and merge overhead. Blocking calls can occupy shared workers. Compare representative sequential and parallel runs before changing the design.

JAVA EXAMPLE / SCENARIO

```
// For a small in-memory transform:
   List<String> out = names.stream().map(String::trim)
           .collect(Collectors.toList());
   // For remote calls, design deadlines and admission separately.
```

DRAW THE IDEA



Watch out: A lower average latency alone can hide a worse p99 and cross-request interference.

INTERVIEWER FOLLOW-UP - ANSWERED

What would a meaningful benchmark include besides the isolated function?

Include representative input distribution, warmup, JIT/GC effects, allocations, concurrency, latency percentiles, error rate and real resource limits.

Original source: Core Java Zero to Hero, page 113 · Q164 · Interview study edition

Q165 An API returns an unmodifiable list, but its data keeps changing. Why?

20-SECOND ANSWER

An unmodifiable result may be a live view or contain mutable elements; define whether the API promises a snapshot.

WHY IT WORKS · EASY EXPLANATION

The caller cannot modify view, but another owner can change its backing list. Copy the structure and, when required, convert mutable elements into immutable values. Document snapshot versus live-view behavior.

JAVA EXAMPLE / SCENARIO

```
List<String> source = new ArrayList<>(Arrays.asList("A"));
   List<String> view = Collections.unmodifiableList(source);
   source.add("B");
   System.out.println(view); // [A, B]
   List<String> snapshot = List.copyOf(source); // Java 10+
```

DRAW THE IDEA

START / CASE
List<String> source = new...

JAVA RULE
An unmodifiable result may be a live view or contain...

RESULT
The caller cannot modify view, but another owner can...

Watch out: A structural snapshot is not a recursive deep copy.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you document a deliberately live read-only view?

State that the returned object is a live backed view, which changes with the source, and document thread safety, ownership, mutation restrictions and expiration.

Original source: Core Java Zero to Hero, page 113 · Q165 · Interview study edition

Q166 A retry charges a customer twice after a future timeout. What failed?

20-SECOND ANSWER

A timeout means the charge outcome may be unknown; retries need a stable idempotency key or reconciliation protocol.

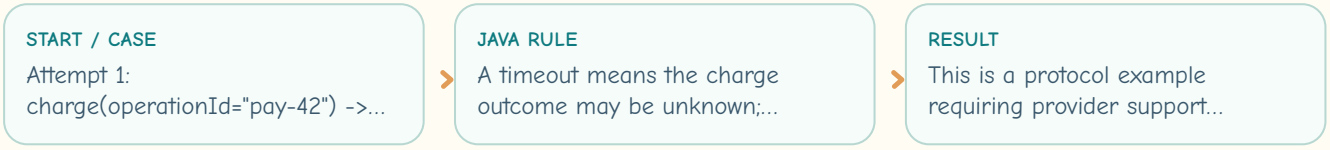
WHY IT WORKS · EASY EXPLANATION

This is a protocol example requiring provider support and durable handling. A fresh key on each retry can create two charges. Future timeout or cancellation cannot undo an external payment already accepted.

JAVA EXAMPLE / SCENARIO

```
Attempt 1: charge(operationId="pay-42") -> caller times out
   Attempt 2: charge(operationId="pay-42") -> same operation
   Provider policy: repeated key returns the existing outcome
```

DRAW THE IDEA



Watch out: CompletableFuture.orTimeout does not cancel or undo an external side effect.

INTERVIEWER FOLLOW-UP - ANSWERED

What should the user see when the outcome is still unknown?

Show a pending or unknown state with a stable operation ID and status-check path; do not report success or failure before reconciliation resolves it.

Original source: Core Java Zero to Hero, page 114 · Q166 · Interview study edition

Q167 ThreadLocal request data leaks between pooled tasks. How do you repair it?

20-SECOND ANSWER

Remove owned ThreadLocal request context in finally because pooled threads survive across requests.

WHY IT WORKS · EASY EXPLANATION

Without removal, the next task on the same worker may see stale authentication or tracing data. Async work may execute on another thread and needs an explicit propagation policy. Nested context ownership may require restoring an earlier value.

JAVA EXAMPLE / SCENARIO

```
try {
       requestContext.set(request);
       handleRequest();
   } finally {
       requestContext.remove();
   }
```

DRAW THE IDEA

START / CASE
try {

JAVA RULE
Remove owned ThreadLocal request context in finally...

RESULT
Without removal, the next task on the same worker may...

Watch out: Setting a null value is different from removing the ThreadLocal entry.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can a ThreadLocal also retain a large object graph?

A long-lived pool thread may retain a ThreadLocal value and everything it references. Always remove request-scoped values in finally.

Original source: Core Java Zero to Hero, page 114 · Q167 · Interview study edition

Q168 A TreeSet drops one of two distinct employees. What went wrong?

20-SECOND ANSWER

TreeSet treats comparator zero as duplicate membership; add identity tie breakers when equal sort fields should remain distinct.

WHY IT WORKS · EASY EXPLANATION

A salary-only comparator merges two different employees earning the same salary. The stable unique id distinguishes them. Define null behavior and keep ordering consistent with the required business identity.

JAVA EXAMPLE / SCENARIO

```
Comparator<Employee> order = Comparator.comparing(Employee::getSalary)
          .thenComparing(Employee::getId);
   Set<Employee> employees = new TreeSet<>(order);
```

DRAW THE IDEA

START / CASE
Comparator<Employee> order =...

JAVA RULE
TreeSet treats comparator zero as duplicate membership;...

RESULT
A salary-only comparator merges two different employees...

Watch out: Replacing equals alone does not change a TreeSet's comparison policy.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you represent several employees at each salary?

Group by salary into Map<Long,List<Employee>> or use a comparator with a unique tie-breaker so equal salaries do not collapse distinct people.

Original source: Core Java Zero to Hero, page 115 · Q168 · Interview study edition

Q169 A large CSV import exhausts heap despite Files.lines. Why?

20-SECOND ANSWER

Streaming a file does not bound retained results, queued work, error reports, or aggregation maps.

WHY IT WORKS · EASY EXPLANATION

collect(toList) would retain every line. An incremental sink avoids that specific issue, but its own buffers must be bounded. Use capped error samples and a clear checkpoint/partial-failure policy for a real import.

JAVA EXAMPLE / SCENARIO

```
try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) {
       lines.forEach(this::writeProcessedRow);
   }
   // Keep output incremental and in-flight batches bounded.
```

DRAW THE IDEA

START / CASE
try (Stream<String> lines = Files.lines(path,...

JAVA RULE
Streaming a file does not bound retained results,...

RESULT
collect(toList) would retain every line.

Watch out: Moving readAllLines to Files.lines is insufficient if the terminal stage still retains all lines.

INTERVIEWER FOLLOW-UP - ANSWERED

When is an external sort or database aggregation needed?

When distinct keys/rows exceed memory or datasets are very large. Delegate to indexed SQL/ database aggregation or external merge sort with bounded memory.

Original source: Core Java Zero to Hero, page 115 · Q169 · Interview study edition

Q170 A singleton is sometimes partly initialized. How do you reason about it?

20-SECOND ANSWER

A partly initialized singleton indicates unsafe publication or this escaping; class-initialization publication is a simple repair.

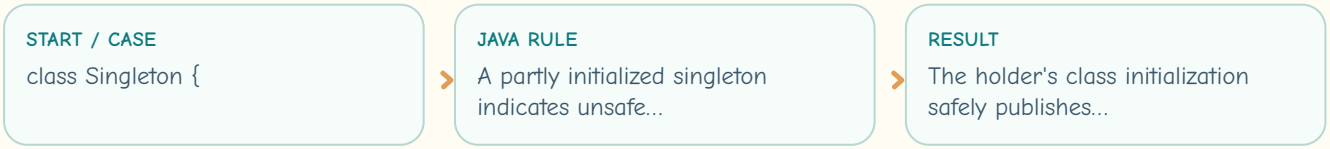
WHY IT WORKS · EASY EXPLANATION

The holder's class initialization safely publishes INSTANCE. Check constructors for callbacks that expose this early. Separate class loaders can still hold separate singleton instances, and mutable singleton methods need synchronization.

JAVA EXAMPLE / SCENARIO

```
class Singleton {
       private Singleton() {}
       private static class Holder {
           static final Singleton INSTANCE = new Singleton();
       }
       static Singleton get() { return Holder.INSTANCE; }
   }
```

DRAW THE IDEA



Watch out: A successful constructor in one thread does not justify an unsynchronized publication pattern.

INTERVIEWER FOLLOW-UP - ANSWERED

Why can a singleton be unique only within a class-loader context?

Static initialization happens per defining class loader. Two class loaders can define the same class name and each initialize a separate singleton.

Original source: Core Java Zero to Hero, page 116 · Q170 · Interview study edition

Q171 A result list changes order after parallelization. What contract matters?

20-SECOND ANSWER

Define source encounter order and terminal-operation ordering before promising a parallel result order.

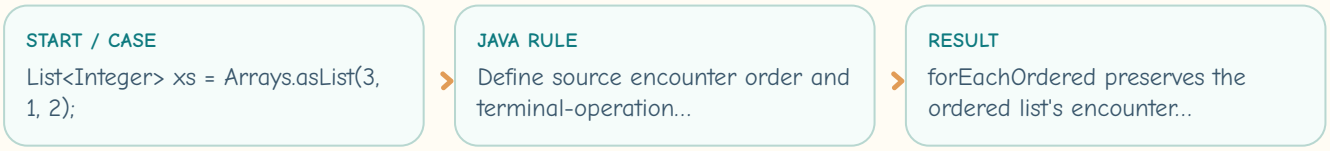
WHY IT WORKS · EASY EXPLANATION

forEachOrdered preserves the ordered list's encounter order. If the required business order is numeric, sort first. An unordered source cannot supply a meaningful 'first' merely by using an ordered terminal operation.

JAVA EXAMPLE / SCENARIO

```
List<Integer> xs = Arrays.asList(3, 1, 2);
   xs.parallelStream().forEachOrdered(System.out::println);
   // Prints 3, then 1, then 2.
   // forEach alone does not promise this action order.
```

DRAW THE IDEA



Watch out: Observed HashMap iteration order is not a durable API promise.

INTERVIEWER FOLLOW-UP - ANSWERED

What ordering should apply to equal timestamps?

Use a secondary stable key, such as unique ID, to make ordering deterministic; timestamps alone may tie across records.

Original source: Core Java Zero to Hero, page 116 · Q171 · Interview study edition

Q172 A service cannot shut down because workers ignore interruption. What next?

20-SECOND ANSWER

Cooperative shutdown requires workers to honor cancellation and blocking operations to have an unblocking policy.

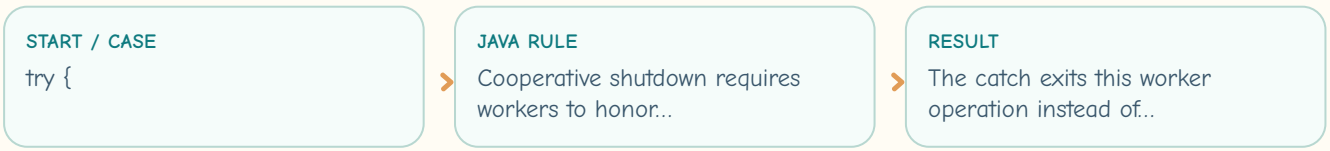
WHY IT WORKS · EASY EXPLANATION

The catch exits this worker operation instead of swallowing interruption and looping forever. Investigate non-interruptible I/O separately, with deadlines or closing task-owned resources. Accepted durable work needs recovery semantics.

JAVA EXAMPLE / SCENARIO

```
try {
       while (!Thread.currentThread().isInterrupted()) {
           process(queue.take());
       }
   } catch (InterruptedException e) {
       Thread.currentThread().interrupt();
   }
```

DRAW THE IDEA



Watch out: shutdownNow is an attempt, not a guaranteed stop.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you test shutdown during a blocked network call?

Inject a controllable blocking dependency, initiate shutdown, enforce a deadline and verify interruption, resource closure, task termination and remaining queued-work policy.

Original source: Core Java Zero to Hero, page 117 · Q172 · Interview study edition

Q173 An immutable record exposes a mutable list. How do you harden it?

20-SECOND ANSWER

A record with a mutable list is only shallowly immutable; defensively copy the structure and control element mutability.

WHY IT WORKS · EASY EXPLANATION

The constructor isolates future structural changes to input. String values are immutable in this example. Real mutable Item objects would need copying or conversion; List.copyOf also rejects null elements.

JAVA EXAMPLE / SCENARIO

```
record Order(List<String> items) {
       Order { items = List.copyOf(items); }
   }
   List<String> input = new ArrayList<>(List.of("Book"));
   Order order = new Order(input);
   input.clear();
   System.out.println(order.items()); // [Book]
```

DRAW THE IDEA

START / CASE
record Order(List<String> items) {

JAVA RULE
A record with a mutable list is only shallowly...

RESULT
The constructor isolates future structural changes to...

Watch out: List.copyOf rejects null elements and does not recursively copy items.

INTERVIEWER FOLLOW-UP - ANSWERED

What should a constructor do with an empty order?

Validate the business contract: reject an empty order if at least one item is required; otherwise support an explicit empty state. Never leave a partially constructed invalid object.

Original source: Core Java Zero to Hero, page 117 · Q173 · Interview study edition

Q174 A cache loads the same key multiple times under load. How do you design coalescing?

20-SECOND ANSWER

Coalesce cache misses by atomically reserving an in-flight future that concurrent callers can share.

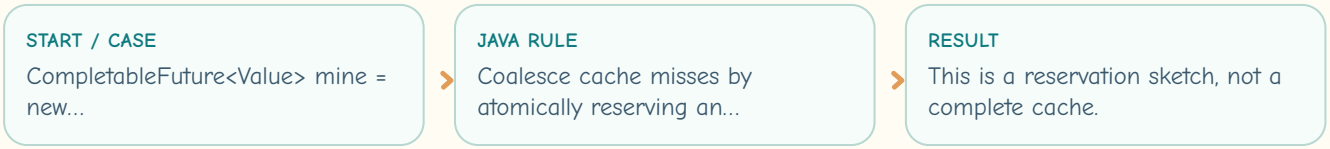
WHY IT WORKS · EASY EXPLANATION

This is a reservation sketch, not a complete cache. The owner must complete exceptionally on failure and clean up safely. Distinguish completed-value caching from in-flight ownership, and prevent one caller cancelling shared work for everyone.

JAVA EXAMPLE / SCENARIO

```
CompletableFuture<Value> mine = new CompletableFuture<>();
   CompletableFuture<Value> existing = inFlight.putIfAbsent(key, mine);
   if (existing != null) return existing;
   // Owner loads outside map mutation, completes mine,
   // then removes with inFlight.remove(key, mine).
```

DRAW THE IDEA



Watch out: Removing an entry must compare the expected in-flight value so it does not remove a newer replacement.

INTERVIEWER FOLLOW-UP - ANSWERED

What happens if invalidation arrives while a load is finishing?

Treat invalidation as a version or generation change. A finishing old load must not overwrite newer state; compare versions or cancel/discard stale completion.

Original source: Core Java Zero to Hero, page 118 · Q174 · Interview study edition

Q175 A latency spike appears after increasing -Xmx. Why could that happen?

20-SECOND ANSWER

A larger heap can change GC behavior and reduce native-memory headroom inside a fixed container.

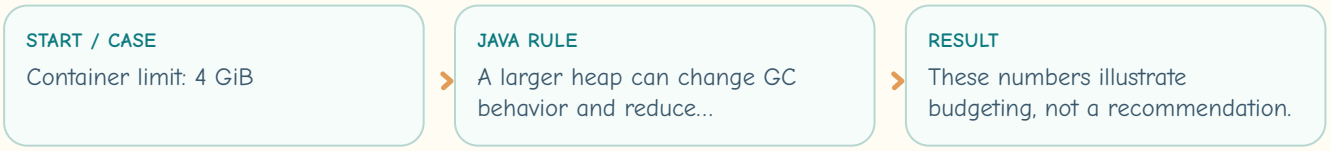
WHY IT WORKS · EASY EXPLANATION

These numbers illustrate budgeting, not a recommendation. Raising heap further can squeeze non-heap resources. Correlate GC pauses, resident memory, swapping, CPU limits, and native allocations before attributing the latency spike.

JAVA EXAMPLE / SCENARIO

```
Container limit: 4 GiB
   Heap budget:     3 GiB
   Remaining:      1 GiB for stacks, metaspace, buffers,
                   JVM internals, and other native memory
```

DRAW THE IDEA



Watch out: Heap size alone does not explain all memory or latency problems.

INTERVIEWER FOLLOW-UP - ANSWERED

Which container-level measurements would you correlate with GC logs?

Correlate heap usage and pauses with process RSS, container memory limits, OOM-kill events, CPU throttling and native-memory indicators.

Original source: Core Java Zero to Hero, page 118 · Q175 · Interview study edition

Q176 A date-time test passes locally and fails on the server. What do you inspect?

20-SECOND ANSWER

Make clock, zone, locale, and other environment defaults explicit when time-dependent tests differ across machines.

WHY IT WORKS · EASY EXPLANATION

The fixed clock makes both instant and zone reproducible. Inject it into business logic instead of calling now with implicit defaults. Include daylight-saving gap and overlap cases for relevant zones.

JAVA EXAMPLE / SCENARIO

```
Clock clock = Clock.fixed(Instant.parse("2026-10-08T03:30:00Z"),
                          ZoneId.of("Asia/Kolkata"));
   System.out.println(LocalDateTime.now(clock)); // 2026-10-08T09:00
```

DRAW THE IDEA

START / CASE
Clock clock =...

JAVA RULE
Make clock, zone, locale, and other environment...

RESULT
The fixed clock makes both instant and zone reproducible.

Watch out: LocalDateTime without a zone does not identify a unique global timestamp.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you represent a recurring appointment in the customer's local zone?

Store the customer's ZoneId plus local recurring calendar rule. Resolve each occurrence using the zone's DST gap/overlap policy before converting to an Instant.

Original source: Core Java Zero to Hero, page 119 · Q176 · Interview study edition

Q177 A comparator occasionally violates its contract. How do you debug it?

20-SECOND ANSWER

Debug comparator sign symmetry, transitivity, and zero equivalence; subtraction overflow is a common defect.

WHY IT WORKS · EASY EXPLANATION

The subtraction wraps and says the larger value is smaller. Use standard compare methods and stable fields with explicit tie breakers. A passing sort does not prove that a comparator satisfies its contract.

JAVA EXAMPLE / SCENARIO

```
int a = Integer.MAX_VALUE, b = -1;
   System.out.println(a - b);             // -2147483648: wrong sign
   System.out.println(Integer.compare(a, b)); // 1: correct order
```

DRAW THE IDEA

START / CASE
int a = Integer.MAX_VALUE, b = -1;

JAVA RULE
Debug comparator sign symmetry, transitivity, and zero...

RESULT
The subtraction wraps and says the larger value is...

Watch out: A comparator depending on current time can change while sorting.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you generate three values that challenge transitivity?

Choose values a,b,c that make compare(a,b)>0, compare(b,c)>0, but compare(a,c)<=0; also check antisymmetry and equality consistency using generated triples.

Original source: Core Java Zero to Hero, page 119 · Q177 · Interview study edition

Q178 A catch block converts every failure to an empty result. Why is that risky?

20-SECOND ANSWER

Do not represent failed data retrieval as legitimate absence; the two outcomes can drive different business decisions.

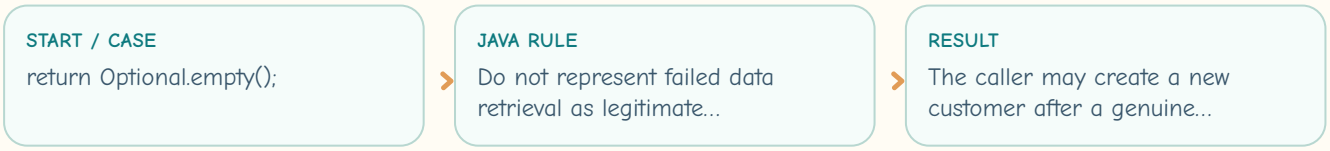
WHY IT WORKS · EASY EXPLANATION

The caller may create a new customer after a genuine absence. Doing so during a database outage would be wrong. Choose explicit fallback only when the contract permits it, preserving cause and degraded-operation metrics.

JAVA EXAMPLE / SCENARIO

```
// Valid absence:
   return Optional.empty();
   // Retrieval failure, at the appropriate boundary:
   // throw new CustomerLookupException("Database unavailable", cause);
```

DRAW THE IDEA



Watch out: An empty collection is not a universal safe fallback.

INTERVIEWER FOLLOW-UP - ANSWERED

How would the API distinguish no credit history from a failed credit-history lookup?

Return distinct states or errors: NOT_FOUND/NO_HISTORY versus DEPENDENCY_FAILED/UNKNOWN. An empty list must not hide an upstream outage.

Original source: Core Java Zero to Hero, page 120 · Q178 · Interview study edition

Q179 Virtual threads raise request count but database throughput stays flat. Why?

20-SECOND ANSWER

Virtual threads increase affordable waiting concurrency, while database throughput remains constrained by connections and query capacity.

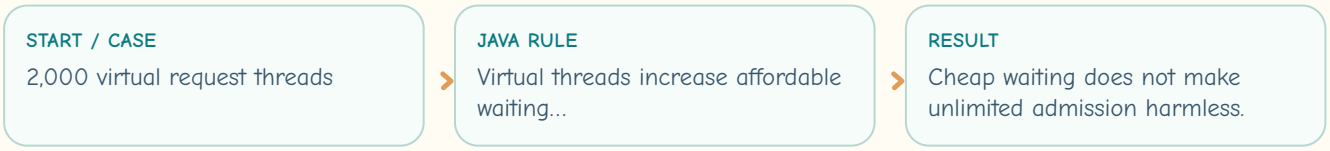
WHY IT WORKS · EASY EXPLANATION

Cheap waiting does not make unlimited admission harmless. Measure connection wait, query latency, database utilization, and p99 response time. Bound scarce-resource access and tune the actual bottleneck.

JAVA EXAMPLE / SCENARIO

```
2,000 virtual request threads
   20 database connections
   At most 20 connection-owning operations at a time
   Extra requests wait or must be rejected within a deadline
```

DRAW THE IDEA



Watch out: Concurrency capacity and CPU or database throughput are different limits.

INTERVIEWER FOLLOW-UP - ANSWERED

How would you compare Java 21 pinning with the same workload on JDK 24+?

Run the same representative workload under Java 21 and JDK 24+ with equal limits and dependencies. Record JFR pinning/carrier use, pool waits, CPU, p99 and throughput.

Original source: Core Java Zero to Hero, page 120 · Q179 · Interview study edition

Q180 How would you review a concurrent class before approving it?

20-SECOND ANSWER

Review shared state through invariants, access policies, publication, compound operations, resource bounds, and lifecycle paths.

WHY IT WORKS · EASY EXPLANATION

Individually thread-safe fields do not guarantee one correct transfer. State the atomic business operation and the evidence for its policy. Targeted stress tests can reveal violations but cannot prove that no race exists.

JAVA EXAMPLE / SCENARIO

```
Transfer invariant: total balance is preserved.
   Shared state: source balance, target balance, transfer record.
   Policy: coordinate the complete update, with consistent locks.
   Check: failure, cancellation, timeout, and recovery branches.
```

DRAW THE IDEA

START / CASE
Transfer invariant: total balance is preserved.

JAVA RULE
Review shared state through invariants, access...

RESULT
Individually thread-safe fields do not guarantee one...

Watch out: Thread-safe fields do not automatically make the entire business operation atomic.

INTERVIEWER FOLLOW-UP - ANSWERED

Which invariant could a test observe to detect an invalid intermediate state?

For a transfer, repeatedly assert that both accounts and the ledger agree and the total balance remains constant at every observable committed snapshot.

Original source: Core Java Zero to Hero, page 121 · Q180 · Interview study edition

PRACTICE WORKBOOK

24

Java 8 Coding Laboratory

Complete methods, worked results, complexity and failure boundaries. Write your own version first - then compare it with these source-faithful reference methods.

24 labs · runnable source · 24 smoke checks

C01

Reverse Unicode code points

> Reverse the code points in a non-null string. State whether this preserves displayed grapheme clusters.

APPROACH - EASY EXPLANATION

Convert to code points, then append them in reverse order. This preserves surrogate pairs. Combining marks and joined emoji need a separate grapheme-aware requirement.

JAVA SOLUTION (REFERENCE METHOD)

```
static String reverseCodePoints(String s) {
    Objects.requireNonNull(s);
    int[] cp = s.codePoints().toArray();
    StringBuilder out = new StringBuilder();
    for (int i = cp.length - 1; i >= 0; i--) {
        out.appendCodePoint(cp[i]);
    }
    return out.toString();
}
```

Worked result: Input: A + U+1F600 + B. Output: B + U+1F600 + A.

Time and space: O(n) time and O(n) extra space, measured in code points.

Edge-case policy: Empty input returns empty. Null rejects. Code-point reversal is not grapheme reversal.

Original guide coding lab: C01 · source page 123

C02

Palindrome with an explicit normalization policy

> Check a palindrome ignoring non-alphanumeric code points and case. Use Locale.ROOT for case conversion.

APPROACH - EASY EXPLANATION

Normalize case, retain letters/digits, and compare symmetric code-point positions. This is a stated technical policy, not universal human-language equivalence.

JAVA SOLUTION (REFERENCE METHOD)

```
static boolean palindrome(String s) {
    int[] cp = Objects.requireNonNull(s)
        .toLowerCase(Locale.ROOT).codePoints()
        .filter(Character::isLetterOrDigit).toArray();
    for (int l = 0, r = cp.length - 1; l < r; l++, r--)
        if (cp[l] != cp[r]) return false;
    return true;
}
```

Worked result: "A man, a plan, a canal: Panama" returns true; "Java" returns false.

Time and space: O(n) time and O(n) normalized storage.

Edge-case policy: Punctuation-only input returns true under this policy. Unicode normalization and case folding are separate choices.

Original guide coding lab: C02 · source page 124

C03

First non-repeating code point

> Return the first code point occurring exactly once, or OptionalInt.empty when none exists.

APPROACH - EASY EXPLANATION

Count in a LinkedHashMap to preserve first encounter order, then select the first count of one. Keep supplementary characters intact.

JAVA SOLUTION (REFERENCE METHOD)

```
static OptionalInt firstUnique(String s) {
    Map<Integer, Integer> counts = new LinkedHashMap<>();
    s.codePoints().forEach(cp -> counts.merge(cp, 1, Integer::sum));
    return counts.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .mapToInt(Map.Entry::getKey).findFirst();
}
```

Worked result: "swiss" returns code point w; "aabb" returns empty.

Time and space: O(n + u) time, O(u) space for u distinct code points.

Edge-case policy: The result is a code point, not necessarily one Java char. Empty input returns empty.

Original guide coding lab: C03 · source page 125

C04

Anagram by code-point counts

> Check whether two strings contain the same multiset of code points. Here case and punctuation are significant.

APPROACH - EASY EXPLANATION

Compare total code-point length and frequency maps. If the business asks for normalized anagrams, normalize both inputs using one declared policy first.

JAVA SOLUTION (REFERENCE METHOD)

```
static Map<Integer, Long> cpCounts(String s) {
    return s.codePoints().boxed().collect(Collectors.groupingBy(
        Function.identity(), Collectors.counting()));
}
static boolean anagram(String a, String b) {
    return cpCounts(a).equals(cpCounts(b));
}
```

Worked result: "listen" and "silent" return true; "Listen" and "silent" return false.

Time and space: O(n + m) expected time and O(u) space.

Edge-case policy: Different normalization forms can look identical but have different code-point counts.

Original guide coding lab: C04 · source page 126

C05

Balanced brackets with a stack

> Check (), [] and {} nesting. Reject every other character under this input contract.

APPROACH - EASY EXPLANATION

Push each opening bracket. For each closing bracket, compare with the most recent opening bracket. At the end the stack must be empty.

JAVA SOLUTION (REFERENCE METHOD)

```
static boolean balanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') stack.push(c);
        else {
            char expected;
            if (c == ')') expected = '(';
            else if (c == ']') expected = '[';
            else if (c == '}') expected = '{';
            else return false;
            if (stack.isEmpty() || stack.pop() != expected) return false;
        }
    }
    return stack.isEmpty();
}
```

Worked result: "{[()]}" returns true; "([)]" returns false.

Time and space: O(n) time and O(n) worst-case stack space.

Edge-case policy: Empty input is balanced. A leading closing bracket fails without popping an empty stack.

Original guide coding lab: C05 · source page 127

C06

Binary search without midpoint overflow

> Find a target index in a sorted ascending int array, or return -1.

APPROACH - EASY EXPLANATION

Maintain an inclusive search range. Each comparison discards half the range. The input must already satisfy the sorted-order precondition.

JAVA SOLUTION (REFERENCE METHOD)

```
static int binarySearch(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Worked result: [1,3,5,7], target 5 gives index 2; target 6 gives -1.

Time and space: O(log n) time, O(1) extra space.

Edge-case policy: Empty array returns -1. For duplicates, this version returns any matching index.

Original guide coding lab: C06 · source page 128

C07

Two sum with overflow-safe complements

> Return two different indices whose integer values sum to a target, or an empty array.

APPROACH - EASY EXPLANATION

For each value, search earlier values for a complement. Calculate the complement using long and only cast when it fits int, avoiding a false match caused by overflow.

JAVA SOLUTION (REFERENCE METHOD)

```
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        long need = (long) target - a[i];
        if (need >= Integer.MIN_VALUE && need <= Integer.MAX_VALUE) {
            Integer j = seen.get((int) need);
            if (j != null) return new int[]{j, i};
        }
        seen.putIfAbsent(a[i], i);
    }
    return new int[0];
}
```

Worked result: [2,7,11,15], target 9 gives [0,1].

Time and space: O(n) expected time and O(n) space.

Edge-case policy: Repeated values are valid at different indices. No solution returns an empty array.

Original guide coding lab: C07 · source page 129

C08

Top-k frequent words with deterministic ties

> Return up to k words ordered by descending count and then ascending word. Reject negative k.

APPROACH - EASY EXPLANATION

Aggregate counts, then sort distinct entries with an explicit tie breaker. This clear baseline costs O(u log u); a bounded heap improves very large u when k is small.

JAVA SOLUTION (REFERENCE METHOD)

```
static List<String> topWords(List<String> words, int k) {
    if (k < 0) throw new IllegalArgumentException("negative k");
    Map<String, Long> freq = words.stream().collect(
        Collectors.groupingBy(Function.identity(), Collectors.counting()));
    return freq.entrySet().stream().sorted((a, b) -> {
        int c = Long.compare(b.getValue(), a.getValue());
        return c != 0 ? c : a.getKey().compareTo(b.getKey());
    }).limit(k).map(Map.Entry::getKey).collect(Collectors.toList());
}
```

Worked result: [b,a,b,a,c], k=2 gives [a,b].

Time and space: O(n + u log u) time, O(u) space before output.

Edge-case policy: k=0 returns empty; k greater than distinct count returns all; null words are excluded by the input contract.

Original guide coding lab: C08 · source page 130

C09

Kth-largest value using a bounded heap

> Return the kth-largest element, including duplicate occurrences, where 1 <= k <= n.

APPROACH - EASY EXPLANATION

Maintain a min-heap containing the largest k values seen so far. Remove the smallest whenever the heap exceeds k. Its head is the kth-largest value at completion.

JAVA SOLUTION (REFERENCE METHOD)

```
static int kthLargest(int[] a, int k) {
    if (k < 1 || k > a.length) throw new IllegalArgumentException("k");
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Worked result: [3,2,1,5,6,4], k=2 gives 5; [5,5,4], k=2 gives 5.

Time and space: O(n log(k+1)) time, O(k) heap space.

Edge-case policy: This is occurrence rank, not distinct rank. Invalid k rejects.

Original guide coding lab: C09 · source page 131

C10

Merge overlapping closed intervals

> Merge closed intervals [start,end], including intervals touching at an endpoint. Preserve the caller arrays.

APPROACH - EASY EXPLANATION

Validate and clone each interval, sort by start, then scan. Extend the current range when the next start is no greater than the current end.

JAVA SOLUTION (REFERENCE METHOD)

```
static List<int[]> mergeIntervals(int[][] input) {
    int[][] a = new int[input.length][];
    for (int i = 0; i < input.length; i++) {
        if (input[i].length != 2 || input[i][0] > input[i][1])
            throw new IllegalArgumentException("interval");
        a[i] = input[i].clone();
    }
    Arrays.sort(a, Comparator.comparingInt(x -> x[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] x : a) {
        if (out.isEmpty() || out.get(out.size()-1)[1] < x[0]) out.add(x);
        else {
            int[] last = out.get(out.size()-1);
            last[1] = Math.max(last[1], x[1]);
        }
    }
    return out;
}
```

Worked result: [[1,3],[2,6],[8,10],[10,12]] becomes [[1,6],[8,12]].

Time and space: O(n log n) time, O(n) copied input/output space.

Edge-case policy: Empty input returns empty. Closed and half-open intervals need different touching rules.

Original guide coding lab: C10 · source page 132

C11

LRU eviction in an access-order map

> Implement a positive-capacity LRU map for single-threaded use. Reading an entry refreshes its recency.

APPROACH - EASY EXPLANATION

Use LinkedHashMap access order and evict the eldest entry after insertion exceeds capacity. Declare the concurrency and feature limits explicitly.

JAVA SOLUTION (REFERENCE METHOD)

```
static final class Lru<K,V> extends LinkedHashMap<K,V> {
    private final int capacity;
    Lru(int capacity) {
        super(16, 0.75f, true);
        if (capacity < 1) throw new IllegalArgumentException("capacity");
        this.capacity = capacity;
    }
    @Override protected boolean removeEldestEntry(Map.Entry<K,V> e) {
        return size() > capacity;
    }
}
```

Worked result: Capacity 2: put A, put B, get A, put C evicts B.

Time and space: Expected O(1) basic operations, O(capacity) stored entries.

Edge-case policy: No TTL, loading or concurrency guarantees. A get changes access order.

Original guide coding lab: C11 · source page 133

C12

Immutable order with a defensive list copy

> Create an order exposing a list of immutable String item names. Validate both the list and each element.

APPROACH - EASY EXPLANATION

Copy input structure before wrapping it, reject null elements, and expose only the unmodifiable copy. A Java 10+ List.copyOf alternative is shorter; this solution keeps Java 8 compatibility.

JAVA SOLUTION (REFERENCE METHOD)

```
static final class Order {
    private final List<String> items;
    Order(List<String> input) {
        List<String> copy = new ArrayList<>(Objects.requireNonNull(input));
        copy.forEach(Objects::requireNonNull);
        this.items = Collections.unmodifiableList(copy);
    }
    List<String> items() { return items; }
}
```

Worked result: Adding to the original input after construction does not change order.items().

Time and space: O(n) construction time and space; O(1) list reference access.

Edge-case policy: String elements are immutable. A mutable Item element type would require an additional ownership decision.

Original guide coding lab: C12 · source page 134

C13

Group employees and sum salaries

> Return total integer salary units by department, using a sorted map. Employee salaries are long values.

APPROACH - EASY EXPLANATION

Use groupingBy with a TreeMap supplier and summingLong downstream. State overflow policy: this baseline assumes totals fit long; exact financial accumulation may use BigDecimal or checked arithmetic.

JAVA SOLUTION (REFERENCE METHOD)

```
static final class Employee {
    final String dept; final long salary;
    Employee(String dept, long salary) {
        this.dept = Objects.requireNonNull(dept); this.salary = salary;
    }
}
static Map<String, Long> totals(List<Employee> employees) {
    return employees.stream().collect(Collectors.groupingBy(
        e -> e.dept, TreeMap::new, Collectors.summingLong(e -> e.salary)));
}
```

Worked result: IT: 100 and 150, HR: 80 gives {HR=80, IT=250}.

Time and space: O(n log d) time with TreeMap, O(d) result space.

Edge-case policy: Empty input returns empty. Long addition can overflow; reject or use a wider decimal contract if needed.

Original guide coding lab: C13 · source page 135

C14

Second-highest distinct integer salary

> Return the second-highest distinct long value or OptionalLong.empty.

APPROACH - EASY EXPLANATION

Distinct numeric values, sort, then select the second-to-last. This educational stream solution is concise; a two-best one-pass algorithm avoids sorting for very large input.

JAVA SOLUTION (REFERENCE METHOD)

```
static OptionalLong secondHighest(long[] salaries) {
    long[] sorted = Arrays.stream(salaries).distinct().sorted().toArray();
    return sorted.length < 2 ? OptionalLong.empty()
        : OptionalLong.of(sorted[sorted.length - 2]);
}
```

Worked result: [100,200,200,150] gives 150; [200,200] gives empty.

Time and space: O(n log n) time in the sorting baseline and O(n) worst-case space.

Edge-case policy: Negative values work. Duplicates do not consume a distinct rank.

Original guide coding lab: C14 · source page 136

C15

Stable deduplication

> Remove duplicate strings while preserving the first encounter order.

APPROACH - EASY EXPLANATION

Use a LinkedHashSet and copy the result into an ordinary ArrayList. This policy uses String.equals and permits null as one distinct value.

JAVA SOLUTION (REFERENCE METHOD)

```
static List<String> stableDistinct(List<String> input) {
    return new ArrayList<>(new LinkedHashSet<>(input));
}
```

Worked result: [b,a,b,c,a] becomes [b,a,c].

Time and space: O(n) expected time and O(u) stored distinct values.

Edge-case policy: The input stays unchanged. A custom business key needs a separate deduplication policy.

Original guide coding lab: C15 · source page 137

C16

Partition valid and invalid records

> Partition strings into valid non-blank values and invalid values. Null is invalid.

APPROACH - EASY EXPLANATION

Use partitioningBy with a predicate that guards null before trim. This simple contract treats Java trim whitespace only; Unicode whitespace rules may require a different policy.

JAVA SOLUTION (REFERENCE METHOD)

```
static Map<Boolean,List<String>> partition(List<String> rows) {
    return rows.stream().collect(Collectors.partitioningBy(
        s -> s != null && !s.trim().isEmpty()));
}
```

Worked result: [a, blank, null, b] gives true=[a,b] and false=[blank,null].

Time and space: O(total inspected characters + n) time, O(n) partition storage.

Edge-case policy: Both partitions exist even for empty input. This separates classification from error explanations.

Original guide coding lab: C16 · source page 138

C17

Word-frequency stream pipeline

> Count lowercased words separated by non-letter/non-digit characters, preserving alphabetical result order.

APPROACH - EASY EXPLANATION

Normalize with Locale.ROOT, tokenize with a Unicode letter/digit regex, discard empty tokens and aggregate using a TreeMap. Declare apostrophe and punctuation behavior.

JAVA SOLUTION (REFERENCE METHOD)

```
static Map<String,Long> wordCounts(String text) {
    return Arrays.stream(text.toLowerCase(Locale.ROOT)
        .split("[^\\p{L}\\p{N}]+"))
        .filter(s -> !s.isEmpty()).collect(Collectors.groupingBy(
            Function.identity(), TreeMap::new, Collectors.counting()));
}
```

Worked result: "Java, java! Streams" gives {java=2, streams=1}.

Time and space: O(n + w log u) approximate time, O(n + u) token/result storage.

Edge-case policy: Empty text returns an empty map. Apostrophes split words under this policy.

Original guide coding lab: C17 · source page 139

C18

Merge duplicate map keys with checked addition

> Convert entries into a map, summing duplicate long values and rejecting overflow.

APPROACH - EASY EXPLANATION

Supply an explicit merge function to toMap. Math.addExact states that overflow is an error, rather than silently wrapping. Use a sorted map when deterministic key order matters.

JAVA SOLUTION (REFERENCE METHOD)

```
static Map<String,Long> mergeEntries(List<Map.Entry<String,Long>> xs) {
    return xs.stream().collect(Collectors.toMap(
        Map.Entry::getKey, Map.Entry::getValue,
        (a,b) -> Math.addExact(a,b), TreeMap::new));
}
```

Worked result: (a,2), (a,3), (b,4) gives {a=5, b=4}.

Time and space: O(n log u) time and O(u) result space.

Edge-case policy: Null keys/values are outside this input contract. Overflow throws ArithmeticException.

Original guide coding lab: C18 · source page 140

C19

Bounded producer-consumer handoff

> Send integers 1 through n from one producer to one consumer, then stop. Return their sum with a five-second completion bound.

APPROACH - EASY EXPLANATION

Use a bounded queue and a sentinel that cannot be a valid item. Await both task outcomes. Cancellation and shutdown happen in finally. The protocol here has exactly one producer and one consumer.

JAVA SOLUTION (REFERENCE METHOD)

```
static long queueSum(int n) throws Exception {
    if (n < 0) throw new IllegalArgumentException("n");
    BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(4);
    ExecutorService pool = Executors.newFixedThreadPool(2);
    try {
        Future<?> producer = pool.submit(() -> {
            try {
                for (int i = 1; i <= n; i++) queue.put(i);
                queue.put(-1);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); throw new RuntimeException(e);
            }
        });
        Future<Long> consumer = pool.submit(() -> {
            long sum = 0; int x;
            while ((x = queue.take()) != -1) sum += x;
            return sum;
        });
        long result = consumer.get(5, TimeUnit.SECONDS);
        producer.get(5, TimeUnit.SECONDS);
        return result;
    } finally { pool.shutdownNow(); }
}
```

Worked result: n=5 gives 15; n=0 gives 0.

Time and space: O(n) work, O(capacity) queue space. Waiting depends on scheduling.

Edge-case policy: The sentinel cannot collide with positive items. More consumers need an expanded termination protocol.

Original guide coding lab: C19 · source page 141

C20

Exact counter under concurrent increments

> Run a chosen number of tasks, each incrementing one exact shared counter a chosen number of times.

APPROACH - EASY EXPLANATION

Use AtomicLong and wait for every Future before reading the result. Keep the test workload bounded; in a long-lived production service use an explicit admission policy rather than this exercise factory.

JAVA SOLUTION (REFERENCE METHOD)

```
static long concurrentCount(int tasks, int increments) throws Exception {
    if (tasks < 0 || increments < 0) throw new IllegalArgumentException();
    AtomicLong counter = new AtomicLong();
    ExecutorService pool = Executors.newFixedThreadPool(4);
    try {
        List<Future<?>> work = new ArrayList<>();
        for (int t = 0; t < tasks; t++) work.add(pool.submit(() -> {
            for (int i = 0; i < increments; i++) counter.incrementAndGet();
        }));
        for (Future<?> f : work) f.get(5, TimeUnit.SECONDS);
        return counter.get();
    } finally { pool.shutdownNow(); }
}
```

Worked result: 20 tasks with 1,000 increments gives exactly 20,000.

Time and space: O(tasks * increments) atomic updates, O(tasks) future storage.

Edge-case policy: Waiting for tasks is necessary; submission alone does not mean completion. Counter overflow policy is separate.

Original guide coding lab: C20 · source page 142

C21

Decimal sum with one final rounding rule

> Sum decimal prices exactly, then round the total to scale 2 using HALF_EVEN.

APPROACH - EASY EXPLANATION

Parse decimal strings, reduce with BigDecimal.ZERO and apply the business rounding rule once at the end. State when line-level rounding would be a different required policy.

JAVA SOLUTION (REFERENCE METHOD)

```
static BigDecimal roundedTotal(List<String> prices) {
    return prices.stream().map(BigDecimal::new)
        .reduce(BigDecimal.ZERO, BigDecimal::add)
        .setScale(2, RoundingMode.HALF_EVEN);
}
```

Worked result: [0.105,0.105] gives 0.21 when rounding only the final total.

Time and space: O(n) additions under a fixed precision bound; real cost grows with operand precision.

Edge-case policy: Empty input yields 0.00. Invalid decimal strings reject. Final rounding differs from rounding each line.

Original guide coding lab: C21 · source page 143

C22

Limit simultaneous work using a semaphore

> Execute tasks while admitting at most two into a protected section. Report the maximum observed concurrency.

APPROACH - EASY EXPLANATION

Use permits to bound entry, count entrants with atomics and release only after successful acquire. A start latch creates simultaneous demand. The test measures the invariant, not throughput.

JAVA SOLUTION (REFERENCE METHOD)

```
static int boundedConcurrency() throws Exception {
    Semaphore permits = new Semaphore(2);
    AtomicInteger active = new AtomicInteger(), peak = new AtomicInteger();
    CountDownLatch start = new CountDownLatch(1);
    ExecutorService pool = Executors.newFixedThreadPool(6);
    List<Future<?>> work = new ArrayList<>();
    try {
        for (int i = 0; i < 6; i++) work.add(pool.submit(() -> {
            start.await(); permits.acquire();
            try {
                int now = active.incrementAndGet();
                peak.accumulateAndGet(now, Math::max);
                try { Thread.sleep(10); }
                finally { active.decrementAndGet(); }
            } finally { permits.release(); }
            return null;
        }));
        start.countDown();
        for (Future<?> f : work) f.get(5, TimeUnit.SECONDS);
        return peak.get();
    } finally { pool.shutdownNow(); }
}
```

Worked result: The peak is between 1 and 2 after all six tasks complete.

Time and space: O(tasks) coordination work, O(tasks) futures; admission is capped at two.

Edge-case policy: A concurrency cap does not impose a requests-per-second rate. Release belongs after acquire succeeds.

Original guide coding lab: C22 · source page 144

C23

Combine independent futures

> Fetch two independent integer results, combine them and bound the caller wait.

APPROACH - EASY EXPLANATION

Use thenCombine rather than blocking one worker waiting for another. This Java 8-compatible example uses timed get to bound waiting. It does not claim that the timeout itself stops underlying work.

JAVA SOLUTION (REFERENCE METHOD)

```
static int combineFutures() throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(2);
    try {
        CompletableFuture<Integer> a = CompletableFuture.supplyAsync(() -> 20, pool);
        CompletableFuture<Integer> b = CompletableFuture.supplyAsync(() -> 22, pool);
        return a.thenCombine(b, Integer::sum).get(2, TimeUnit.SECONDS);
    } finally { pool.shutdownNow(); }
}
```

Worked result: Independent results 20 and 22 combine to 42.

Time and space: O(1) combination work here; real latency depends on task work and scheduling.

Edge-case policy: Failures propagate through the combined stage. Java 9+ orTimeout is a result deadline, not guaranteed task cancellation.

Original guide coding lab: C23 · source page 145

C24

Count lines incrementally with cleanup

> Count lines containing a non-empty search token in a UTF-8 file. Match case-sensitively.

APPROACH - EASY EXPLANATION

Open Files.lines in try-with-resources, filter by the explicit contains policy and count. This terminal operation does not retain all lines. A line can still be large, and I/O failures need a boundary policy.

JAVA SOLUTION (REFERENCE METHOD)

```
static long matchingLines(Path path, String token) throws IOException {
    if (token == null || token.isEmpty()) throw new IllegalArgumentException("token");
    try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) {
        return lines.filter(line -> line.contains(token)).count();
    }
}
```

Worked result: Lines [Java, java, Java streams], token Java gives 2.

Time and space: O(total file characters) time and streaming working memory proportional to current line/ buffering.

Edge-case policy: Empty files return 0. Missing files fail. Errors during lazy traversal can surface as UncheckedIOException.

Original guide coding lab: C24 · source page 146

YOUR DAILY PLAN

30 days to interview-ready

Use the sequence and topics from the original guide. Aim for a daily reading block, code practice, a spoken answer and a short error log.

Day	Focus	Study	Practice
1	Execution and types	Q001-Q008	Hello.java, overflow, boxing
2	Objects and contracts	Q009-Q016	Composition and abstract classes
3	Equality and immutability	Q017-Q024	C12 and mutable keys
4	Text and numbers	Q025-Q032	C01-C04, C21
5	Exceptions and cleanup	Q033-Q040	Suppressed exceptions
6	Generics and metadata	Q041-Q048	PECS and annotations
7	Weekly mock 1	Q001-Q048	45-minute spoken mock
8	Lists, sets and queues	Q049-Q056	C05, C06, C15
9	Map internals	Q057-Q064	C07, C11
10	Concurrent collections	Q065-Q072	C19
11	Lambdas and Optional	Q073-Q080	Lazy fallback and capture
12	Streams	Q081-Q088	C13, C14, C16
13	Collectors and time	Q089-Q096	C08, C17, C18
14	Weekly mock 2	Q049-Q096	Coding and output prediction
15	Thread behavior	Q097-Q104	Interruption, wait, join
16	Memory model	Q105-Q112	C20 and safe publication
17	Locks and coordination	Q113-Q120	C22 and lock ordering
18	Executors	Q121-Q128	Queues and shutdown
19	Future composition	Q129-Q136	C23 and recovery
20	JVM and diagnostics	Q137-Q144	Thread dumps and retention
21	Weekly mock 3	Q097-Q144	Concurrency diagnosis
22	I/O and reflection	Q145-Q152	C24 and cleanup
23	Modern Java	Q153-Q160	Records and sealed classes
24	Algorithm practice 1	C05-C10	Timed pattern coding

Day	Focus	Study	Practice
25	Algorithm practice 2	C01-C04, C11-C12	Text, LRU, immutable state
26	Stream practice	C13-C18, C21	Duplicates and overflow
27	Concurrency practice	C19-C24	Termination and bounds
28	Senior scenarios	Q161-Q180	Evidence, invariant, trade-off
29	Full mock interview	Mock A + B	Record answers and repair gaps
30	Final revision	Error log	Repeat missed problems

QUICK FINAL REVISION

Interview readiness checklist

Foundation: You can explain value vs reference, primitive conversion, class initialization, constructors, overriding, equality and immutable classes without guessing.

Collections and streams: You can choose lists, sets and maps; explain HashMap lookup and mutable-key failures; reason about lazy pipelines, ordering and reductions.

Concurrency: You can identify the shared invariant, distinguish visibility from atomicity, name the happens-before edge, bound queues and own executor shutdown.

JVM and modern Java: You can tell heap leaks from native memory, explain class loading and GC, and name which release is required for modern Java syntax and APIs.

Coding: You can implement the 24 laboratory tasks from a blank editor, test edge cases, explain Big-O and say whether your implementation mutates the input.

How to use this commercial-study edition: The original question wording and coding methods are preserved from the supplied PDF. Follow-up answers, design and simplified study presentation are added for learning. Confirm you have rights to redistribute the source content before selling.